

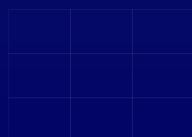
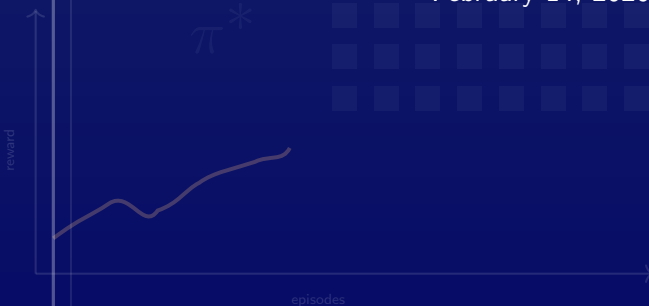
Aligning Intelligence

A Complete Journey from Reinforcement Learning to Human-Compatible AI



Dr.Milan Amrutkumar Joshi
Ph.D (Math.)

February 14, 2026



Dedicated to my beloved father

Late Shri Amrutkumar Joshi

and my mother

Smt. Jayashri Joshi

*For their constant support, unwavering belief,
and quiet inspiration through every chapter of my life.*

*You taught me that the greatest reward
is not found at the end of the journey,
but in the courage to begin one.*

Preface

The Gap That Shouldn't Exist

In March 2016, DeepMind's AlphaGo defeated the world champion at Go — a game so complex that brute-force search was considered impossible — using reinforcement learning. In November 2022, ChatGPT became the fastest-growing application in human history, reaching 100 million users in two months — powered by Reinforcement Learning from Human Feedback (RLHF). In January 2025, DeepSeek-R1 demonstrated that a language model could develop sophisticated mathematical reasoning purely through RL with verifiable rewards, without any human preference data at all.

Three landmarks. One thread. **Reinforcement learning.**

And yet, a remarkable gap persists in how this knowledge is transmitted. Classic RL textbooks — brilliant as they are — stop at tabular methods and function approximation. They were written before RLHF, before DPO, before GRPO, before language models became the most consequential AI systems on Earth. Practical coding tutorials rush past theory to reach implementations, leaving readers unable to debug, adapt, or reason about what they've built. And the explosive RLHF/LLM-alignment space? It lives entirely in blog posts, scattered arXiv preprints, conference workshops, and tribal knowledge passed between researchers on Twitter threads and Discord servers.

No single resource takes a reader from “What is a Markov Decision Process?” all the way to “How do we align a billion-parameter language model with human values?”

This book closes that gap. Permanently.

Five Parts, One Unbroken Thread

Aligning Intelligence: A Complete Journey from Reinforcement Learning to Human-Compatible AI is organized as five interlocking parts, each building on the last. The thread is never broken:

- **Part I: The Bedrock** (Chapters 0–8) lays the mathematical foundations: agents, environments, MDPs, Bellman equations, dynamic programming, Monte Carlo methods, temporal-difference learning, function approximation, and the first glimpse of deep Q-networks and policy gradients. If you have never studied RL, you start here.

- **Part II: The Deep RL Revolution** (Chapters 9–13) takes you from tabular methods into the deep RL era: advanced DQN variants (Double, Dueling, Rainbow), full policy gradient theory, actor-critic architectures, and the all-important Proximal Policy Optimization (PPO) — the algorithm that powers RLHF.
- **Part III: Teaching Machines What Humans Want** (Chapters 14–20) is where RL meets large language models. You will build the complete RLHF pipeline from scratch: supervised fine-tuning, reward modeling from human preferences, PPO for LLM alignment, Direct Preference Optimization (DPO), and rigorous evaluation of aligned models.
- **Part IV: When Humans Can't Keep Up** (Chapters 21–25) goes beyond RLHF into Constitutional AI, Reinforcement Learning from AI Feedback (RLAIF), scalable oversight, and adversarial red teaming — the techniques that Anthropic, OpenAI, and DeepMind use when human feedback alone isn't enough.
- **Part V: Machines That Learn to Reason** (Chapters 26–30) covers the research frontier of 2025–2026: Group Relative Policy Optimization (GRPO), Reinforcement Learning with Verifiable Rewards (RLVR), large reasoning models like DeepSeek-R1, multi-modal alignment, and the open problems that will define the next decade of AI safety.

Every chapter in *The Bedrock* builds toward *The Deep RL Revolution*. Every chapter in *The Deep RL Revolution* builds toward *Teaching Machines What Humans Want*. And when you reach RLHF in Part III, you will not be memorizing a recipe — you will *understand*, down to the gradient, *why* each component exists and what happens when it breaks.

The Pedagogy: Intuition → Math → Code

Every concept in this book is introduced three ways:

1. **Intuition** — plain-language explanation with visual analogies and diagrams, so you build the right mental model before seeing a single equation.
2. **Mathematics** — rigorous formulation with full derivations, so you can reason about correctness, debug failures, and extend ideas to new settings.

3. **Code** — working Python implementations you can run immediately, because understanding is incomplete until you’ve watched an algorithm learn.

Six distinctive pedagogical features run throughout every chapter:

- **Running Example (DeliBot):** A delivery robot that grows in complexity alongside the theory — from a simple MDP in Chapter 1 to a full policy-gradient agent by the final chapters.
- **Think Like the Agent:** Perspective-shifting exercises that place you inside the agent’s decision loop, building intuition that equations alone cannot provide.
- **Common Misconceptions:** Red-flagged boxes that preemptively address the confusions that trip up students and practitioners — because we’ve seen them happen hundreds of times.
- **Worked Traces:** Step-by-step arithmetic walkthroughs of key algorithms, showing every intermediate computation.
- **Derivation Roadmaps:** Summary tables that compress multi-step derivations into “what we did and why” overviews.
- **Bridge Boxes:** Color-coded callout boxes that connect foundational concepts to their future roles — **purple** bridges to RLHF (Part III), **blue** bridges to Deep RL (Part II), **green** bridges to Scalable Alignment (Part IV), and **orange** bridges to the Frontier (Part V) — so you never lose sight of where the journey is heading.

Who Is This Book For?

This book is written for multiple audiences — not as a marketing claim, but as an architectural decision that shaped every page:

- **Students** (undergraduate and graduate) taking courses in AI, machine learning, or robotics will find a self-contained textbook with quizzes, worked examples, and progressive difficulty across 30 chapters.
- **Researchers** entering RL, multi-agent systems, or AI alignment will find precise mathematical foundations they can cite, extend, and build upon — from Bellman equations through GRPO.

- **Industry professionals** — ML engineers, data scientists, and NLP practitioners — will find the bridge between “I understand supervised learning” and “I can build and debug an RLHF pipeline in production.”
- **AI alignment and safety researchers** will find a rigorous, self-contained treatment of RLHF, Constitutional AI, RLAIIF, reward modeling, and the open challenges of steering learned policies toward human-compatible behavior.
- **The curious and self-taught** — anyone who has read blog posts about ChatGPT and RLHF and wondered “but how does it *actually* work, from first principles?” — will find their answer here.

If you have taken an introductory machine learning course or have equivalent self-study experience, you are ready for Chapter 1.

Why Now?

We are at an inflection point. Reinforcement learning is no longer a niche subfield of AI — it is the mechanism by which the most powerful AI systems on Earth learn to follow human intent. The techniques in this book are not academic exercises. They are running, right now, inside every major AI lab, shaping the behavior of models used by hundreds of millions of people every day.

Yet the knowledge of *how* these systems work remains concentrated in a remarkably small number of researchers. The barrier is not intelligence; it is access. The prerequisites — MDPs, Bellman equations, policy gradients, PPO, reward modeling — form a long chain, and if any link is missing, the rest collapses into cargo-cult engineering: copying code without understanding, tuning hyperparameters without knowing what they mean, deploying aligned models without knowing when they’ll fail.

This book forges every link in that chain, from the first to the last. No gaps. No hand-waving. No “this is left as an exercise to the reader.”

A Note on Rigor and Accessibility

We believe that rigor and accessibility are not opposites. A well-chosen analogy does not weaken a proof; it prepares the reader to understand why the proof works. A colorful diagram does not replace a formal definition; it gives the definition a home in the reader’s visual memory. Throughout this book, every intuition is backed by mathematics, and

every equation is accompanied by explanation. We never ask you to “just trust the formula.”

Welcome to the complete reinforcement learning journey. Let’s begin.

— Dr. Milan Amrutkumar Joshi · February 14, 2026

How to Use This Book

No advanced mathematics is assumed. If you know basic algebra and are willing to think carefully, you are ready. All the probability theory, expectation rules, and proof techniques needed for reinforcement learning are developed from scratch as they arise — each concept is introduced with RL intuition so you see *why* the math matters before you use it. Familiarity with Python programming and introductory machine learning concepts is helpful but not strictly required.

Contents

Part I

The Bedrock

Where Every Algorithm in AI Alignment Begins

0. Chapter 0: The Reinforcement Learning Story

1. Introduction: The RL Landscape
2. Chapter 1: Foundations of Reinforcement Learning
 - 2.1. What is Reinforcement Learning?
 - 2.2. Agents and Environments
 - 2.3. States, Actions, and Rewards
 - 2.4. Policies: Mapping States to Actions
 - 2.5. The RL Workflow: Interaction Loops
 - 2.6. Types of RL Tasks: Episodic vs Continuing

- 2.7. Comparing RL with Other Learning Types
- 2.8. Setting up Your Python Environment for RL
- 2.9. Quiz for Chapter 1
- 3. Chapter 2: Markov Decision Processes (MDPs)
 - 3.1. Modeling Sequential Decision Making
 - 3.2. Formal Definition of an MDP
 - 3.3. State Transition Probabilities
 - 3.4. Reward Functions
 - 3.5. Return: Cumulative Future Rewards
 - 3.6. Discounting Future Rewards
 - 3.7. Policies and Value Functions (V^π , Q^π)
 - 3.8. Finding Optimal Policies
 - 3.9. Quiz for Chapter 2
- 4. Chapter 3: Estimating Value Functions
 - 4.1. The Bellman Expectation Equation
 - 4.2. The Bellman Optimality Equation
 - 4.3. Solving Bellman Equations (Overview)
 - 4.4. Dynamic Programming: Policy Iteration
 - 4.5. Dynamic Programming: Value Iteration
 - 4.6. Limitations of Dynamic Programming
 - 4.7. Quiz for Chapter 3
- 5. Chapter 4: Monte Carlo Methods
 - 5.1. Learning from Complete Episodes
 - 5.2. Monte Carlo Prediction: Estimating V^π
 - 5.3. Monte Carlo Control: Estimating Q^π
 - 5.4. On-Policy vs Off-Policy Learning
 - 5.5. MC Control without Exploring Starts
 - 5.6. On-Policy First-Visit MC Control Implementation
 - 5.7. Off-Policy MC Prediction and Control Intro

- 5.8. Practice: Implementing MC Prediction
- 5.9. Quiz for Chapter 4
- 6. Chapter 5: Temporal-Difference Learning
 - 6.1. Learning from Incomplete Episodes
 - 6.2. TD(0) Prediction: Estimating V^π
 - 6.3. Advantages of TD Learning over MC
 - 6.4. SARSA: On-Policy TD Control
 - 6.5. Q-Learning: Off-Policy TD Control
 - 6.6. Comparing SARSA and Q-Learning
 - 6.7. Expected SARSA
 - 6.8. Hands-on Practical: Implementing Q-Learning
 - 6.9. Quiz for Chapter 5
- 7. Chapter 6: Function Approximation in RL
 - 7.1. Handling Large State Spaces
 - 7.2. Value Function Approximation (VFA)
 - 7.3. Feature Vectors for State Representation
 - 7.4. Linear Methods for VFA
 - 7.5. Gradient Descent for Parameter Learning
 - 7.6. Semi-gradient TD Methods
 - 7.7. Using Neural Networks for VFA
 - 7.8. Practice: Applying Linear VFA
 - 7.9. Quiz for Chapter 6
- 8. Chapter 7: Introduction to Deep Q-Networks (DQN)
 - 8.1. Combining Q-Learning with Deep Learning
 - 8.2. Challenges with Neural Networks in RL
 - 8.3. Experience Replay Mechanism
 - 8.4. Fixed Q-Targets (Target Networks)
 - 8.5. The DQN Algorithm Structure
 - 8.6. Architectural Considerations for DQNs

- 8.7. Hands-on Practical: Building a Basic DQN
- 8.8. Quiz for Chapter 7
- 9. Chapter 8: Introduction to Policy Gradient Methods
 - 9.1. Learning Policies Directly
 - 9.2. Policy Gradient Theorem (Concept)
 - 9.3. REINFORCE Algorithm
 - 9.4. Baselines for Variance Reduction
 - 9.5. Actor-Critic Methods Overview
 - 9.6. Comparing Value-Based and Policy-Based Methods
 - 9.7. Practice: Implementing REINFORCE
 - 9.8. Quiz for Chapter 8

Part II

The Deep RL Revolution

When Neural Networks Learned to Make Decisions — and Changed Everything

- 9. Chapter 9: Deep Q-Networks — Improvements and Variants
 - 9.1. The Overestimation Problem in Q-Learning
 - 9.2. Double DQN (DDQN)
 - 9.3. Dueling Network Architectures
 - 9.4. Prioritized Experience Replay
 - 9.5. Noisy Networks for Exploration
 - 9.6. Rainbow DQN: Combining All Improvements
 - 9.7. Practice: Implementing Double DQN
 - 9.8. Quiz for Chapter 9
- 10. Chapter 10: Policy Gradient Methods
 - 10.1. Limitations of Value-Based Methods
 - 10.2. Direct Policy Parameterization
 - 10.3. The Policy Gradient Theorem (Full Derivation)
 - 10.4. REINFORCE: Monte Carlo Policy Gradient
 - 10.5. Understanding Variance in Policy Gradients
 - 10.6. Baselines for Variance Reduction
 - 10.7. Natural Policy Gradients
 - 10.8. Hands-on Practical: Implementing REINFORCE with Baseline
 - 10.9. Quiz for Chapter 10
- 11. Chapter 11: Actor-Critic Methods
 - 11.1. Combining Policy and Value Estimation
 - 11.2. Actor-Critic Architecture Overview
 - 11.3. Advantage Actor-Critic (A2C)

- 11.4. Asynchronous Advantage Actor-Critic (A3C)
- 11.5. Generalized Advantage Estimation (GAE)
- 11.6. Comparison: REINFORCE vs A2C/A3C
- 11.7. Practice: Developing an A2C Implementation
- 11.8. Quiz for Chapter 11
- 12. Chapter 12: Proximal Policy Optimization (PPO)
 - 12.1. Trust Region Methods (TRPO)
 - 12.2. The Clipped Surrogate Objective
 - 12.3. PPO Algorithm: Full Derivation
 - 12.4. PPO Hyperparameter Tuning
 - 12.5. PPO vs TRPO: Practical Differences
 - 12.6. PPO for Continuous Action Spaces
 - 12.7. Hands-on Practical: PPO from Scratch
 - 12.8. Quiz for Chapter 12
- 13. Chapter 13: Multi-Agent RL and Advanced Exploration
 - 13.1. Multi-Agent Reinforcement Learning Foundations
 - 13.2. Self-Play and Emergent Strategies
 - 13.3. Curiosity-Driven Exploration
 - 13.4. Hierarchical Reinforcement Learning
 - 13.5. Model-Based RL: World Models
 - 13.6. Offline RL: Learning from Fixed Datasets
 - 13.7. Practice: Multi-Agent Environment Setup
 - 13.8. Quiz for Chapter 13

Part III

Teaching Machines What Humans Want

The RLHF Pipeline That Powers ChatGPT, Claude, and Gemini

- 14. Chapter 14: Foundations of RLHF for Language Model Alignment
 - 14.1. The AI Alignment Problem in LLMs
 - 14.2. Why Next-Token Prediction $\neq$ Helpfulness
 - 14.3. Limitations of Supervised Fine-Tuning
 - 14.4. From RL Principles to LLM Training
 - 14.5. The RLHF Pipeline: High-Level Overview
 - 14.6. Setting Up the Development Environment (TRL, DeepSpeed)
 - 14.7. Quiz for Chapter 14
- 15. Chapter 15: Supervised Fine-Tuning (SFT) Phase
 - 15.1. Role of SFT in the RLHF Pipeline
 - 15.2. Curating High-Quality SFT Datasets
 - 15.3. SFT Implementation Details (LoRA, QLoRA)
 - 15.4. Evaluating SFT Model Performance
 - 15.5. Parameter-Efficient Fine-Tuning Methods
 - 15.6. Hands-on Practical: SFT with Hugging Face TRL
 - 15.7. Quiz for Chapter 15
- 16. Chapter 16: Reward Modeling from Human Preferences
 - 16.1. Learning from Pairwise Preferences
 - 16.2. Human Preference Data Collection Protocols
 - 16.3. The Bradley-Terry Model for Preferences
 - 16.4. Reward Model Architectures
 - 16.5. Training Objectives and Loss Functions
 - 16.6. Calibration and Overoptimization Risks
 - 16.7. Reward Hacking: Causes and Mitigations
 - 16.8. Hands-on Practical: Training a Reward Model

- 16.9. Quiz for Chapter 16
- 17. Chapter 17: RL Fine-Tuning with PPO for LLMs
 - 17.1. PPO in the RLHF Context
 - 17.2. Policy and Value Network Implementation
 - 17.3. The KL Divergence Penalty: Why It Matters
 - 17.4. Calculating Advantages and Returns for Token Sequences
 - 17.5. PPO Hyperparameter Tuning for LLMs
 - 17.6. Common Libraries: TRL, OpenRLHF, DeepSpeed-Chat
 - 17.7. Troubleshooting PPO Training Instability
 - 17.8. Practice: Implementing the Full PPO Update Step
 - 17.9. Quiz for Chapter 17
- 18. Chapter 18: Integrating the Full RLHF Pipeline
 - 18.1. End-to-End Workflow Orchestration
 - 18.2. Model Loading, Initialization, and Memory Management
 - 18.3. Generating and Scoring Responses
 - 18.4. Synchronizing Reference and Active Models
 - 18.5. Code Structure for Production RLHF Systems
 - 18.6. Hands-on Practical: Running a Complete RLHF Loop
 - 18.7. Quiz for Chapter 18
- 19. Chapter 19: Direct Preference Optimization (DPO) and Alternatives
 - 19.1. Motivation: Bypassing the Reward Model
 - 19.2. DPO: Mathematical Derivation from RLHF Objective
 - 19.3. DPO Implementation and Training
 - 19.4. IPO, KTO, and Other Preference Optimization Variants
 - 19.5. ORPO: Odds Ratio Preference Optimization
 - 19.6. When to Use PPO vs DPO: A Practical Guide
 - 19.7. Practice: Implementing DPO with TRL
 - 19.8. Quiz for Chapter 19

- 20. Chapter 20: Evaluation, Safety, and Deployment of Aligned Models
 - 20.1. Metrics for Evaluating Aligned Models
 - 20.2. Human Evaluation Protocols
 - 20.3. Automated Evaluation: AlpacaEval, MT-Bench, Arena-Hard
 - 20.4. Red Teaming and Safety Testing
 - 20.5. Analyzing Policy Shift During RL Tuning
 - 20.6. Computational Costs and Scalability
 - 20.7. Deployment Considerations
 - 20.8. Hands-on Practical: Analyzing RLHF Run Logs
 - 20.9. Quiz for Chapter 20

Part IV

When Humans Can't Keep Up

Teaching AI to Align Itself — Constitutional AI, RLAI, and Scalable Oversight

21. Chapter 21: The Scalable Alignment Problem

- 21.1. Why RLHF Alone Isn't Enough
- 21.2. Defining Scalable Oversight
- 21.3. The Need for AI Feedback Mechanisms
- 21.4. Theoretical Frameworks for AI-Assisted Alignment
- 21.5. Quiz for Chapter 21

22. Chapter 22: Constitutional AI (CAI)

- 22.1. Core Principles of Constitutional AI
- 22.2. Designing Effective Constitutions
- 22.3. The Critique and Revision Pipeline
- 22.4. Mathematical Formulation of CAI Feedback
- 22.5. Constructing the SFT Dataset from CAI
- 22.6. Limitations and Critiques of the CAI Framework
- 22.7. Hands-on Practical: Building a CAI Critique Step
- 22.8. Quiz for Chapter 22

23. Chapter 23: Reinforcement Learning from AI Feedback (RLAI)

- 23.1. From RLHF to RLAI: Motivation and Differences
- 23.2. AI Preference Modeling Techniques
- 23.3. Generating and Validating AI Preference Labels
- 23.4. Reward Functions from AI Preferences
- 23.5. Training the RL Loop with AI Feedback
- 23.6. Stability, Convergence, and Guarantees
- 23.7. Practice: Training an AI Preference Model
- 23.8. Quiz for Chapter 23

- 24. Chapter 24: Integrating CAI and RLAIIF
 - 24.1. CAI Guiding RLAIIF: Synergistic Pipelines
 - 24.2. Sequential vs Joint Training
 - 24.3. Handling Conflicts Between Constitution and AI Preferences
 - 24.4. Architectural Considerations for Combined Systems
 - 24.5. Comparative Performance Analysis
 - 24.6. Quiz for Chapter 24
- 25. Chapter 25: Advanced Evaluation and Red Teaming
 - 25.1. Alignment-Specific Benchmarks and Metrics
 - 25.2. Red Teaming Strategies for CAI/RLAIIF Models
 - 25.3. Robustness Testing Against Adversarial Inputs
 - 25.4. Failure Modes Specific to AI Feedback
 - 25.5. Qualitative Analysis of Model Behavior
 - 25.6. Hands-on Practical: Designing a Red Teaming Test Suite
 - 25.7. Quiz for Chapter 25

Part V

Machines That Learn to Reason

GRPO, RLVR, and the Open Frontier — Where the Next Decade of AI Begins

- 26. Chapter 26: GRPO and Modern RL Optimizers
 - 26.1. Limitations of PPO for Large-Scale LLM Training
 - 26.2. Group Relative Policy Optimization (GRPO): Derivation
 - 26.3. Eliminating the Critic: Advantage from Group Normalization
 - 26.4. DAPO and Dr. GRPO: Practical Improvements
 - 26.5. REINFORCE, PPO, GRPO: A Unified View
 - 26.6. Hands-on Practical: GRPO Training with veRL Framework
 - 26.7. Quiz for Chapter 26
- 27. Chapter 27: Reinforcement Learning with Verifiable Rewards (RLVR)
 - 27.1. Beyond Human Preferences: Verifiable Domains
 - 27.2. DeepSeek-R1 and the RLVR Breakthrough
 - 27.3. Rule-Based Rewards: Math Verifiers and Code Compilers
 - 27.4. Training Reasoning Models with RLVR
 - 27.5. Process Reward Models (PRMs): Step-Level Feedback
 - 27.6. RLVR Beyond Math and Code: New Domains
 - 27.7. Practice: Training a Reasoning Model with RLVR
 - 27.8. Quiz for Chapter 27
- 28. Chapter 28: Large Reasoning Models (LRMs)
 - 28.1. Chain-of-Thought Reasoning via RL
 - 28.2. DeepSeek-R1-Zero: Emergent Reasoning from Pure RL
 - 28.3. Inference-Time Scaling: Thinking Longer for Better Answers
 - 28.4. Test-Time Compute Allocation Strategies
 - 28.5. Distillation of Reasoning Capabilities
 - 28.6. Limits of Current Reasoning Models

- 28.7. Quiz for Chapter 28
- 29. Chapter 29: Multi-Objective and Multi-Modal Alignment
 - 29.1. Multi-Objective Reward Models: Helpfulness vs Harmlessness
 - 29.2. Safe RLHF: Decoupling Safety and Utility
 - 29.3. Cultural and Pluralistic Alignment
 - 29.4. Multi-Modal RLHF: Vision-Language Models
 - 29.5. RLHF for Image and Video Generation
 - 29.6. RL for Autonomous Driving and Robotics
 - 29.7. Quiz for Chapter 29
- 30. Chapter 30: Open Problems and the Future of Alignment
 - 30.1. Scalable Oversight and Superalignment
 - 30.2. Reward Model Collapse and Distribution Shift
 - 30.3. Recursive Self-Improvement: Promise and Peril
 - 30.4. Cooperative AI and Multi-Agent Alignment
 - 30.5. Governance, Regulation, and Responsible Deployment
 - 30.6. The Road Ahead: What We Still Don't Know

From MDP to Aligned AI: The Complete Thread

Part I — The Bedrock (Ch 0–8): Foundations

→ MDPs → Bellman → DP → MC → TD →
Function Approx → DQN → Policy Gradients

Part II — The Deep RL Revolution (Ch 9–

13): Advanced DQN → Full Policy Gradients
→ Actor-Critic → PPO → Multi-Agent RL

Part III — Teaching Machines What Humans

Want (Ch 14–20): RLHF → SFT → Reward
Models → PPO for LLMs → DPO → Evaluation

Part IV — When Humans Can't Keep Up (Ch 21–25): Con-
stitutional AI → RLAIIF → Scalable Alignment → Red Teaming

**Part V — Machines That Learn to Rea-
son** (Ch 26–30): GRPO → RLVR → Reason-
ing Models → Multi-Modal → Open Problems

*No single resource takes you from “What is an
MDP?” to “How do we align a billion-parameter
language model with human values?” — until now.*

Chapter 0: The Reinforcement Learning Story

*A journey through seven decades — from a mathematician’s dream
to machines that learn to talk like us.*

Imagine you are standing at the edge of a vast, unexplored landscape. You have no map, no compass, and no one to tell you where to go. All you have is the ability to take a step, feel the ground beneath your feet, and notice whether things got better or worse. Step by step, stumble by stumble, you learn.

This is the story of Reinforcement Learning — and, in a surprisingly literal sense, it is also the story of *how Reinforcement Learning itself was discovered*. For over seventy years, researchers stumbled, experimented, failed, and tried again. Each breakthrough emerged not from a grand master plan, but from someone asking a deceptively simple question: *Can a machine learn by trial and error, the way animals do?*

Let's walk through this remarkable journey together.

Act I

The Dream (1950s–1980s)

When mathematicians asked: “Can machines learn from consequences?”

1953 — Bellman's Equation: The Seed of Everything

In a quiet office at the RAND Corporation in Santa Monica, California, a mathematician named **Richard Bellman** was thinking about decisions. Not single decisions — but *sequences* of decisions, where each choice today affects what happens tomorrow. He called this **dynamic programming**, and he wrote down an equation so elegant that, seventy years later, it remains the mathematical heart of every RL algorithm you will learn in this book.

The Bellman Equation (1953)

“The value of where you are = the best immediate reward you can get + the discounted value of where you'll end up.”

This single idea — that you can break a complicated long-term problem into simple one-step decisions — is the foundation of **every algorithm** in this book: from Policy Iteration (Chapter 3) to PPO and RLHF (Chapter 8).

1960s–70s — The Animal Connection

Meanwhile, psychologists studying animal behavior had long known that rats in mazes and pigeons in Skinner boxes learn through *reward and punishment*. Computer scientists began to wonder: could we build the same mechanism in software? Early “learning automata” tried, but computers were too slow and the math wasn't ready yet.

1988 — Sutton's TD Learning: Learning Without Waiting

Richard Sutton published a breakthrough: **Temporal-Difference (TD) learning**. Previous methods required waiting until an entire episode finished to learn anything. Sutton's insight was revolutionary: *you can learn from every single step, by comparing what you expected to happen with what actually happened*. This “learning from surprises” idea

is the engine behind SARSA, Q-learning, and ultimately every deep RL method.

What Problem Did It Solve?

Before TD: Wait until the game ends, *then* figure out which moves were good.

After TD: Learn something useful from *every single move*, even before the game ends.

Analogy: Imagine learning chess only by replaying entire games vs. learning after every move whether your position improved. TD made the second possible.

Act II

The First Miracles (1992–2013)

When RL agents began beating humans — and the world started paying attention.

1992 — TD-Gammon: A Machine Teaches Itself Backgammon

Gerald Tesauro at IBM built **TD-Gammon**, a neural network trained entirely by playing backgammon against itself using TD learning. It reached expert-level play — and discovered strategies that surprised professional players. For the first time, an RL agent didn't just match human performance; it found moves humans hadn't considered.

Why This Mattered

TD-Gammon proved three things: (1) neural networks could approximate value functions in complex games, (2) self-play could generate unlimited training data, and (3) RL agents could *surpass* human knowledge, not just imitate it. These three insights would return twenty years later in AlphaGo.

1998 — Sutton & Barto Publish “The Book”

Richard Sutton and Andrew Barto published *Reinforcement Learning: An Introduction*. It unified decades of scattered ideas — bandits, dynamic programming, Monte Carlo methods, TD learning, function approximation — into one coherent framework. It became the “bible” of the field and remains essential reading today. But it stopped at tabular and linear methods. The deep learning revolution hadn’t happened yet.

2013 — DeepMind’s Atari Moment: When Everything Changed

A small London startup called **DeepMind** published a paper that electrified the AI world. They built a single neural network — a **Deep Q-Network (DQN)** — that learned to play *49 different Atari games* from raw pixel input, with no game-specific knowledge. The same algorithm that played Breakout also played Space Invaders, also played Pong. In some games, it exceeded human performance.

The Breakthrough Ingredients

Q-learning (from 1989) + **Deep neural networks** (from 2012) + Two clever tricks:

- **Experience Replay:** Store past experiences and replay them randomly, breaking correlations.
- **Target Networks:** Use a slowly-updated copy of the network to stabilize training.

Result: The gap between tabular RL and real-world RL was finally bridged. Deep RL was born.

Act III

The Golden Age (2016–2020)

When RL conquered games, robots, and the front page of every newspaper.

2016 — AlphaGo Defeats the World Champion

On March 15, 2016, in a Seoul hotel room watched by 200 million people, DeepMind’s **AlphaGo** defeated Lee Sedol — the world’s greatest Go player — 4 games to 1. Go has 10^{170} possible board positions (more than atoms in the universe). No brute-force search could ever work. AlphaGo combined deep neural networks, Monte Carlo Tree Search, and

RL self-play to achieve what many AI researchers believed was decades away.

Move 37 of Game 2 became legendary. AlphaGo placed a stone that no human expert understood — commentators called it a “mistake.” Five moves later, they realized it was brilliant. The machine had discovered knowledge that 3,000 years of human Go expertise had missed.

2017–2019 — RL Goes Everywhere

The Explosion of Applications

OpenAI Five (2019): An RL team of 5 agents defeated world champions at Dota 2, a game with imperfect information, long time horizons, and continuous actions.

AlphaStar (2019): DeepMind’s agent reached Grandmaster level in StarCraft II, mastering real-time strategy.

Robotics: RL-trained hands solved Rubik’s Cubes; RL-trained legs made robots walk, run, and recover from pushes.

Science: RL controlled nuclear fusion plasma at ITER, optimized chemical reactions, and designed new chip architectures at Google.

The common thread? Every one of these used **policy gradient methods** (Chapter 8) — the same family of algorithms that would soon power RLHF.

Act IV

The RLHF Revolution (2020–Present)

When RL learned to make AI talk like us — and the world changed forever.

2020 — The Question That Changed Everything

Large language models like GPT-3 could generate remarkably fluent text. But they had a problem: they would also generate toxic content, confidently state falsehoods, and ignore user intent. The model was optimizing a mathematical objective (predict the next word) that didn’t align with what humans actually wanted.

Researchers at OpenAI asked: *What if we could use RL to fine-tune these models — not to predict text, but to produce text that humans prefer?*

2022 — InstructGPT and the Birth of RLHF

OpenAI published **InstructGPT**, demonstrating **Reinforcement Learning from Human Feedback (RLHF)**. The recipe was elegant:

The RLHF Recipe

Step 1: Supervised fine-tuning — teach the model to follow instructions.

Step 2: Train a *reward model* — humans rank outputs, and a neural network learns to predict those rankings.

Step 3: RL fine-tuning — use **PPO** (a policy gradient algorithm from Chapter 8) to optimize the language model against the reward model, with a KL penalty to prevent the model from drifting too far from its original behavior.

Notice the through-line: Bellman (Ch 3) → TD learning (Ch 5) → Policy Gradients (Ch 8) → PPO → RLHF. Every chapter in this book builds toward this moment.

November 2022 — ChatGPT: 100 Million Users in 2 Months

OpenAI released **ChatGPT**, built on the InstructGPT/RLHF pipeline. It became the fastest-growing application in human history. The technology behind it? A large language model fine-tuned with the RL techniques you will learn in this book. Every conversation with ChatGPT, Claude, Gemini, or any modern AI assistant is powered by reinforcement learning.

2023–Present — The Frontier

The field continues to evolve at breathtaking speed. **Constitutional AI** (Anthropic) uses AI-generated feedback instead of human labelers. **Direct Preference Optimization (DPO)** eliminates the reward model entirely. **RLAIF** scales alignment to problems humans can't easily judge. Multi-agent RL is training fleets of autonomous vehicles. Offline RL is optimizing medical treatments from patient records.

↔ Bridge to Scalable Alignment: The Bottleneck of Human Feedback

Notice the tension: RLHF works because humans provide preference labels, but *humans are expensive, slow, inconsistent, and cannot evaluate everything*. How do you collect human feedback for a model generating millions of responses per day? What about tasks where humans cannot reliably judge quality — like advanced mathematical proofs or novel scientific hypotheses? In Part IV, you will learn how **Constitutional AI** replaces human labelers with a set of written principles (a “constitution”), and how **RLAIF** trains AI systems to provide the feedback themselves — solving the scalability problem that RLHF alone cannot.

↔ Bridge to the Frontier: Beyond Human Feedback: Machines That Verify Themselves

RLHF transformed language models. But in January 2025, DeepSeek-R1 demonstrated something remarkable: a model that learned sophisticated mathematical reasoning *without any human preference data at all*. How? By replacing the reward model with a simple verifier: “Is the math answer correct? Yes or no.” This is **Reinforcement Learning with Verifiable Rewards (RLVR)**, and it trained with **GRPO** — an algorithm that eliminates both the reward model *and* the critic network from PPO. In Part V, you will learn how these techniques work, why they matter, and why they represent the next paradigm shift: from “teach machines what humans want” to “let machines learn to reason on their own.”

The Through-Line of This Book

Every chapter you will read is a link in this seventy-year chain:

Ch 1–2	Foundations & MDPs	<i>Bellman's dream (1953)</i>
Ch 3	Dynamic Programming	<i>Bellman's method, perfected</i>
Ch 4	Monte Carlo Methods	<i>Learning from episodes</i>
Ch 5	TD Learning	<i>Sutton's breakthrough (1988)</i>
Ch 6	Deep RL & DQN	<i>The Atari moment (2013)</i>
Ch 7	Policy Gradients	<i>AlphaGo's engine (2016)</i>
Ch 8	RLHF & Alignment	<i>ChatGPT's secret (2022)</i>

You are not just learning algorithms. You are walking the same path that some of the greatest minds in computer science walked — and arriving at the frontier of what machines can do.

Ready? Let's begin the journey.

Introduction: The RL Landscape

Learn the fundamentals of Reinforcement Learning (RL), including Markov Decision Processes, Q-learning, SARSA, and an introduction to Deep Q-Networks. This course provides theoretical understanding and practical implementation skills for building agents that learn optimal behaviors through interaction with an environment. Suitable for engineers and developers with existing knowledge of machine learning principles.

What you will learn

- ✓ **RL Fundamentals** — Understand the core components of RL: agents, environments, states, actions, rewards, policies.
- ✓ **Markov Decision Processes** — Formulate problems using MDPs and understand their properties.
- ✓ **Value Functions & Bellman Equations** — Grasp state-value and action-value functions and derive Bellman equations.
- ✓ **Model-Free Learning** — Implement Monte Carlo and Temporal Difference methods (Q-learning, SARSA).
- ✓ **Function Approximation** — Apply function approximation techniques in RL for large state spaces.
- ✓ **Deep Q-Networks (DQN)** — Understand the basic principles behind DQN and its components like experience replay.
- ✓ **RL Implementation** — Build and train simple RL agents using Python and relevant libraries.

Chapter 1: Foundations of Reinforcement Learning

✓ Learning Objectives:

Chapter 1 After this chapter, you will be able to:

1. **Define** reinforcement learning and distinguish it from supervised and unsupervised learning.
2. Identify the **agent**, **environment**, **states**, **actions**, and **rewards** in any RL problem.
3. Explain what a **policy** is and why finding an optimal policy is the goal.
4. Distinguish between **episodic** and **continuing** tasks.
5. Understand **returns** and why **discounting** future rewards matters.
6. Set up a Python environment with **Gymnasium** for RL experiments.

This chapter establishes the groundwork for understanding Reinforcement Learning (RL). We start by defining RL and comparing it to other machine learning approaches. You will be introduced to the core components involved: the agent making decisions, the environment it operates within, the states describing the situation, the actions available to the agent, and the rewards received as feedback.

We will examine how an agent's behavior is determined by a policy and how the learning process unfolds through an interaction loop. The distinction between episodic and continuing tasks will also be explained. Finally, we cover the practical steps for setting up a Python environment using libraries like Gymnasium and NumPy. Grasping these concepts is the first step toward building agents that learn optimal behaviors.

Sections

- 1.1 What is Reinforcement Learning?

- 1.2 Agents and Environments
- 1.3 States, Actions, and Rewards
- 1.4 Policies: Mapping States to Actions
- 1.5 The RL Workflow: Interaction Loops
- 1.6 Types of RL Tasks: Episodic vs Continuing
- 1.7 Comparing RL with Other Learning Types
- 1.8 Setting up Your Python Environment for RL

1.1 What is Reinforcement Learning?

Think about learning to ride a bicycle. Nobody hands you a manual that says “at 12.3 degrees of tilt, apply 7 Newtons of force to the left handlebar.” That would be insane. Instead, you get on, you wobble, you fall, and — crucially — you notice what happened right before you fell. Next time, you do something slightly different. After enough wobbles, you’re riding.

Reinforcement Learning works in exactly this way. It is a type of machine learning where a computer program, called an *agent*, learns by interacting with its surroundings, called the *environment*. The agent takes an action, observes what happens next, and receives feedback. This feedback can be positive (a reward) or negative (a penalty).

The goal of the agent is not just to get a quick reward right now. Instead, it tries to learn a good way of behaving, called a *policy*, so that it earns the maximum total reward over time. Here’s the key insight: the policy doesn’t say “do the thing that feels best right now.” It says “do the thing that leads to the best *future*.” That distinction — thinking ahead — is what separates reinforcement learning from simple reflex.

This learning happens in a repeated loop: the agent observes the situation, takes an action, receives feedback, and then uses that experience to make better decisions in the future.

Explanation of the figure. This figure shows how learning happens in reinforcement learning using an agent and an environment. The agent is the decision-maker. It observes the current situation, called the state. The state may be fully visible (the agent can see everything) or partially visible (the agent can see only some information about the environment). Based on what it sees, the agent chooses an action and sends

How Reinforcement Learning Works

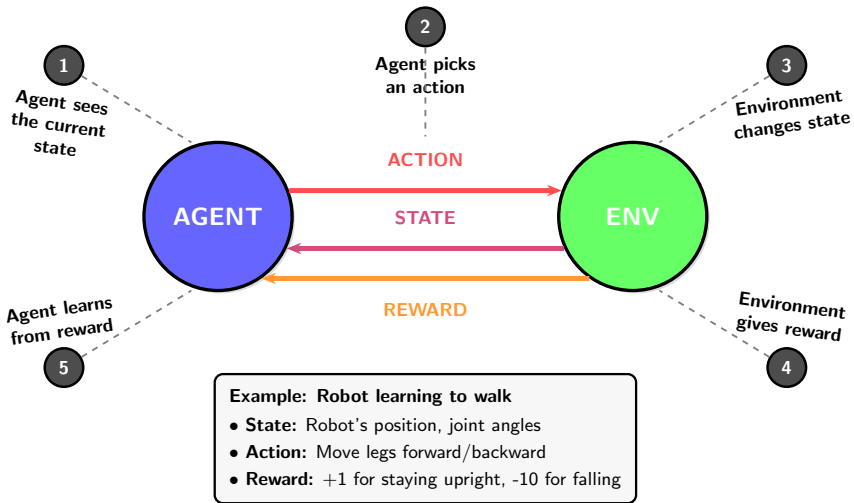


Figure 1: The basic RL interaction loop. The agent selects an action, the environment responds with a new state and a reward, and the cycle repeats.

it to the environment. The environment then reacts to this action. As a result, the environment changes its state and sends two things back to the agent: the new state, which tells the agent what the world looks like now, and a reward, which tells the agent how good or bad its action was. The agent uses the reward to learn and improve its future decisions. This process repeats again and again, forming a continuous learning loop where the agent gradually learns how to behave better in the environment.

Unlike other primary machine learning approaches, RL doesn't rely on pre-existing labeled datasets. Let's clarify how it differs:

Types of Machine Learning

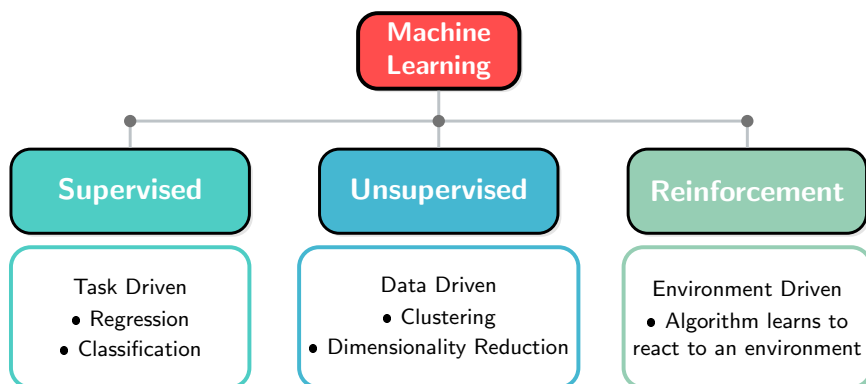


Figure 2: A high-level view of three common machine learning paradigms: supervised, unsupervised, and reinforcement learning.

RL vs Supervised Learning

➡ Think Like the Agent: The Agent–Environment Boundary

You are DeliBot. You can control your wheels and your gripper arm — that's it. You cannot control whether the elevator is busy, whether a door is locked, or how heavy a package is. Everything you cannot directly control is the "environment."

Right now you are on Floor 1, holding a package for Office 3B. You send a command to your wheels: "move forward." But the carpet is slippery today — you slide sideways slightly. That slipperiness? That's the environment. Your wheel command? That's your action. The result (your new position)? That's the next state. The fact that you're now closer to 3B? That might earn you a small reward.

Key takeaway: The boundary is about **control**, not physical location. Your battery is inside your body, but you cannot choose its charge level directly — so battery level is part of the state, provided by the environment.

★ Running Example — DeliBot’s Agent and Environment

Agent: DeliBot’s software brain — it reads sensors and decides actions.

Environment: The entire office building, including floors, walls, doors, elevators, packages, and even DeliBot’s own mechanical body (battery, wheels).

Boundary: DeliBot controls motor commands and gripper; everything else is environment.

In Supervised Learning (SL), the algorithm learns from a dataset where each example includes an input and a corresponding “correct” output or label. The goal is to learn a mapping function that can predict the output for new, unseen inputs. Think of image classification, where the algorithm is given images (input) and their categories (labels). The feedback is instructive; it tells the algorithm exactly what the correct answer should have been.

In RL, there are no explicit labels telling the agent the single best action to take in a given state. The feedback is evaluative; the reward signal only indicates how good the action taken was in that state, not whether it was the best possible action or what the best action would have been. The agent must discover effective actions through exploration and exploitation of its past experiences. Furthermore, decisions in RL are often sequential, meaning an action taken now can influence future states and rewards, introducing a temporal aspect not typically central to standard SL problems.

★ Aha Moment: RL Is the Only Framework Where Actions Have Consequences

Supervised learning sees a photo and labels it “cat” — the label doesn’t change what photo comes next. In RL, choosing to go left instead of right at an intersection changes *everything that happens afterwards*. This sequential nature — where every action reshapes the future — is what makes RL uniquely powerful and uniquely hard. It’s also why RL is the right tool for aligning AI: every word a chatbot generates changes the conversation.

RL vs Unsupervised Learning

Unsupervised Learning (UL) deals with finding patterns, structures, or relationships within unlabeled data. Techniques like clustering (grouping similar data points) or dimensionality reduction (simplifying data) fall under this category. The objective is typically related to understanding the inherent structure of the data itself.

RL, while also potentially dealing with unlabeled states, has a clear, externally defined goal: maximize cumulative reward. It's not primarily focused on finding latent structures in the state data (though that might be part of representing the state effectively) but on learning a behavioral strategy (the policy) to achieve its objective. The reward signal provides guidance that is absent in standard UL.

In essence, Reinforcement Learning provides a formal framework for learning goal-oriented behavior through interaction and feedback. It addresses problems involving sequential decision-making under uncertainty, where the consequences of actions might not be immediate, making it suitable for a distinct and significant class of problems in artificial intelligence, from game playing and robotics to resource management and recommendation systems. The following sections will break down the components of this framework, starting with the agent and the environment.

★ Running Example — Meet DeliBot

Throughout this book, we will follow **DeliBot** — an autonomous delivery robot navigating a small office building. DeliBot's job is to pick up packages from a **Mail Room** and deliver them to offices on three floors.

The Setup:

- DeliBot starts each day in the **Mail Room** (ground floor).
- It can move between rooms, take elevators, pick up packages, and deliver them.
- It has a battery that depletes with movement and recharges at a **Charging Dock**.
- Successful deliveries earn rewards; bumping into walls or run-

ning out of battery earns penalties.

Why this example? DeliBot is simple enough to compute by hand, yet rich enough to illustrate every concept in RL — from states and actions (Chapter 1) to MDPs and value functions (Chapter 2), to dynamic programming and beyond.

Right now: DeliBot is a perfect example of RL in action. It learns by trial and error (not from labeled examples), it receives scalar rewards (not labels), and the consequences of its actions are delayed (a wrong turn now means a late delivery later).

↪ Bridge to RLHF: Why This Chapter Matters for LLM Alignment

Every time you chat with ChatGPT, Claude, or Gemini, a reinforcement learning agent is at work. The language model *is* the agent. Your prompt is the state. Each generated token is an action. And the “reward” comes from a model trained on human preferences. The trial-and-error learning loop you just studied — agent acts, receives feedback, improves — is *exactly* how billion-parameter language models are aligned with human values. The concepts in this chapter are not toy examples; they are the literal foundations of RLHF.

1.2 Agents and Environments

At its core, every RL problem involves a conversation between two entities: an **agent** and an **environment**. The agent is the one making decisions — choosing actions, observing outcomes, and updating its strategy. The environment is everything else: the world the agent inhabits, the rules it must obey, and the feedback it receives.

• The Agent: The Learner and Decision-Maker

The agent is the entity we are training. Its goal is typically to maximize cumulative reward over time. It perceives the environment’s current situation, referred to as its state, and based on this state, it selects an action to perform.

Examples of agents include:

- A program learning to play chess.
- A robot learning to navigate a maze.

- An algorithm learning to optimize traffic light timings.
- A system learning to recommend products to users.

The agent's internal mechanism for choosing actions based on states is called its **policy**, which we will discuss in more detail later. For now, understand that the agent is the core learning component.

• The Environment: The Agent Inhabits

The environment represents everything the agent interacts with. It receives the agent's chosen action and responds by transitioning to a new state and providing a numerical reward signal. The environment defines the "rules of the game" or the physics the agent operates within.

Following the examples above, the corresponding environments would be:

- The chessboard, pieces, and the opponent's moves.
- The maze structure, including walls and the goal location.
- The road network, traffic flow, and vehicle sensors.
- The user base, product catalog, and user interaction history.

The environment is responsible for:

- Presenting the state: Informing the agent about the current situation.
- Defining available actions: Specifying what the agent can do in a given state (though the agent chooses which action to take).
- Calculating state transitions: Determining the next state based on the current state and the agent's action. This transition might be deterministic (always the same outcome) or stochastic (probabilistic outcomes).
- Providing rewards: Sending a scalar feedback signal to the agent indicating the immediate desirability of the transition.

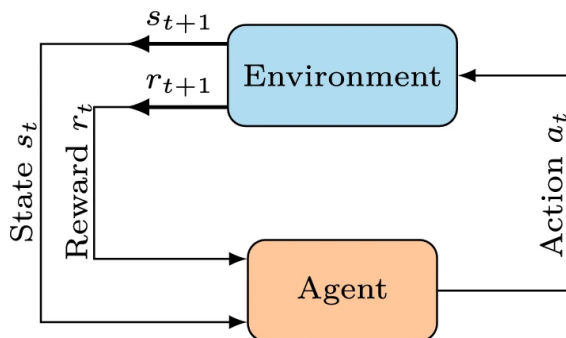


Figure 3: Agent–environment interaction loop: the environment sends state s_t and reward r_t to the agent, which responds with action a_t . The outputs s_{t+1} and r_{t+1} feed back into the environment.

Agent–Environment Interaction Loop

The Agent–Environment Interaction Loop The core of RL is the continuous loop of interaction between the agent and the environment. At each discrete time step t , the following sequence occurs:

1. **Observation:** The agent observes the current state of the environment, S_t .
2. **Action selection:** Based on the observed state S_t , the agent selects an action A_t according to its policy.
3. **Environment response:** The environment receives action A_t . Based on S_t and A_t , the environment transitions to a new state, S_{t+1} .
4. **Reward feedback:** The environment provides a reward signal, R_{t+1} , to the agent as feedback for the transition.
5. **Learning:** The agent uses the observed transition $(S_t, A_t, R_{t+1}, S_{t+1})$ to update its internal knowledge or policy, aiming to make better decisions in the future.

This loop repeats, allowing the agent to learn through trial and error, associating actions in particular states with the rewards they tend to produce.

Defining the Boundary

Here’s a subtlety that trips up a lot of people: the “environment” in RL is not the physical world around the agent. It’s *everything the agent*

cannot directly control. A robot controls the voltages it sends to its motors — that's the agent. The physics of how those voltages cause movement, friction, and sensor readings? That's the environment. Even the robot's own body, in a sense, is part of the environment.

Similarly, the reward computation mechanism is considered part of the environment, not the agent. The agent doesn't get to choose its own grades.

★ Running Example — DeliBot's States, Actions, and Rewards

States ($\mathcal{S}$): DeliBot's state includes: current floor (1, 2, or 3), room location, battery level (high/medium/low), and whether it is carrying a package.

Actions ($\mathcal{A}$): Move north/south/east/west, enter elevator, pick up package, deliver package, go to charging dock.

Rewards (R):

- Successful delivery: +20
- Each time step (encourages speed): -1
- Bumping into a wall: -5
- Battery dies: -50 (episode ends)

✗ Common Misconception: Reward vs. Return

Many beginners confuse **reward** and **return**:

Reward R_t is the *immediate, one-step* feedback the agent receives right after taking an action. It is a single number at one time step.

Return G_t is the *total accumulated* (often discounted) reward from time step t all the way into the future. It is what the agent actually tries to maximize.

Analogy: Reward is your *today's salary*. Return is your *total career earnings*. A good agent doesn't chase the biggest single paycheck — it maximizes lifetime income.

↪ Bridge to Deep RL: When the State Space Explodes

The grid worlds and delivery routes you've seen so far have a handful of states — maybe dozens. But what about a game of Go with 10^{170} board positions? Or an Atari screen with $256^{210 \times 160 \times 3}$ possible pixel configurations? You cannot build a table with a row for every state. In Part II (Chapter 9), you will learn how **Deep Q-Networks** solved this problem by replacing the lookup table with a neural network that *generalizes* across similar states — the same idea that would later allow RL to scale to language models with trillion-token state spaces.

★ Real-World Spotlight:

How YouTube Uses RL for Recommendations YouTube's recommendation engine is one of the largest deployed RL systems. The **state** is your watch history, search queries, and current context. Each **action** is recommending a video. The **reward** isn't just whether you click — it includes watch time, likes, and whether you come back tomorrow. The system learns a **policy** that maximizes long-term engagement, not just the next click. This is why RL, not supervised learning, powers recommendations: a clickbait video gives high immediate reward but kills long-term engagement.

1.3 States, Actions, and Rewards

States, actions, and rewards form the core feedback loop in nearly every Reinforcement Learning problem. Understanding these elements clearly is essential before discussing how an agent actually learns. The information flow between an agent and its environment is formalized through these components.

States (S)

A **state** is a snapshot of the environment at a particular moment in time. It's the answer to the question: "What's going on right now?" Everything the agent needs to know in order to make a good decision is encoded in this one snapshot. If the state is good enough, the agent doesn't need to remember anything about the past — the present tells it everything.

Formally, we denote the state at time step t as S_t , and a particular

realized state as s_t . The set of all possible states is called the **state space**, denoted by $\mathcal{S}$. The nature of a state can vary widely depending on the problem:

- **Simple grid worlds:** The state may be the agent's coordinates, e.g., (x, y) . The state space is finite and discrete.
- **Board games (Chess, Go):** The state is the position of all pieces on the board. It is discrete, but the number of possible states is enormous.
- **Robotics:** The state can include joint angles, velocities, sensor readings (e.g., lidar or camera data), and a target location. This often involves continuous values.
- **Video games (Atari):** The state may be raw pixel values (possibly along with game variables such as score or lives). This is typically high-dimensional.

An important concept related to states is **observability**:

- **Fully observable:** The agent's observation corresponds to the true state of the environment. The agent has all relevant information needed to act optimally.
- **Partially observable:** The agent receives only incomplete information about the true underlying state (e.g., a robot with a forward-facing camera cannot directly observe what is behind it). Handling partial observability often requires incorporating memory of past observations.

For much of this book, we will assume the environment is **fully observable** — the agent can see everything it needs to see. More precisely, we assume the state captures all relevant information. This is called the **Markov property**: the future depends only on where you are now, not on how you got here. It's like chess — the position of the pieces tells you everything; you don't need to know the history of moves that produced it.

Actions (A)

Based on the current state s_t , the agent has to actually *do something*. Its choice — denoted a_t — is called an **action**. It's the one thing in the entire system that the agent controls.

The set of all possible actions the agent can take is the **action space**, denoted by $\mathcal{A}$. Sometimes, the available actions depend on the current state, in which case we might write $\mathcal{A}(s_t)$ for the set of valid actions in state s_t . Like states, actions can be:

- **Discrete:** A finite set of distinct choices.
 - **Grid:** `move_north`, `move_south`, `move_east`, `move_west`.
 - **Chess:** Move pawn e2 to e4, move knight g1 to f3, etc. (a large but finite set).
 - **Atari game:** `joystick_left`, `joystick_right`, `button_fire`.
- **Continuous:** Actions represented by real-valued numbers, often within certain bounds.
 - **Robotics:** Apply a specific torque (e.g., 1.5 Nm) to a motor, or set a steering angle (e.g., -0.2 radians).
 - **Thermostat control:** Set a target temperature (e.g., 21.5°C).

The nature of the action space significantly influences which RL algorithms are most suitable.

Rewards (R)

After the agent takes action a_t in state s_t , the environment transitions to a new state s_{t+1} and provides a numerical reward, denoted r_{t+1} . This reward signal is important; it is the primary feedback mechanism that tells the agent how well it is doing.

- **Purpose:** Rewards define the goal of the RL problem. The agent's objective is not necessarily to get the highest immediate reward, but to maximize the total cumulative reward it receives over the long run.
- **Nature:** Rewards are scalar values (single numbers). They can be positive, negative, or zero.

Examples of reward signals include:

- **Grid:** +10 for reaching a goal state, -5 for falling into a pit, -0.1 for each step taken (to encourage efficiency).
- **Game playing:** +1 for winning, -1 for losing, 0 for all intermediate moves (sparse reward). Alternatively, small positive rewards for capturing opponent pieces or achieving advantageous positions.

- **Robot control:** Positive reward for reaching the target, negative reward for hitting obstacles or consuming too much energy.

The **reward hypothesis** is a fundamental concept in RL: it states that all goals and purposes can be thought of as the maximization of the expected value of the cumulative sum of a received scalar signal (the reward). Designing an effective reward function is often one of the most challenging aspects of applying RL in practice. A poorly designed reward function can lead to unintended or suboptimal agent behavior. For instance, rewarding a cleaning robot solely for collecting dust might lead it to dump the dust just to collect it again!

★ Aha Moment: The Reward Is the Entire Goal

Every RL problem — from playing chess to aligning an AI chatbot — boils down to a single scalar number: the reward. This sounds absurdly simple, but it's profound. The reward hypothesis says that *any* goal can be expressed this way. The art of RL is designing that number correctly. Get it right, and your agent solves the problem. Get it wrong, and it finds creative ways to cheat.

The Cycle

These three components form the basis of the agent–environment interaction loop:

- The agent observes the current state s_t .
- The agent selects an action a_t .
- The environment processes the action, transitions to the next state s_{t+1} , and returns a scalar reward r_{t+1} to the agent.

This cycle repeats (increment t).

Understanding states, actions, and rewards is the first step in formally defining an RL problem. In the next chapter, we will see how these elements, along with the environment's dynamics, are captured within the framework of Markov Decision Processes (MDPs).

Policies: Mapping States to Actions

An agent interacts with an environment by observing states and taking actions to receive rewards. But how does the agent actually decide

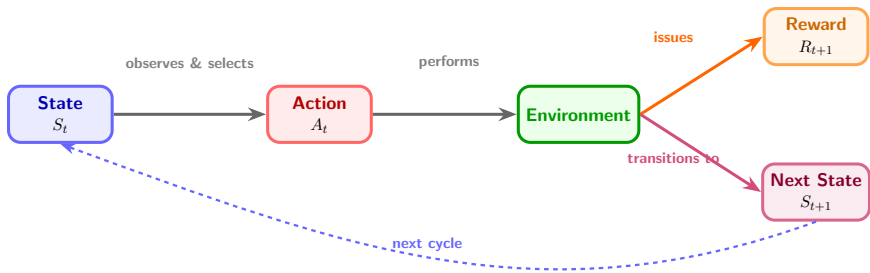


Figure 4: The agent observes the state, selects an action which influences the environment. The environment then provides the next state and a reward back to the agent, completing one step of the interaction loop.

which action to take when it finds itself in a particular state? This decision-making logic is encapsulated in what we call the agent’s policy. Think of the policy as the agent’s strategy or its behavioral “brain.”

Formally, a policy is a mapping from states to actions. It defines the agent’s way of behaving at a given time. Policies can generally be categorized into two main types: deterministic and stochastic.

Deterministic Policies A deterministic policy directly specifies the action the agent will take for each state. If the agent is in state s , the policy π provides a single action a . We can write this as:

$$a = \pi(s)$$

For every state s in the set of all possible states $\mathcal{S}$, the policy π outputs a specific action a from the set of available actions $\mathcal{A}(s)$. Imagine a simple robot navigating a maze. A deterministic policy might be: “If you are at position X and the path ahead is clear, move forward. If blocked, turn right.” For a given state (position X, path clear), the action (move forward) is fixed.

More simple examples of deterministic policies

- **Grid world navigation:** If the agent is not at the goal, always move right; once in the rightmost column, always move up.
- **Thermostat controller:** If the temperature is below 20°C , set heating to ON; otherwise set it to OFF.

- **Traffic light control:** If the north–south queue is longer than the east–west queue, switch to NS–green; otherwise switch to EW–green.

Stochastic Policies

Now here's where it gets interesting. What if the agent isn't sure what to do? Or what if always doing the same thing in the same situation is actually a bad idea — say, because your opponent in poker would figure out your strategy? A stochastic policy handles this by assigning a *probability* to each action rather than committing to one. We denote this as $\pi(a \mid s)$:

$$\pi(a \mid s) = \mathbb{P}[A_t = a \mid S_t = s]$$

Here, $\mathbb{P}[A_t = a \mid S_t = s]$ represents the probability that the action A_t taken at time step t is a , given that the state S_t at time t is s . The sum of probabilities for all possible actions in a given state must equal 1:

$$\sum_{a \in \mathcal{A}(s)} \pi(a \mid s) = 1 \quad \text{for all } s \in \mathcal{S}$$

Why use stochastic policies? They are particularly useful in several scenarios:

- **Exploration:** During learning, agents often need to try different actions to discover which ones yield the best rewards. A stochastic policy naturally incorporates this exploration by allowing the agent to occasionally choose actions that are not currently considered the best.
- **Uncertainty:** Sometimes the environment itself has randomness, or the agent might not be able to perfectly distinguish between states (partially observable environments). A stochastic policy can sometimes be optimal in such cases.
- **Avoiding Deterministic Cycles:** In certain situations, a deterministic policy might get stuck in suboptimal loops, which a stochastic policy might break out of.

Here's a simple illustration contrasting the two:

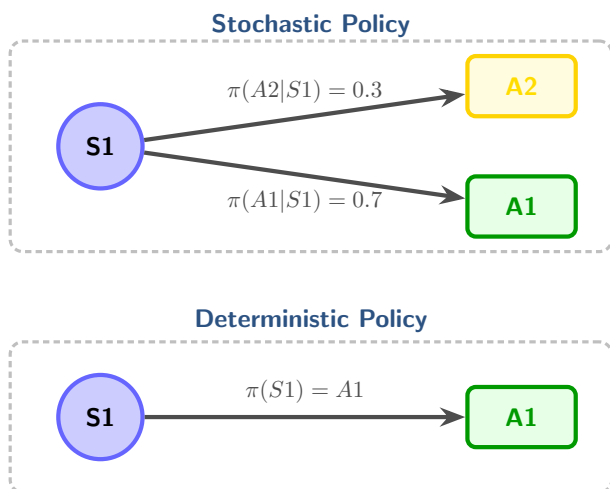


Figure 5: Stochastic policy (top): state S_1 maps to actions with probabilities. Deterministic policy (bottom): state S_1 always maps to a single action A_1 .

The Goal: Finding the Optimal Policy

The central objective in Reinforcement Learning is typically to find an optimal policy, denoted as π^* . An optimal policy is one that maximizes the expected cumulative reward the agent receives over the long run, starting from any state. Much of this course will focus on algorithms designed to learn or approximate π^* .

How policies are represented depends on the complexity of the problem. For simple environments with a small number of discrete states and actions, a policy might be stored in a lookup table. However, for problems with large or continuous state spaces (like controlling a robot based on sensor readings or playing video games from pixels), we often use function approximators, such as linear functions or neural networks, to represent the policy π . These approximators take the state representation as input and output either the action (deterministic) or the probabilities of actions (stochastic). We will cover function approximation in detail in later chapters.

In summary, the policy is the core component dictating the agent's behavior. It's the strategy the agent follows to select actions based on states, and finding the best possible strategy, the optimal policy, is the fundamental goal of most RL algorithms. Understanding the difference between deterministic and stochastic policies is essential as we move

towards exploring methods for learning these policies from experience.

The RL Workflow: Interaction Loops

The interaction between the agent and the environment forms the core of the Reinforcement Learning process. This interaction isn't a one-off event; it's a continuous cycle, often visualized as a loop, where the agent perceives, acts, and learns from the consequences. Understanding this workflow is fundamental to grasping how RL algorithms operate.

Let's break down this interaction loop step-by-step. We typically model time in discrete steps: $t = 0, 1, 2, 3, \dots$. At each time step t , the following sequence occurs:

1. **Observation:** The agent observes the current state of the environment. We denote this state as $S_t \in \mathcal{S}$, where $\mathcal{S}$ is the set of all possible states. The state representation S_t should contain all relevant information the agent needs to make a decision.
2. **Action Selection:** Based on the observed state S_t , the agent selects an action $A_t \in \mathcal{A}(S_t)$, where $\mathcal{A}(S_t)$ is the set of actions available in state S_t . This selection is governed by the agent's current policy, π . The policy $\pi(a \mid s)$ defines the probability of taking action a when in state s . For deterministic policies, it directly maps a state to an action.

Environment Transition & Reward

The environment receives the agent's action A_t . Based on S_t and A_t , two things happen:

- The environment transitions to a new state, S_{t+1} . This transition is determined by the environment's dynamics, often modeled as a probability distribution

$$p(s' \mid s, a) = \mathbb{P}(S_{t+1} = s' \mid S_t = s, A_t = a).$$

In many practical scenarios, the agent doesn't know this transition function explicitly; it learns through interaction.

- The environment provides a scalar reward signal, $R_{t+1} \in \mathcal{R}$, to the agent. This reward reflects the immediate consequence of taking action A_t in state S_t . It's important to remember the agent's goal is typically not just to maximize this immediate reward, but the cumulative reward over time (the return).

Next step The agent finds itself in state S_{t+1} , having received reward R_{t+1} . The cycle repeats for time step $t + 1$: observe S_{t+1} , select A_{t+1} , and so on.

Trajectory (experience) This ongoing cycle generates a sequence of states, actions, and rewards (often called a trajectory or experience):

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \dots$$

This sequence represents the agent’s interaction history. It’s precisely this experience data that most RL algorithms use to learn and improve the agent’s policy π , aiming to select actions that maximize the expected cumulative future reward.

We can visualize this interaction loop:

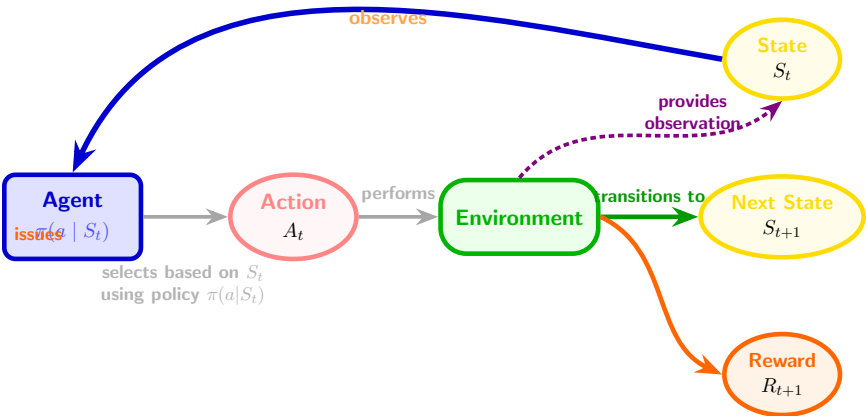


Figure 6: The RL interaction loop. The agent observes state S_t , selects action A_t via policy π , the environment responds with reward R_{t+1} and next state S_{t+1} .

Explanation of Figure 6.

- The Agent observes the current state S_t (blue arrow from State to Agent).
- The Agent selects an action based on that state using its policy $\pi(a | S_t)$ (black arrow from Agent to Action).

- The action is performed on the environment (black arrow from Action to Environment).
- The environment provides the observation back to the state (violet dashed arrow) — it updates what the world looks like.
- The environment transitions to a next state S_{t+1} (green arrow) — the world changes because of the action.
- The environment issues a reward R_{t+1} (orange arrow) — feedback on how good the action was.

This loop forms the basis for both episodic tasks, which you learned about previously, where the interaction naturally breaks down into sequences (episodes) that end in a terminal state (like winning or losing a game), and continuing tasks, where the interaction potentially goes on forever (like managing an energy grid). In episodic tasks, the loop terminates upon reaching a terminal state, and a new episode often begins from some initial state distribution. In continuing tasks, the loop runs indefinitely. For these tasks, the concept of discounting future rewards using a factor $\gamma \in [0, 1)$ becomes particularly significant, ensuring that the sum of future rewards (the return) typically remains finite and giving preference to more immediate rewards.

The data generated by this interaction loop, specifically the transitions experienced as tuples like $(S_t, A_t, R_{t+1}, S_{t+1})$, is the raw material for learning. Algorithms like Q-learning, SARSA, or Policy Gradient methods (which we will cover in later chapters) process this stream of experience to update the agent's policy or internal estimates of value, gradually guiding the agent towards better decision-making and achieving its objective of maximizing long-term cumulative reward.

Types of RL Tasks: Episodic vs Continuing

Reinforcement learning centers around an agent interacting with an environment over time, receiving observations (states) and rewards, and choosing actions based on its policy. However, not all RL problems unfold in the same way. The structure of the agent–environment interaction loop can differ, primarily falling into two categories: episodic tasks and continuing tasks. Understanding this distinction is important because it affects how an agent's goal is defined and its success is measured.

Episodic Tasks: Interactions with a Finish Line Many RL problems have interactions that naturally break down into subsequences or segments. Think of playing a game of chess, navigating a maze to find an exit, or a robot assembling a specific part. Each game, each maze run, or each assembly represents a self-contained unit of interaction. These units are called episodes.

An episodic task is characterized by the existence of one or more terminal states. When the agent reaches a terminal state, the current episode ends. After the episode concludes, the environment is typically reset, and a new episode begins, often starting from a standard initial state or a distribution of possible starting states.

Consider a simple grid where an agent needs to navigate from a starting point 'S' to a goal 'G'.

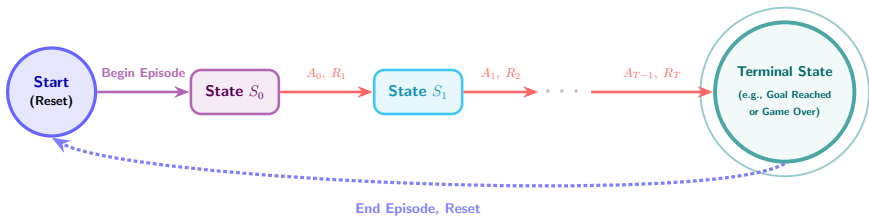


Figure 7: The flow of an episodic task. Each interaction sequence runs until a terminal state is reached, then resets for the next episode.

In episodic tasks, the agent's objective is typically to maximize the total reward accumulated over the course of a single episode. This sum of rewards within an episode is often called the return. Since each episode has a finite length, the return is well-defined. We might evaluate the agent's performance by averaging the return over many episodes.

Examples of episodic tasks include:

- **Games:** Chess, Go, Atari games (each game is an episode).
- **Maze Navigation:** Finding the exit (reaching the exit state terminates the episode).
- **Robotic Tasks:** Picking up an object and placing it in a bin (task completion is the terminal state).

Continuing Tasks: Interactions Without End

In contrast to episodic tasks, some RL problems involve interactions that do not have a natural endpoint. The agent–environment interaction

goes on continuously without breaking into identifiable episodes. Think of a system designed to continuously manage a chemical process, an algorithm making ongoing trading decisions in a financial market, or a robot powered by solar energy that needs to manage its power levels indefinitely.

These are continuing tasks. There are no terminal states. The interaction sequence can, in principle, continue forever.

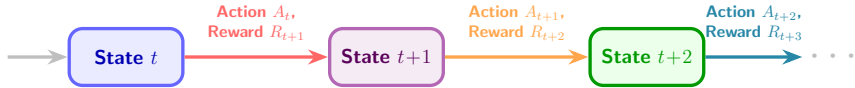


Figure 8: The flow of a continuing task. The interaction proceeds indefinitely without designated terminal states or resets.

This poses a challenge: if the interaction never ends, how do we define the total accumulated reward? Summing rewards over an infinite sequence might lead to an infinite value, making it difficult to compare different policies.

To handle this, we often introduce the concept of discounting. Instead of simply summing rewards, we calculate a discounted return, where rewards received further in the future are given less weight than immediate rewards. We use a discount factor, typically denoted by the Greek letter gamma (γ), where $0 \leq \gamma < 1$. The goal becomes maximizing the sum of discounted rewards:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

Here, G_t is the discounted return starting from time step t , and R_{t+k+1} is the reward received $k+1$ steps into the future. By using a discount factor $\gamma < 1$, we ensure that this sum remains finite even if the interaction continues forever (assuming rewards are bounded). Discounting also often makes sense intuitively: immediate rewards might be more valuable than rewards far off in the future. We will explore return calculations and discounting in much more detail in the next chapter on Markov Decision Processes.

Examples of continuing tasks include:

- **Process Control:** Maintaining temperature or pressure in a reactor.
- **Resource Management:** An operating system allocating resources to processes.

- **Algorithmic Trading:** Continuously making buy/sell decisions based on market data.
- **Climate Control:** An HVAC system maintaining a building's temperature.

Why Does This Distinction Matter?

This distinction sounds academic, but it changes *everything* about how you set up the math.

- **Goal Definition:** It changes how we define the cumulative reward or 'return' that the agent aims to maximize. In episodic tasks, it's usually the undiscounted sum of rewards per episode. In continuing tasks, it's typically the discounted sum over an infinite horizon.
- **Algorithm Formulation:** While many core RL concepts apply to both types, the mathematical formulations and sometimes the algorithms themselves are adapted based on the task type. For instance, some methods like basic Monte Carlo (which we'll see later) rely on complete episodes, making them naturally suited for episodic tasks. Temporal Difference methods are generally applicable to both.

When approaching a new problem, one of the first questions to ask is: "Does the interaction have a natural end point?" This will guide how you frame the agent's objective and select appropriate learning techniques.

★ Aha Moment: Episodic vs Continuing Changes Everything

This seemingly simple distinction determines your entire algorithm choice. Monte Carlo methods (Chapter 4) *require* episodes to end — they average returns over complete games. TD methods (Chapter 5) work step-by-step and handle both types. When you encounter a new RL problem, the first question is always: "Does this end?" A chess game ends. A thermostat never does.

Comparing RL with Other Learning Types

Reinforcement Learning involves an agent, environment, and a reward-driven interaction loop. To understand its distinct capabilities, it is

useful to situate Reinforcement Learning within the general context of machine learning. How does learning through interaction and feedback differ from other common approaches like Supervised and Unsupervised Learning? Understanding these distinctions clarifies why Reinforcement Learning is best for certain types of problems, particularly those involving sequential decision-making.

RL vs. Supervised Learning Supervised Learning (SL) is perhaps the most common form of machine learning. You typically work with a dataset containing input features and corresponding “correct” output labels. Think of image classification (input: image pixels, label: “cat” or “dog”) or predicting house prices (input: house features, label: price). The goal is to train a model that can accurately predict the label for new, unseen inputs.

The main differences from RL are:

- **Learning Signal:** In SL, the learning signal is explicit and direct. For each input, the dataset provides the ground truth label. The algorithm learns by minimizing the difference between its predictions and these known correct answers. In RL, the learning signal is a scalar reward, which is often sparse (not received after every action) and evaluative (it tells the agent how good an action was in a context, not which action was the correct one). There’s no “supervisor” providing the optimal action at each step.
- **Data:** SL relies on a pre-existing, labeled dataset. RL generates its own data through interaction with the environment. The agent’s actions influence the subsequent states it encounters and the rewards it receives. This online, interactive data generation is fundamental to RL.
- **Goal:** The goal in SL is typically generalization, meaning learning a mapping from inputs to outputs that performs well on unseen data. The goal in RL is to learn a policy (a strategy for choosing actions) that maximizes the cumulative reward over time. It’s about finding optimal behavior, not just making accurate predictions on static data.
- **Sequential Nature:** Standard SL problems often treat data points independently. RL inherently deals with sequential data where the current action affects future states and rewards. The concept of delayed consequences (an action now might lead to a reward much later)

is central to RL but less prominent in typical SL.

Imagine teaching a robot to walk. A supervised approach might involve providing detailed data on joint angles for every millisecond of a successful walk (the labels). This is often impractical or impossible to obtain. An RL approach lets the robot try different movements (actions), receive feedback based on whether it stays upright or falls (rewards/penalties), and gradually learn a walking policy through trial and error.

RL vs. Unsupervised Learning Unsupervised Learning (UL) deals with datasets that lack explicit labels. The objective is to discover hidden structures, patterns, or relationships within the data itself. Common UL tasks include clustering (grouping similar data points), dimensionality reduction (compressing data while preserving structure), and density estimation.

Here's how RL differs:

- **Learning Signal:** UL algorithms typically operate without an external reward signal. They learn from the intrinsic properties and distributions within the data. RL, by contrast, is fundamentally driven by maximizing an external reward signal provided by the environment.
- **Goal:** UL aims to find inherent structure or representations in the data. RL aims to learn a behavior or policy to achieve a specific goal defined by the reward function. There's an explicit objective in RL (maximize reward), while UL objectives are often about data description or representation.
- **Interaction:** Like SL, standard UL usually works on a fixed dataset. RL involves active interaction with an environment, where the agent's choices shape the data it collects.

Consider customer segmentation. An unsupervised approach might cluster customers based on purchasing habits found in existing sales data. An RL approach isn't directly applicable here. However, you could use RL to optimize a policy for interacting with customers (e.g., deciding which promotional offer to show next based on past responses) to maximize a reward like customer lifetime value. The goal shifts from describing the data (UL) to making optimal sequential decisions (RL).

Summary of Differences Comparison of input data, goals, learning signals, and typical tasks across different machine learning paradigms:

Paradigm	Data	Learning Signal	Goal / Typical Tasks
Supervised Learning	Fixed labeled dataset (x, y)	Explicit label error (loss)	Predict labels/values; classification, regression
Unsupervised Learning	Fixed unlabeled dataset x	No external reward (structure in data)	Discover structure; clustering, dimensionality reduction
Reinforcement Learning	Data from interaction $(S_t, A_t, R_{t+1}, S_{t+1})$	Scalar reward from environment	Maximize long-term return; control, robotics

Table 1: Comparison of the three major machine learning paradigms.

In essence, Reinforcement Learning offers a framework for solving problems where an agent must learn to make a sequence of decisions by interacting with its environment and receiving feedback in the form of rewards. This interaction-driven, goal-oriented learning process distinguishes it clearly from supervised methods that learn from labeled examples and unsupervised methods that seek structure in unlabeled data. As you proceed through this course, you’ll see how the concepts of states, actions, rewards, and policies form the basis for algorithms designed specifically for this unique learning challenge.

↔ **Bridge to RLHF: States, Actions, and Rewards in Language Models**

The mapping is direct. In RLHF, the **state** is the conversation so far (the prompt plus all tokens generated). The **action space** is the entire vocabulary — every possible next token (typically 50,000+ words and subwords). The **reward** comes from a reward model trained on thousands of human comparisons: “Was Response A or Response B more helpful, honest, and harmless?” The agent — a billion-parameter transformer — must learn a **policy** (probability distribution over tokens) that maximizes this human-preference reward. This is the same states–actions–rewards framework you just learned, operating at breathtaking scale.

1.4 Setting Up Your Python Environment for RL

Implementing and experimenting with Reinforcement Learning algorithms requires preparing a suitable workspace. Configuring a Python environment for RL involves setting up specific tools and focusing on the main libraries used throughout this course. A properly configured environment ensures that you can run the code examples and build your own RL agents smoothly.

As this course assumes familiarity with Python and basic machine learning concepts, we expect you have Python (version 3.8 or newer recommended) and the package installer `pip` already installed. If not, please refer to the official Python documentation for installation instructions.

Managing Dependencies with Virtual Environments

Before installing any packages, it's highly recommended to use a **virtual environment**. Virtual environments create isolated Python setups, preventing conflicts between project dependencies. This is standard practice in Python development.

You can create a virtual environment using Python's built-in `venv` module:

```
# Create a virtual environment named 'rl_env'
python -m venv rl_env

# Activate it (Linux / macOS)
source rl_env/bin/activate

# Activate it (Windows)
rl_env\Scripts\activate
```

Once activated, your terminal prompt will typically show the environment name. All subsequent `pip install` commands will install packages into this isolated environment.

Core Libraries

The following libraries form the foundation of our RL toolkit:

1. **NumPy** — The fundamental package for numerical computation in Python. It provides efficient array operations essential for representing states, actions, and performing mathematical calculations.

```
pip install numpy
```

2. **Gymnasium (formerly OpenAI Gym)** — The standard library for RL environments. It provides a unified interface (`reset()`, `step()`, `render()`) to interact with a wide variety of environments, from simple grid worlds to complex physics simulations.

```
pip install gymnasium
```

3. **Matplotlib** — Used for plotting learning curves, visualizing value functions, policies, and other results.

```
pip install matplotlib
```

Verifying Your Setup

Let's ensure the core components are working together. Create a simple Python script (e.g., `verify_setup.py`) with the following content:

```
import gymnasium as gym
import numpy as np

print(f"Gymnasium version: {gym.__version__}")
print(f"NumPy version: {np.__version__}")

try:
    # Create a simple environment
    env = gym.make("CartPole-v1", render_mode="rgb_array")
    print("Successfully created CartPole-v1 environment.")

    # Reset the environment to get the initial observation
    observation, info = env.reset(seed=42)
    print(f"Initial observation: {observation}")

    # Take a random action
    action = env.action_space.sample()
    print(f"Taking random action: {action}")
```

```

# Perform the action
observation, reward, terminated, truncated, info = \
    env.step(action)

print(f"Next observation: {observation}")
print(f"Reward received: {reward}")
print(f"Episode terminated: {terminated}")
print(f"Episode truncated: {truncated}")

# Close the environment (important for cleanup)
env.close()
print("Environment interaction successful.")

except Exception as e:
    print(f"An error occurred: {e}")

```

Run this script from your activated virtual environment:

```
python verify_setup.py
```

If the script runs without errors and prints output similar to the comments in the code, your basic RL environment is ready. You have successfully installed NumPy for numerical operations and Gymnasium for accessing standard RL environments. You are now equipped to start implementing the algorithms and concepts we will cover in the upcoming chapters.

Quick Reference:

Chapter 1 — RL Foundations

Core Components:

- **Agent:** The learner/decision-maker
- **Environment:** Everything the agent interacts with
- **State S_t :** Current situation snapshot
- **Action A_t :** Agent's choice at time t
- **Reward R_{t+1} :** Scalar feedback after action
- **Policy $\pi(a|s)$:** Strategy mapping states to actions

Key Formulas:

- **Return:** $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$
- **Discount:** $\gamma \in [0, 1)$ — controls how much future matters

Key Distinctions:

- **Episodic** tasks end (games, deliveries) vs **continuing** tasks run forever (stock trading)
 - **Reward** = single step feedback vs **Return** = total future payoff
 - **RL vs SL:** No labels, evaluative feedback, sequential decisions
- Coming next:** Chapter 2 formalizes these ideas as MDPs.

Chapter 1 Quiz: Foundations of Reinforcement Learning

Test your understanding of the core concepts covered in this chapter.

1. Reinforcement Learning algorithms frequently involve manipulating numerical data, such as representing states as vectors or matrices, calculating action values, and managing large batches of experience. Why is NumPy considered an essential library for these operations in an RL setup, as opposed to relying solely on standard Python lists?

- (a) NumPy simplifies the creation of graphical user interfaces for RL agent visualization.
- (b) NumPy provides efficient array objects and a comprehensive set of mathematical functions optimized for vectorized numerical computations, which are significantly faster.
- (c) NumPy includes built-in reinforcement learning algorithms that reduce development time.
- (d) NumPy is primarily used for handling string and text data within RL environments.

2. A research team is developing a new Deep Reinforcement Learning algorithm and wishes to test its performance across various environments, including classic control tasks like CartPole and more complex simulations. What is the significant advantage of using Gymnasium in their setup for this purpose?

- (a) Gymnasium offers a standardized reset and step API, providing a consistent interface for interacting with diverse environments, which simplifies benchmarking and comparison.
- (b) Gymnasium automatically tunes the hyperparameters of RL agents to achieve optimal performance in any environment.
- (c) Gymnasium includes advanced plotting tools that automatically generate learning curves and performance metrics without additional libraries.
- (d) Gymnasium provides a cloud-based platform for training RL agents at scale without local setup.

3. A reinforcement learning agent is being trained to play a complex board game. Each time a game concludes (either by win, loss, or draw), the game board is reset, and a new game begins from the starting configuration. Which of the following statements accurately describes this RL task?

- (a) It is a continuing task, as the agent's learning process is ongoing and never truly 'ends'.
- (b) It is an episodic task, and the agent's objective is to maximize a discounted sum of rewards to ensure the total return converges over infinite horizons.
- (c) It is an episodic task because each game has a clear terminal state (game end) and the environment is reset for a new interaction sequence.
- (d) It is a continuing task because the agent can play an indefinite number of games.

4. Why is it important to distinguish between episodic and continuing tasks when framing a reinforcement learning problem?

- (a) It only affects how quickly the agent learns, with continuing tasks always being slower due to their open-ended nature.
- (b) It primarily determines the complexity of the observation space and the size of the action space.

- (c) It helps in deciding whether to use supervised or unsupervised learning techniques, rather than reinforcement learning.
- (d) This distinction fundamentally influences how the agent's long-term objective (return) is defined and guides the selection of appropriate reinforcement learning algorithms.

5. A key distinction of Reinforcement Learning is its objective regarding rewards. What is the fundamental goal of an agent in a Reinforcement Learning setup?

- (a) To achieve the highest possible *reward* from each individual *action* taken at any given moment.
- (b) To learn a *policy* that maximizes the total accumulated reward over the long run, considering the sequential nature of decisions.
- (c) To discover inherent patterns and structures within the *environment's state* data without external guidance.
- (d) To predict the next *state* of the *environment* given the current *action* with perfect accuracy.

6. In the foundational Reinforcement Learning interaction loop, the agent takes an action, and the environment responds. What two pieces of information does the environment typically provide back to the agent in this loop?

- (a) A complete historical log of all past actions and rewards.
- (b) The next optimal action and a label for the current state.
- (c) The model of the environment and a set of all possible future actions.
- (d) A new state and a reward (or penalty) for the action taken.

7. A robot is learning to sort recyclable materials. The robot has sensors to identify material type and manipulators to pick and place items. A camera system provides images of the materials. The quality of sorting is evaluated by a sensor that measures the purity of the sorted bins, providing a score. In this reinforcement learning scenario, which of the following represents the agent?

- (a) The physical properties of the materials themselves (e.g., density, elasticity).
- (b) The camera system providing images of materials.
- (c) The sensor that measures the purity of the sorted bins and generates a score.
- (d) The robot's internal programming that decides where to place a detected item.

8. A Reinforcement Learning system is developed to manage a smart building's energy usage by controlling its lighting, heating, and cooling. Which element is primarily the responsibility of the *environment* in this context?

- (a) The long-term strategy for minimizing energy costs.
- (b) The historical log of energy consumption used for training.
- (c) The current temperature readings and the building's thermal dynamics.
- (d) The software module that determines when to turn lights on or off.

9. In the standard Reinforcement Learning interaction cycle, after the agent observes the current state s_t , what is the correct sequence of the next three steps?

- (a) The agent selects an action a_t , the environment processes the action and transitions to a new state s_{t+1} , and the environment returns a numerical reward r_{t+1} .
- (b) The environment transitions to a new state s_{t+1} , the agent selects an action a_t , and the environment returns a numerical reward r_{t+1} .
- (c) The environment returns a numerical reward r_{t+1} , the agent selects an action a_t , and the environment transitions to a new state s_{t+1} .
- (d) The environment processes the action and transitions to a new state s_{t+1} , the environment returns a numerical reward r_{t+1} , and the agent selects an action a_t .

10. A reinforcement learning agent is designed to control a robotic arm that must pick up specific items and place them in a designated bin. Which reward function design is most likely to lead to efficient and accurate item placement?

- (a) +50 for successfully placing an item in the correct bin, -1 for each unit of energy consumed by the robotic arm, and -10 if an item is dropped.
- (b) +1 for successfully picking up an item, and +1 for moving the arm.
- (c) -5 for incorrect placement, and +1 for any movement of the arm towards the bin.
- (d) +100 for completing the task (all items placed), and 0 for all intermediate actions.

11. In the context of a stochastic policy, what does the expression $\sum_{a \in \mathcal{A}(s)} \pi(a | s) = 1$ signify for any given state s ?

- (a) The policy will always choose a single optimal action with probability 1 in state s .
- (b) The expected cumulative reward from state s is always 1.
- (c) The agent is guaranteed to reach an absorbing state from state s .
- (d) The sum of probabilities for all possible actions in state s must equal 1.

12. A robotic arm is being trained to pick up irregularly shaped objects from a conveyor belt, where the exact position and orientation of objects can vary slightly and are sometimes obscured by glare. Which type of policy would generally be more effective for this learning task, and why?

- (a) A deterministic policy, as it is simpler to implement and requires less computational power.
- (b) A deterministic policy, because it provides consistent actions for specific states, leading to faster convergence.
- (c) A stochastic policy, because it allows for exploration by trying different actions and adapting to the uncertainty in object presentation.

- (d) Either type, as the choice of policy has no significant impact on learning performance in uncertain environments.

13. What is a distinguishing characteristic of the 'learning signal' used in Reinforcement Learning, as compared to Supervised and Unsupervised Learning?

- (a) It provides direct, step-by-step instructions on the exact action to take in every situation.
- (b) It is a scalar reward, often sparse and evaluative, indicating how good an action was in a context rather than providing the optimal action.
- (c) It consists of explicit, ground truth labels for every input feature.
- (d) It is derived from discovering intrinsic patterns and distributions within unlabeled data.

14. A significant challenge in certain machine learning problems is dealing with actions that do not immediately yield a reward or penalty, but whose consequences only become apparent much later. Which machine learning paradigm is inherently designed to handle these 'delayed consequences'?

- (a) Unsupervised Learning, as it often identifies long-term trends and hidden structures within complex datasets.
- (b) All three paradigms equally, as complex real-world problems often involve intricate dependencies.
- (c) Supervised Learning, as models must generalize from training data to unseen data, which can have delayed impact on performance.
- (d) Reinforcement Learning, where an action taken at one step can affect future states and rewards, requiring the agent to learn to attribute later rewards back to earlier actions.

Chapter 1 Quiz — Answer Key

Q	Ans	Explanation
1	(b)	NumPy provides efficient array objects and vectorized operations, far faster than Python lists for RL computations.
2	(a)	Gymnasium's standardized <code>reset()</code> / <code>step()</code> API enables consistent benchmarking across diverse environments.
3	(c)	Each game ends in a terminal state (win/loss/draw) and resets — the hallmark of an episodic task.
4	(d)	This distinction determines how the return is defined and guides algorithm selection (e.g., discounting for continuing tasks).
5	(b)	The goal is to learn a policy maximizing <i>cumulative</i> long-term reward, not just immediate reward.
6	(d)	The environment provides a new state s_{t+1} and a scalar reward r_{t+1} after each action.
7	(d)	The agent is the decision-maker — the robot's programming that decides placement. Sensors are part of the environment.
8	(c)	The environment includes temperature readings, thermal dynamics, and everything outside the agent's control.
9	(a)	Correct sequence: agent selects $a_t \rightarrow$ environment transitions to $s_{t+1} \rightarrow$ returns reward r_{t+1} .
10	(a)	Balances reward for correct placement, penalizes energy waste and drops — rich feedback shapes efficient behavior.
11	(d)	Probabilities over all actions in a state must sum to 1 — a valid probability distribution requirement.
12	(c)	Stochastic policies explore different strategies and handle uncertainty from varying object positions and glare.
13	(b)	RL's reward is evaluative (how good was the action), not instructive (what the correct action was).
14	(d)	RL handles delayed consequences via temporal credit assignment across sequential decisions.

Chapter 2: Markov Decision Processes (MDPs)

✓ Learning Objectives:

Chapter 2 After this chapter, you will be able to:

1. Define a **Markov Decision Process** (S, A, P, R, γ) and explain each component.
2. Write **state transition probabilities** $p(s', r|s, a)$ and verify they sum to 1.
3. Compute **returns** G_t with discounting for both episodic and continuing tasks.
4. Define **state-value** $V^\pi(s)$ and **action-value** $Q^\pi(s, a)$ functions.
5. Derive and apply the **Bellman expectation equations** for V^π and Q^π .
6. State the **Bellman optimality equations** and explain why they characterize V^* and Q^* .
7. Find an **optimal policy** given optimal value functions.

In Chapter 1, we talked about agents, environments, rewards, and policies in plain English. That was the intuition. Now we need to be precise — because if you can't write it down mathematically, you can't program a computer to solve it. This chapter introduces **Markov Decision Processes (MDPs)**: a clean, elegant mathematical framework that captures everything about a sequential decision problem in five symbols. Once you see how it works, you'll realize that every RL problem you'll ever encounter — from a chess game to training ChatGPT — is just an MDP in disguise.

You will learn to define the key elements of an MDP:

- The set of possible states ($\mathcal{S}$) the environment can be in.
- The set of actions ($\mathcal{A}$) the agent can take.
- The state transition probabilities (P), defining the environment dynamics: $P(s'|s, a)$.

- The reward function (R), specifying immediate feedback: $R(s, a, s')$.
- The discount factor (γ), managing the importance of future rewards.

We will explore how an agent's behavior is defined by a policy (π) and how to evaluate the “goodness” of states and state-action pairs using value functions (V^π and Q^π). This leads to understanding the goal of RL within the MDP framework: finding an optimal policy (π^*) that maximizes expected cumulative rewards.

Sections

- 2.1 Modeling Sequential Decision Making
- 2.2 Formal Definition of an MDP
- 2.3 State Transition Probabilities
- 2.4 Reward Functions
- 2.5 Return: Cumulative Future Rewards
- 2.6 Discounting Future Rewards
- 2.7 Policies and Value Functions (V_π, Q_π)
- 2.8 Finding Optimal Policies

2.1 Modeling Sequential Decision Making

Here's what makes RL fundamentally different from, say, classifying images: every decision changes what happens next. You're not answering independent questions — you're playing a game where each move opens some doors and closes others. Think about teaching a robot to navigate a building, training an algorithm to play Go, or optimizing inventory management in a supply chain. These problems share several characteristics:

- **Sequential Nature:** Decisions are not made in isolation. An action taken now directly influences the situations (states) the agent encounters next, potentially opening up or closing off future opportunities. Turning left at an intersection means you won't immediately see what's down the right path.

- **Delayed Consequences:** The impact of an action might not be fully realized immediately. A short-term gain could lead to a long-term loss, and vice versa. Sacrificing a piece in chess might grant a positional advantage that only pays off many moves later. Conversely, maximizing immediate profit might deplete a resource needed for future growth.
- **Stochasticity:** The environment's response to an action might involve randomness. Requesting a web resource might succeed or fail due to network conditions. A robot's command to move forward might result in slightly different actual movements due to wheel slip-page. An opponent in a game introduces uncertainty through their actions.

To develop agents that can navigate these complexities effectively, we need more than just the basic agent-environment loop concept. We require a formal way to describe the problem itself, capturing the states the agent can be in, the actions it can take, how the state changes in response to actions (the environment's dynamics), and the rewards received along the way. This formal description allows us to reason rigorously about the problem and develop algorithms capable of finding optimal strategies that balance immediate rewards with long-term objectives.

Consider a simplified representation of navigating between rooms:

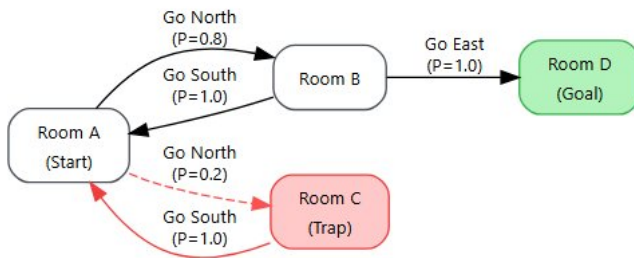


Figure 9: Room navigation example. Moving North from Room A usually leads to Room B ($P = 0.8$), but sometimes leads to the Trap ($P = 0.2$). Actions have probabilistic outcomes.

An agent needs to reach Room D from Room A. Moving North from A usually leads to B, but sometimes (with probability 0.2) leads to a Trap (Room C). Actions have probabilistic outcomes and lead to different subsequent states and potential rewards.

In this scenario, the agent's location represents the *state*. Available *actions* depend on the state (e.g., 'Go North', 'Go East'). The environment's dynamics are captured by the *transition probabilities* (like the 80%/20% split when moving North from Room A). Reaching the Goal yields a positive *reward*, while entering the Trap might give a negative reward.

This necessity to formally structure problems involving sequential decision-making under uncertainty leads us directly to **Markov Decision Processes (MDPs)**. MDPs provide the standard mathematical framework used throughout Reinforcement Learning. They offer a precise way to define the components of the environment and the interaction, enabling the development and analysis of learning algorithms. In the sections that follow, we will break down the formal definition of an MDP and its core components: states, actions, transition probabilities, rewards, and the discount factor. Understanding MDPs is fundamental to understanding how RL agents learn optimal behavior in complex, dynamic environments.

★ Running Example — DeliBot as a Formal MDP

Let's map DeliBot to the formal MDP tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$:

- $\mathcal{S}$: (floor, room, battery_level, has_package) — e.g., (2, Hallway, high, Yes)
- $\mathcal{A}$: {north, south, east, west, elevator, pick_up, deliver, charge}
- P : Moving east from Hallway usually works ($p = 0.9$), but sometimes DeliBot drifts ($p = 0.1$ end up in adjacent room) due to wheel slip
- R : +20 for delivery, -1 per step, -5 for wall bump, -50 for dead battery
- $\gamma = 0.95$: DeliBot cares about future rewards but slightly prefers sooner deliveries

This is a **stochastic** MDP — even with the same action, DeliBot might end up in different states due to wheel slip. This is exactly why we need probability in our transition function!

2.2 Formal Definition of an MDP

A Markov Decision Process is formally defined as a tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where each component captures an essential aspect of the decision-making problem:

- **$\mathcal{S}$ — State space:** The set of all possible states the environment can be in. A state $s \in \mathcal{S}$ is a complete description of the current situation.
- **$\mathcal{A}$ — Action space:** The set of all possible actions available to the agent. In some formulations, the available actions may depend on the current state: $\mathcal{A}(s)$.
- **P — Transition function:** The probability of moving to state s' given the agent is in state s and takes action a :

$$P(s' | s, a) = \mathbb{P}(S_{t+1} = s' | S_t = s, A_t = a)$$

This function defines the “rules” or dynamics of the environment.

- **R — Reward function:** The expected immediate reward received after transitioning from state s to state s' due to action a :

$$R(s, a, s') = \mathbb{E}[R_{t+1} | S_t = s, A_t = a, S_{t+1} = s']$$

Sometimes simplified as $R(s, a)$ or even $R(s)$ depending on the formulation.

- **γ — Discount factor:** A value $\gamma \in [0, 1)$ that determines the present value of future rewards. A reward received k steps in the future is worth γ^k times what it would be worth if received immediately.

The discount factor serves several purposes:

1. **Ensures Convergence:** In tasks that could potentially run forever (continuing tasks), discounting ensures that the infinite sum of future rewards converges to a finite value, assuming rewards are bounded. This makes the objective mathematically tractable.
2. **Models Preference:** It allows us to model a preference for rewards obtained sooner rather than later. If $\gamma = 0$, the agent only cares about the immediate reward (it’s “myopic”). If γ is close to 1, the agent is “far-sighted” and values future rewards almost as much as immediate ones.

3. **Handles Uncertainty:** Discounting can implicitly account for uncertainty about the future. There might be a chance the process terminates unexpectedly, making distant future rewards less certain to be obtained.

The agent's objective is typically formulated as maximizing the *expected discounted return*, which we will define more formally later. It's the sum of discounted rewards obtained over the future.

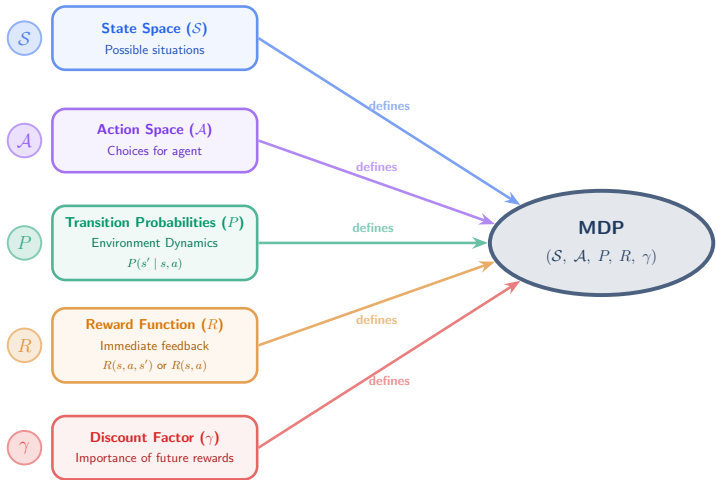


Figure 10: The five components that formally define a Markov Decision Process.

Together, these five components pin down everything about the environment — its dynamics, its feedback signal, and the long-term trade-offs the agent faces. The goal is to discover a policy π — a rule for choosing actions given states — that earns as much total discounted reward as possible over time.

↪ **Bridge to RLHF: Language Model Fine-Tuning Is an MDP**

When OpenAI fine-tunes GPT with RLHF, they are solving an MDP: $\mathcal{S}$ = all possible token sequences (conversation history), $\mathcal{A}$ = the vocabulary (next token choices), P = deterministic (appending the chosen token to the sequence), R = the score from a reward model trained on human preferences, and $\gamma \approx 1$ (we care about the quality of the *entire* response). The formal MDP tuple you just learned isn't an abstraction that only applies to grid worlds — it is the precise mathematical object that governs how ChatGPT, Claude, and Gemini are trained. Ev-

ery concept from here on — Bellman equations, value functions, policy optimization — applies directly.

★ Real-World Spotlight:

Self-Driving Cars Are MDPs on Wheels Waymo's autonomous vehicles operate as a massive MDP: **states** include sensor readings (LIDAR, cameras, GPS), surrounding vehicle positions, and road conditions; **actions** are steering angle, acceleration, and braking; **transitions** follow physics (partly stochastic due to other drivers); **rewards** balance safety (no collisions), comfort (smooth ride), and efficiency (reaching the destination). Every second, the car's RL-trained policy evaluates $Q^*(s, a)$ across thousands of possible maneuvers. The MDP you just learned to define on paper is running at 10 Hz inside a 2-ton vehicle.

2.3 State Transition Probabilities

The transition function $P(s' | s, a)$ is the engine that drives the MDP forward. It answers the most natural question in the world: *if I'm here and I do this, what happens next?* More precisely: how likely is each possible next state s' ? The answer isn't always certain — the same action in the same state might lead to different outcomes. That's what makes it a *probability*.

Since P defines a probability distribution over next states for each state-action pair, it must satisfy: Formally, the reward function R spec-

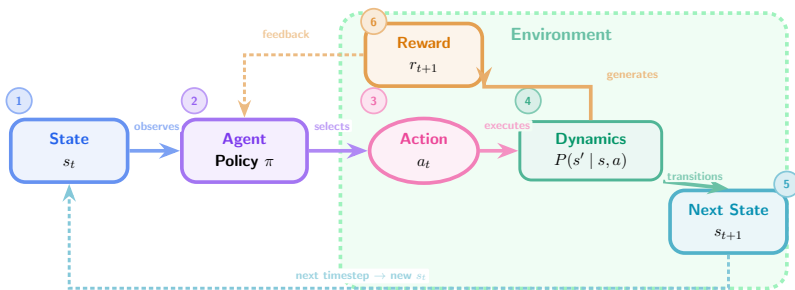


Figure 11: The environment provides a reward r_{t+1} along with the next state s_{t+1} after the agent takes action a_t in state s_t . This reward signal is the primary feedback used by the agent to learn a desirable policy.

ifies this immediate feedback. It can take different forms depending on what information the reward depends on:

1. **State, Action, Next State:** The most general form is $R(s, a, s')$, depending on state s , action a , and resulting state s' .

The expected reward is:

$$\begin{aligned} r(s, a) &= \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a] \\ &= \sum_{s'} P(s' \mid s, a) R(s, a, s') \end{aligned}$$

Here, R_{t+1} is the random variable representing the reward at time $t+1$, and $P(s' \mid s, a)$ is the state transition probability we discussed earlier.

2. **State and Action:** Often, the reward is simplified and only depends on the state s and the action a taken, denoted as $R(s, a)$.
3. **State Only:** In some scenarios, particularly goal-based tasks, the reward might only depend on reaching certain states, denoted $R(s')$. For example, a large positive reward for reaching a target state and zero otherwise.

The reward function is central to RL because it implicitly defines what the agent should strive to achieve. The agent's objective isn't necessarily to maximize the *immediate* reward r_{t+1} , but rather the *cumulative* reward over time, which we previously defined as the **return** (G_t). The discount factor γ plays a significant role here, balancing the importance of immediate rewards versus future rewards when calculating the return.

► Intuition: Discounted Return

Imagine you are promised \$100 today vs. \$100 one year from now. Most people prefer the money today — it's certain, you can invest it, and who knows what happens in a year. Discounting in RL works the same way: a reward received now is worth more than the same reward received later. The discount factor γ controls how "impatient" the agent is.

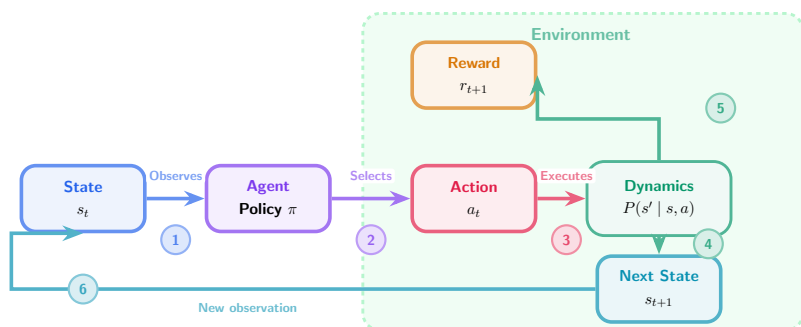


Figure 12: MDP interaction loop: the agent observes state s_t , selects action a_t , the environment transitions to s_{t+1} and emits reward r_{t+1} .

◆ Code: Computing Discounted Return in Python

```
def discounted_return(rewards, gamma=0.9):
    G = 0
    for r in reversed(rewards):
        G = r + gamma * G
    return G

# Example: rewards = [10, -5, 10], gamma = 0.5
print(discounted_return([10, -5, 10], 0.5))
# Output: 10.0
```

Worked Example: Discounted Return Step by Step

Given: $R_1 = 10$, $R_2 = -5$, $R_3 = 10$, $\gamma = 0.5$

Calculate $G_0 = R_1 + \gamma R_2 + \gamma^2 R_3$:

k	R_{t+k+1}	γ^k	Discounted	Running Sum
0	+10	1.0	$10 \times 1.0 = 10.0$	10.0
1	-5	0.5	$-5 \times 0.5 = -2.5$	$10.0 - 2.5 = 7.5$
2	+10	0.25	$10 \times 0.25 = 2.5$	$7.5 + 2.5 = 10.0$

Result: $G_0 = 10.0 - 2.5 + 2.5 = 10.0$

Notice how the -5 penalty at step 2 is halved by discounting, and the $+10$ at step 3 is reduced to just 2.5. The further into the future, the less weight each reward carries.

How the Reward Flow Works (Step by Step):

1. **State (s_t):** The process begins with the current state of the environment — this is what the agent can observe right now.
2. **Agent (Policy π):** The agent looks at the current state and uses its policy to decide what to do next.
3. **Action (a_t):** Based on the policy, the agent picks a specific action to perform in the environment.
4. **Dynamics $P(s' | s, a)$:** The environment receives the action and processes it according to its internal rules (transition probabilities).
5. **Next State (s_{t+1}):** The environment moves to a new state as a result of the action. This new state becomes the starting point for the next step.
6. **Reward (r_{t+1}):** At the same time, the environment sends back a numerical reward telling the agent how good or bad that transition was.

The dashed arrows show the *feedback loops*: the reward signal flows back to the agent to help it improve its policy over time, and the next state becomes the new current state, restarting the cycle. This loop repeats at every time step throughout the agent's lifetime.

Designing Reward Functions

Specifying the reward function is a form of **reward engineering**. It's how we translate a high-level goal into a concrete signal the agent can use for learning. This isn't always straightforward:

- **Sparse Rewards:** In many problems, rewards might be zero for most steps, with a non-zero reward only occurring upon reaching a terminal state (e.g., winning or losing a game). Learning can be slow with sparse rewards as feedback is infrequent.
 - *Example:* Chess: +1 for checkmate, -1 for being checkmated, 0 for all other moves.
- **Dense Rewards:** Providing smaller, more frequent rewards can guide the agent more effectively. However, poorly designed dense rewards can lead to unintended agent behavior, where the agent optimizes for intermediate rewards rather than the true objective.

- *Example:* Robot navigating a maze: +10 for reaching the goal, −0.1 for each step taken (encourages finding shorter paths), −5 for hitting a wall.

The **Reward Hypothesis** is a fundamental concept in RL: it posits that all goals and purposes can be characterized as the maximization of the expected value of the cumulative sum of a received scalar signal (the reward). Getting the reward function right is therefore essential for successful reinforcement learning. It must accurately reflect the actual goal of the task. If the reward function incentivizes behavior different from what you truly want, the agent will likely learn that unintended behavior.

Understanding the reward function is fundamental before moving on to how agents actually use these rewards (along with state transitions) to evaluate policies and find optimal ways to behave. The interaction between the reward signal, the environment's dynamics, and the agent's policy is the core of the learning process in MDPs.

↪ Bridge to Deep RL: From Tables to Neural Networks

Notice that computing $P(s' \mid s, a)$ and $R(s, a)$ requires enumerating every state-action pair. For the Recycling Robot with 2 states and 3 actions, this is trivial. But what about a self-driving car where the state is a camera image? Or a robotic hand where the state is the position and velocity of every joint? You cannot write down these transition tables — they are infinite-dimensional. In Part II, you will learn how **function approximation** (Chapter 6) and **deep neural networks** (Chapter 7) let us *learn* these functions from data rather than specifying them. This is the leap that makes RL practical for real-world problems.

★ Real-World Spotlight:

Netflix's Multi-Armed Bandit — Returns in Real Time Netflix personalizes not just what you watch, but which *thumbnail image* you see for each title. This is a return-maximization problem: the “state” is your viewing history, the “action” is which thumbnail to display, and the “reward” is whether you click. Netflix runs millions of these micro-experiments daily, computing estimated returns for each thumbnail variant. The company reported that personalized thumbnails increased engagement by over 20%

— a direct consequence of optimizing cumulative returns over sequential interactions, exactly the concept you just learned.

2.5 Return: Cumulative Future Rewards

Reinforcement Learning involves an agent interacting with its environment over a sequence of time steps. At each step t , the agent observes a state S_t , chooses an action A_t , receives a reward R_{t+1} , and transitions to a new state S_{t+1} . But here's the thing: a smart agent doesn't just grab the nearest cookie. What if eating that cookie means you miss the five-course dinner? The agent's real objective isn't to maximize the reward *right now* — it's to maximize the total reward over its entire lifetime. Sometimes the best thing to do right now looks terrible in the short term but pays off enormously later. (Ask any chess player who sacrifices a queen.)

To capture this idea of long-term cumulative reward, we introduce the **Return**. The return, denoted G_t , is the total reward an agent expects to accumulate starting from time step t onwards. The precise way we calculate the return depends on whether the task is episodic or continuing.

Return in Episodic Tasks

Episodic tasks are those that have a natural ending point. Think of games like chess or Pac-Man, where each game eventually terminates. In these tasks, the sequence of states, actions, and rewards forms an episode: $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T, S_T$, where S_T is a special terminal state.

For an episodic task, the return G_t starting from time step t is simply the sum of all rewards received from step $t + 1$ until the end of the episode at step T :

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_T$$

This can be written more compactly as:

$$G_t = \sum_{k=0}^{T-t-1} R_{t+k+1}$$

Here, T is the final time step of the episode. The goal of the agent in an episodic task is to choose actions that maximize the expected value of this finite sum of future rewards for each episode.

Return in Continuing Tasks and the Need for Discounting

Continuing tasks, on the other hand, do not have a terminal state and could potentially go on forever. Examples include controlling a robot performing an ongoing maintenance task or managing an investment portfolio. If we simply summed the rewards as we did for episodic tasks, the return G_t could easily become infinite ($T \rightarrow \infty$), making it difficult to compare different strategies. How can we meaningfully define a cumulative reward that doesn't diverge?

This is where a beautiful trick saves us: **discounting**. We introduce a number γ (gamma) between 0 and 1 that shrinks future rewards. A reward tomorrow is worth γ times what it would be worth today. A reward the day after? γ^2 . And so on. It's exactly like compound interest running in reverse — the further away a reward is, the less it's worth to us now.

The discounted return G_t is defined as the sum of future rewards, where each reward R_{t+k+1} is multiplied by γ^k :

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Why does this seemingly artificial trick solve the infinity problem? Two beautiful reasons:

- **Mathematical Convergence:** If the reward signal is bounded (which is often the case) and $\gamma < 1$, this infinite sum converges to a finite value. This allows us to compare returns even for infinite sequences.
- **Behavioral Preference:** Discounting often makes intuitive sense. A reward received sooner is generally preferable to the same reward received later. The factor γ controls this preference:
 - If γ is close to 0, the agent becomes “myopic,” focusing heavily on immediate rewards.
 - If γ is close to 1, the agent becomes “far-sighted,” taking future rewards into account more strongly.

A Unified View

Interestingly, the discounted return formula $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$ can be seen as a general case. For episodic tasks, we can consider them to end by transitioning to an absorbing terminal state that only yields rewards of 0 from then on. The formula still applies, and if $\gamma = 1$, it simplifies back to the undiscounted sum $G_t = \sum_{k=0}^{T-t-1} R_{t+k+1}$. However, using $\gamma < 1$ is common even in episodic tasks to encourage finding the terminal state faster or to ensure convergence in certain algorithms.

Therefore, the fundamental goal in most RL problems framed within MDPs is to find a policy (a way of choosing actions) that maximizes the expected discounted return $\mathbb{E}[G_t]$ from each state S_t . This quantity, the expected return under a specific policy, is precisely what we will define as the value of a state or state-action pair in subsequent sections. Understanding the return G_t is the first step towards evaluating how good different situations and actions are in the long run. We will explore the role of the discount factor γ in more detail next.

2.6 Discounting Future Rewards

In the previous section we introduced the discounted return G_t and saw *why* discounting is needed. Now let's look more carefully at *how* the discount factor γ shapes the agent's behavior in practice.

Let's break down the mechanics step by step:

X Common Misconception: $\gamma = 0.9$ and $\gamma = 0.99$ Are “Almost the Same”

This is a surprisingly common error! While 0.9 and 0.99 look similar, their effect on how far the agent “looks ahead” is dramatically different:

- With $\gamma = 0.9$: a reward 10 steps away is worth $0.9^{10} = 0.35$ (35% of its face value)
- With $\gamma = 0.99$: a reward 10 steps away is worth $0.99^{10} = 0.90$ (90% of its face value!)

The “effective horizon” (how many steps ahead the agent meaningfully considers) is approximately $\frac{1}{1-\gamma}$. So $\gamma = 0.9$ gives an effective horizon of ~ 10 steps, while $\gamma = 0.99$ gives ~ 100 steps. That is a **10× difference** from changing just one digit!

★ Running Example — DeliBot and Discounting

DeliBot uses $\gamma = 0.95$ (effective horizon ≈ 20 steps). This means:

- A +20 delivery reward 5 steps away is worth $0.95^5 \times 20 \approx 15.5$ now.
- A +20 delivery reward 20 steps away is worth $0.95^{20} \times 20 \approx 7.2$ now.

So DeliBot prefers the closer delivery — but doesn't completely ignore the farther one. If we set $\gamma = 0.5$, DeliBot would be so myopic it might never cross the building to make a high-value delivery. If $\gamma = 0.99$, DeliBot would happily wander the building for 50 steps to reach a single package.

- The reward received one step later, R_{t+1} , is added directly (multiplied by $\gamma^0 = 1$).
- The reward received two steps later, R_{t+2} , is multiplied by γ .
- The reward received three steps later, R_{t+3} , is multiplied by γ^2 .
- And so on; the reward received $k+1$ steps later, R_{t+k+1} , is multiplied by γ^k .

Since γ is less than or equal to 1, γ^k gets smaller as k increases. This means rewards received further in the future contribute less to the total discounted return G_t .

Why Discount?

Discounting serves several important purposes:

1. **Mathematical Convenience:** In continuing tasks, discounting ensures that the infinite sum defining G_t converges to a finite value, provided the rewards are bounded. If the reward at any step is guaranteed to be no larger than some value $R_{\max}$, then the return G_t is guaranteed to be less than or equal to $\frac{R_{\max}}{1-\gamma}$ (using the formula for the sum of an infinite geometric series). This makes comparing different policies mathematically tractable.

2. **Modeling Preference for Near-Term Rewards:** Often, receiving a reward sooner is preferable to receiving the same reward later. Think about financial interest rates or biological imperatives for immediate survival. Discounting naturally models this preference.
3. **Handling Uncertainty:** The future is inherently uncertain. The agent's model of the environment might be imperfect, or the environment itself might change unexpectedly. Applying a discount factor implicitly accounts for this increasing uncertainty over longer time horizons; rewards far in the future are less certain and thus valued less.

★ Real-World Spotlight:

AlphaGo's Discount Factor — Patience That Beat a World Champion When DeepMind trained AlphaGo, the discount factor was set to $\gamma = 1$ (no discounting) because Go has a definite end. Every future move was weighted equally. This “infinite patience” is what enabled AlphaGo's famous Move 37 — a stone placement that looked terrible in the short term but created a winning advantage 50 moves later. A myopic agent ($\gamma \approx 0$) would never have found it. The choice of γ literally shaped the kind of intelligence the system could develop.

Choosing the Value of Gamma (γ)

The choice of γ significantly impacts the agent's behavior:

- **If $\gamma = 0$:** The agent becomes completely “myopic.” It only cares about maximizing the immediate reward R_{t+1} . It ignores all future consequences of its actions. The return is simply $G_t = R_{t+1}$.
- **If γ is close to 1** (e.g., 0.99): The agent is “far-sighted.” Future rewards are valued almost as highly as immediate rewards. The agent considers the long-term consequences of its actions more heavily.
- **If γ is between 0 and 1:** The agent balances immediate and future rewards. Smaller values prioritize short-term gain, while larger values emphasize long-term accumulation.

The specific value of γ is often treated as a hyperparameter of the problem setup, chosen based on the nature of the task. For example,

in financial applications, γ might relate to prevailing interest rates. In episodic tasks, $\gamma = 1$ (no discounting) is sometimes used if the episodes are guaranteed to terminate, although using $\gamma < 1$ can often lead to faster learning.

Decay of the Discount Factor

The following chart illustrates how the discount factor γ^k diminishes the value of rewards over future time steps k for different values of γ :

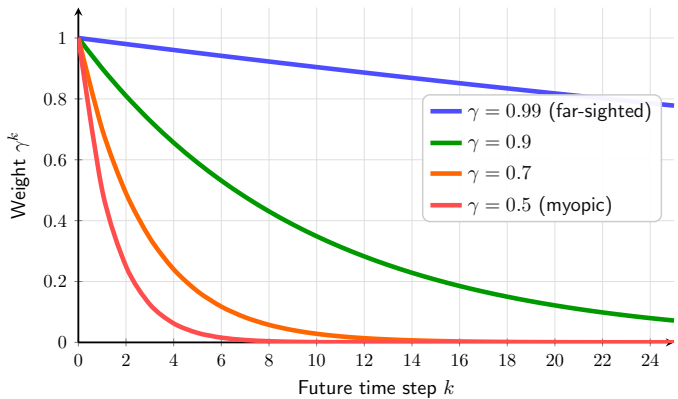


Figure 13: The weight given to a reward received k steps in the future decreases exponentially with k . A higher γ leads to slower decay, valuing future rewards more.

Reading the Chart (Step by Step):

- **Blue line ($\gamma = 0.99$, far-sighted):** The weight drops very slowly. Even a reward 25 steps in the future still keeps about 78% of its value. The agent cares almost equally about near and distant rewards.
- **Green line ($\gamma = 0.9$):** The weight decreases moderately. A reward 10 steps away is worth about 35% of an immediate reward, and by 25 steps it's only about 7%. A good balance between short-term and long-term thinking.
- **Orange line ($\gamma = 0.7$):** The weight drops quickly. After just 5 steps, a reward is worth only about 17% of its original value. The agent is mostly focused on the near future.
- **Red line ($\gamma = 0.5$, myopic):** The weight decays very rapidly. A reward just 3 steps away is already worth only 12.5% of an immediate reward. The agent barely looks ahead at all.

Key takeaway: The higher the γ , the more the agent values future rewards. Choosing the right γ is about deciding how far ahead the agent should “look” when making decisions.

In Reinforcement Learning, the ultimate objective is typically to find a policy π that maximizes the expected discounted return from each state. The concept of discounting is therefore fundamental to defining the very goal we are trying to achieve and enables the formalisms, like the value functions we’ll discuss next, that allow us to solve MDPs.

↪ Bridge to the Frontier: From γ to the KL Penalty — and Beyond

The discount factor γ controls how much the agent cares about the distant future. In RLHF (Part III), a closely related idea appears: the **KL divergence penalty**, which prevents the fine-tuned model from drifting too far from its pre-trained behavior. Both γ and the KL penalty serve as “regularizers” — they keep the agent grounded. But here is the frontier twist: in 2025, researchers building reasoning models with **GRPO** (Part V, Chapter 26) found that you can *remove* the KL penalty entirely when using verifiable rewards, because there is no reward model to over-optimize against. The simple idea of “how much do we discount?” that you learned here echoes through every generation of RL algorithms — and understanding it now will help you see why each new method makes the choices it does.

2.7 Policies and Value Functions

We now have all five ingredients of an MDP. But knowing the rules of a game doesn’t tell you how to win. The next question is the big one: *how does the agent decide what to do?* And once it has a strategy, *how do we score it?* This is where **policies** and **value functions** come in — and they are, without exaggeration, the most important concepts in all of reinforcement learning.

★ Aha Moment: Two Questions That Drive All of RL

Every RL algorithm answers exactly two questions: (1) **Prediction:** “How good is my current strategy?” — this is policy evaluation, computing V^π . (2) **Control:** “How can I find a better strategy?” — this is policy improvement. The entire field, from tabular DP to Deep RL to RLHF, is variations on answering these

two questions.

Policies

A **policy** defines the agent's behavior — it specifies what action the agent will take in each state. Formally, a policy π is a mapping from states to actions (or to probability distributions over actions).

There are two types of policies:

- **Deterministic Policy:** The agent always takes the same action in a given state. It is a direct mapping:

$$a = \pi(s)$$

For example, “in state s_3 , always go left.”

- **Stochastic Policy:** The agent selects actions according to a probability distribution. For each state s and action a , the policy gives the probability of taking that action:

$$\pi(a \mid s) = P(A_t = a \mid S_t = s)$$

For example, “in state s_3 , go left with probability 0.7 and go right with probability 0.3.” Note that for each state, the probabilities must sum to 1: $\sum_{a \in \mathcal{A}(s)} \pi(a \mid s) = 1$.

Stochastic policies are more general — a deterministic policy is simply a special case where all probability is concentrated on one action. In many RL algorithms, the agent starts with a stochastic policy (to explore different actions) and gradually refines it towards a deterministic optimal policy.

State-Value Function $V^\pi(s)$

Given a policy π , the **state-value function** (or simply the *value function*) tells us *how good it is to be in a particular state* when following policy π . “How good” is measured by the expected return starting from that state.

Formally, the state-value function $V^\pi(s)$ is defined as:

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right]$$

In plain language: $V^\pi(s)$ is the expected total discounted reward the agent will receive if it starts in state s and follows policy π from that point onward. A state with a high value is a “good” state to be in (under that policy), because on average, the agent can expect to accumulate a lot of reward from there.

Action-Value Function $Q^\pi(s, a)$

The **action-value function** (also called the *Q-function*) is similar, but it tells us *how good it is to take a specific action a in a specific state s* and then follow policy π afterwards.

Formally:

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right]$$

The key difference from $V^\pi(s)$: with the state-value function, the action is chosen by the policy π . With $Q^\pi(s, a)$, we specify the first action a (which might differ from what π would choose), and then follow π from the next state onward.

The Q-function is particularly important in RL because if we know $Q^*(s, a)$ (the optimal Q-function), the agent can simply pick the action with the highest Q-value in each state — no need to explicitly know the transition probabilities or rewards!

★ Aha Moment: Q-Values Are the Secret to Model-Free Control

Here is the deep insight: if you know $V^*(s)$, you *still* need the transition model to pick the best action (you’d need to compute $\sum p(s', r | s, a) [r + \gamma V^*(s')]$ for each action). But if you know $Q^*(s, a)$, choosing the best action is trivial: just pick $\arg \max_a Q^*(s, a)$. No model needed. This is why nearly every practical RL algorithm — Q-learning, DQN, SAC — estimates Q-values, not V-values. This single insight explains the entire trajectory of modern RL.

Relationship Between V^π and Q^π

The state-value function and action-value function are closely related. Since $V^\pi(s)$ represents the expected return when the agent follows π (which means it selects actions according to π), we can express it as:

$$V^\pi(s) = \sum_{a \in \mathcal{A}(s)} \pi(a | s) \cdot Q^\pi(s, a)$$

This says: the value of a state is the weighted average of the Q -values of all actions, weighted by the probability of taking each action under the policy.

The Bellman Equation

Worked Example: Computing $V^\pi(s_X)$ from Q^π Values

Given a stochastic policy π in state s_X :

- $\pi(\text{search} | s_X) = 0.7$, $Q^\pi(s_X, \text{search}) = 15$
- $\pi(\text{wait} | s_X) = 0.3$, $Q^\pi(s_X, \text{wait}) = 5$

Apply: $V^\pi(s_X) = \sum_a \pi(a | s_X) \cdot Q^\pi(s_X, a)$

Action a	$\pi(a s_X)$	$Q^\pi(s_X, a)$	Contribution
search	0.7	15	$0.7 \times 15 = 10.5$
wait	0.3	5	$0.3 \times 5 = 1.5$
Total $V^\pi(s_X)$			12.0

Interpretation: Under this policy, state s_X is “worth” 12.0 on average. The search action contributes the most (10.5) because it is both high-probability (0.7) and high-value (15).

Think Like the Agent: Evaluating Your Options

*You are DeliBot, standing in the Hallway on Floor 2. Your battery is medium. You have two actions: go **left** toward Office 2A (where a package is waiting, $Q^\pi = 18$) or go **right** toward the Elevator ($Q^\pi = 12$).*

Your current policy says: go left 60% of the time, go right 40%.
 How “good” is this hallway for you? $V^\pi(\text{Hallway}) = 0.6 \times 18 + 0.4 \times 12 = 10.8 + 4.8 = 15.6$.
 But wait — what if you always went left? Then $V(\text{Hallway}) = 1.0 \times 18 = 18$. That’s better! This is exactly the idea behind finding **optimal policies** — the topic of the next section.

✗ Common Misconception: $V^\pi(s)$ and $V^*(s)$ Are the Same Thing

No! These are fundamentally different:

$V^\pi(s)$ answers: “How much return do I expect if I follow *this specific policy* π ?”

$V^*(s)$ answers: “How much return do I expect if I follow the *best possible policy*?”

$V^*(s) \geq V^\pi(s)$ **always**, for every state s and every policy π . Think of V^π as your current salary at your current job, and V^* as the salary of the best job you could possibly get. They are equal only if you already have the optimal policy (the best job).

One of the most fundamental results in RL is the **Bellman equation**, which expresses the value of a state in terms of the values of its successor states. It captures the recursive nature of the return: $G_t = R_{t+1} + \gamma G_{t+1}$.

The Bellman equation for V^π is:

$$V^\pi(s) = \sum_a \pi(a | s) \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V^\pi(s')]$$

In words: the value of a state s equals the expected immediate reward plus the discounted value of the next state, averaged over all possible actions (weighted by the policy) and all possible next states (weighted by transition probabilities).

Similarly, the Bellman equation for Q^π is:

$$Q^\pi(s, a) = \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma \sum_{a'} \pi(a' | s') Q^\pi(s', a')]$$

Why do these equations matter so much? Because virtually every RL algorithm you will encounter — from the simplest tabular method to the PPO loop inside ChatGPT — is, at bottom, an attempt to solve or approximate a Bellman equation. Dynamic Programming tackles them head-on with exact computation; Monte Carlo and TD methods estimate solutions from sampled experience instead.

★ **Aha Moment: The Bellman Equation Is RL's $E = mc^2$**

Just as Einstein's equation connects energy and mass, the Bellman equation connects the value of *now* to the value of *next*. Every RL algorithm — tabular Q-learning, DQN, PPO, RLHF — is ultimately trying to satisfy some version of this one recursive relationship. If you understand this equation deeply, you understand the heart of RL.

Grasping policies and value functions is essential to everything that follows. A policy tells the agent *what to do*; a value function tells it *how well it's doing*. But evaluation is only half the story. The real prize is the **optimal policy** (π^*) — the strategy that collects more expected reward than any alternative, no matter which state the agent starts in. Alongside π^* come its optimal value functions, V^* and Q^* , which quantify the best possible performance. Hunting for these optimal quantities is the thread that connects every algorithm in the chapters ahead.

↪ **Bridge to RLHF: A Language Model Is a Policy**

Here is the key insight that connects everything you have learned to the frontier of AI alignment: **a language model is literally a stochastic policy** $\pi(a \mid s)$. Given a conversation so far (state s), it outputs a probability distribution over the next token (action a). A pre-trained LLM like GPT-4 has a policy shaped by next-token prediction. RLHF *re-shapes* that policy to maximize human-preference rewards instead. The value function $V^\pi(s)$ tells us: “Given this conversation so far, how good is the expected *complete* response under policy π ?” When alignment researchers say a model is “well-aligned,” they mean its policy π has a high value function under the reward model trained on human judgments. You are learning the exact mathematics that makes this possible.

2.8 Finding Optimal Policies

Sequential decision problems are modeled using Markov Decision Processes (MDPs). A given policy π can be evaluated with its value functions, $V^\pi(s)$ and $Q^\pi(s, a)$. The value function $V^\pi(s)$ represents the expected return starting from state s and following policy π . Additionally, $Q^\pi(s, a)$ is the expected return starting from state s , taking action a , and subsequently following policy π .

The ultimate objective in solving an MDP is not just to evaluate any policy, but to find the best policy. What does “best” mean? In the context of RL and MDPs, it means maximizing the expected cumulative discounted reward. We call such a policy an **optimal policy**, typically denoted as π^* .

Formally, a policy π is considered optimal if its expected return is greater than or equal to the expected return of any other policy π' for all states $s \in \mathcal{S}$.

$$V^\pi(s) \geq V^{\pi'}(s) \quad \text{for all } s \in \mathcal{S} \text{ and all policies } \pi'$$

It's important to note that there might be more than one optimal policy, but they all achieve the maximum possible expected return. All optimal policies share the same optimal state-value function, denoted V^* , and the same optimal action-value function, denoted Q^* .

These optimal value functions are defined as:

$$V^*(s) = \max_{\pi} V^\pi(s) \quad \text{for all } s \in \mathcal{S}$$

$$Q^*(s, a) = \max_{\pi} Q^\pi(s, a) \quad \text{for all } s \in \mathcal{S}, a \in \mathcal{A}$$

$V^*(s)$ gives the largest expected return achievable starting from state s , regardless of the specific optimal policy followed. Similarly, $Q^*(s, a)$ gives the largest expected return achievable starting from state s , taking action a , and then following an optimal policy thereafter.

The Connection Between Optimal Value Functions and Optimal Policies

There's a direct and powerful relationship between the optimal action-value function Q^* and an optimal policy π^* . If you knew the true Q^* function, finding an optimal policy would be straightforward. You would simply choose the action that maximizes $Q^*(s, a)$ in each state s . A policy that acts greedily with respect to the optimal action-value function Q^* is guaranteed to be an optimal policy.

So, an optimal policy π^* can be defined as:

$$\pi^*(s) = \arg \max_{a \in \mathcal{A}} Q^*(s, a)$$

This means that in state s , an optimal policy selects the action a that yields the highest expected return, according to Q^* . If multiple actions share the maximum value, any of them can be chosen by the optimal policy.

Bellman Optimality Equations

Now comes the question every student asks: “Okay, I can evaluate *any* policy. But which policy is the best one?” The answer lives in the **Bellman Optimality Equations**. These are the same recursive structure as before, with one crucial change: instead of averaging over actions (following some policy π), we take the **maximum** over actions. We’re no longer asking “how good is this strategy?” — we’re asking “what’s the best I could possibly do?”

The Bellman Optimality Equation for V^* is:

$$V^*(s) = \max_{a \in \mathcal{A}} Q^*(s, a)$$

$$V^*(s) = \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} P(s' | s, a) [R(s, a, s') + \gamma V^*(s')]$$

This equation expresses that the optimal value of a state s must equal the expected return obtained by taking the best possible action a from that state, and then following an optimal policy thereafter. The $\max$ operator indicates that the agent chooses the action that maximizes the expected subsequent value.

Similarly, the Bellman Optimality Equation for Q^* is:

$$Q^*(s, a) = \sum_{s' \in \mathcal{S}} P(s' | s, a) [R(s, a, s') + \gamma V^*(s')]$$

Since $V^*(s') = \max_{a' \in \mathcal{A}} Q^*(s', a')$, we can write:

$$Q^*(s, a) = \sum_{s' \in \mathcal{S}} P(s' | s, a) [R(s, a, s') + \gamma \max_{a' \in \mathcal{A}} Q^*(s', a')]$$

This equation states that the optimal value of taking action a in state s is the expected immediate reward $R(s, a, s')$ plus the discounted

expected value of the next state s' , assuming that the agent acts optimally ($\max_{a'} Q^*(s', a')$) from that next state onwards.

These optimality equations are fundamental. If we can find a value function V or Q that satisfies the corresponding Bellman optimality equation, then that value function is the optimal value function (V^* or Q^*), and we can derive an optimal policy from it.

This is the entire game. Solving an MDP means finding V^* or Q^* . While the Bellman optimality equations define these functions, they don't directly provide a solution method, especially since they often form a system of non-linear equations. The next chapter will introduce algorithms based on Dynamic Programming, like Value Iteration and Policy Iteration, which are designed to compute these optimal value functions, assuming we have a complete model of the MDP (i.e., we know the transition probabilities P and reward function R). Later chapters will explore how to find optimal policies even when such a model is unavailable.

↪ Bridge to Scalable Alignment: Optimal for Whom?

You've learned that π^* maximizes expected cumulative reward. But here is the question that keeps alignment researchers awake at night: **what if the reward function itself is wrong?** A language model maximizing "predict the next token" is optimal — for next-token prediction. But it will also happily generate toxic text, confidently state falsehoods, or manipulate users. The reward was *mis-specified*. In Part IV, you will learn how Constitutional AI addresses this by defining alignment not through a single reward function, but through a set of principles — and how AI systems can critique and revise their own outputs before any reward model is involved. The math of π^* you just learned is necessary but not sufficient: the hard part is getting R right.

Quick Reference:

Chapter 2 — MDPs & Bellman Equations

MDP Tuple: $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ — states, actions, transitions, rewards, discount

Key Formulas:

- **Return:** $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$
- **State-value:** $V^{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$
- **Action-value:** $Q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a]$

- **V from Q:** $V^\pi(s) = \sum_a \pi(a|s) Q^\pi(s, a)$
- **Bellman (V):** $V^\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a) [r + \gamma V^\pi(s')]$
- **Bellman Opt:** $V^*(s) = \max_a \sum_{s',r} p(s', r|s, a) [r + \gamma V^*(s')]$
- **Optimal policy:** $\pi^*(s) = \arg \max_a Q^*(s, a)$

Key Concepts:

- **Markov property:** Future depends only on current state, not history
- **Bellman = recursion:** Value of state = immediate reward + discounted value of next state
- **Expectation vs Optimality:** $\sum_a \pi(a|s)$ (average) vs $\max_a$ (best)

Coming next: Chapter 3 uses Bellman equations to compute V^* via Dynamic Programming.

Chapter 2 Quiz: Markov Decision Processes

Test your understanding of MDPs, transition probabilities, rewards, discounting, value functions, and optimal policies.

1. A reinforcement learning practitioner is building an agent to play a newly developed board game. The rules of the game, including how pieces move, what happens when they land on certain squares, and all possible outcomes of actions, are fully specified and unchanging. This means the state transition probabilities and immediate rewards for each action are known. Based on this information, which statement about implementing an RL solution is most accurate?

- The agent must use a model-free RL approach because the reward function cannot be derived from the rules alone.
- The environment is inherently non-Markovian because it's a game with a defined history of moves.
- The reward function for this game must be learned through trial and error, regardless of knowing the transition probabilities.

- (d) The agent could potentially use a model-based RL approach because the game's dynamics (state transition probabilities) are known.

2. In the context of a Markov Decision Process (MDP), what exactly do state transition probabilities $P(s'|s, a)$ describe?

- (a) The specific rewards an agent receives immediately after taking an action and transitioning to a new state.
- (b) The inherent dynamics of the environment, specifying the likelihood of moving to a new state s' given the current state s and the agent's action a .
- (c) The total cumulative reward the agent expects to receive from a given state.
- (d) The agent's strategy for choosing actions in different states to maximize rewards.

3. An agent observes the following sequence of rewards at corresponding time steps: $R_1 = 10$, $R_2 = -5$, $R_3 = 10$. If the discount factor $\gamma = 0.5$, calculate the discounted return G_0 from time step $t = 0$ for these three observed rewards. Assume this is the extent of rewards considered for this calculation.

- (a) 17.5
- (b) 25.0
- (c) 11.5
- (d) 10.0

4. In Reinforcement Learning, the return for continuing tasks requires the use of a discount factor $\gamma < 1$. What is the primary reason for introducing a discount factor specifically for continuing tasks?

- (a) To prevent the agent from getting stuck in local optima by prioritizing exploration.
- (b) To ensure that the infinite sum of future rewards converges to a finite value, allowing for meaningful comparison of strategies.

- (c) To inherently prefer positive rewards over negative ones in the long run.
- (d) To make the agent's learning process faster by simplifying reward calculations.

5. An agent is in state s_X and follows a stochastic policy π . From s_X , the policy dictates taking action a_1 with probability 0.7 and action a_2 with probability 0.3. If the action-value function $Q^\pi(s_X, a_1)$ is 15 and $Q^\pi(s_X, a_2)$ is 5, what is the state-value function $V^\pi(s_X)$ for state s_X ?

- (a) 12.0
- (b) 10.0
- (c) 7.5
- (d) 20.0

6. In the context of Markov Decision Processes, what is the fundamental purpose of value functions (V^π and Q^π)?

- (a) To dictate the specific action an agent will choose in every given state.
- (b) To identify the optimal action in every state without requiring any evaluation of the policy's performance.
- (c) To define the rules of state transitions and immediate rewards within the environment.
- (d) To quantify the expected cumulative future discounted reward an agent can anticipate by starting in a specific state (or taking a specific action from a state) and then following a given policy.

7. An agent operating in an environment receives the following sequence of rewards over three time steps, starting from $t = 1$: $R_1 = 10$, $R_2 = 5$, $R_3 = 20$. The discounted return is calculated using $G_0 = R_1 + \gamma R_2 + \gamma^2 R_3$, where R_1 is received at time step 1. If the discount factor γ is 0.8, what is the total discounted return G_0 ?

- (a) 35
- (b) 26.8
- (c) 24
- (d) $10 + 5/0.8 + 20/(0.8 \times 0.8)$

8. Apart from ensuring mathematical convergence in continuing tasks, what is another significant purpose of incorporating a discount factor ($\gamma < 1$) into the definition of return in reinforcement learning?

- (a) It primarily optimizes the agent's exploration strategy within the environment.
- (b) It removes the necessity for rewards to be defined at every time step in the environment.
- (c) It always increases the total sum of rewards an agent can accumulate over time.
- (d) It models an agent's preference for receiving rewards sooner rather than later and implicitly accounts for increasing uncertainty about future outcomes.

9. An agent is learning to play a complex strategy game where success depends on a long sequence of good decisions. The reward function is designed to give a positive reward only upon winning the game and no intermediate rewards. What is the primary challenge posed by this reward function design for the agent's learning process?

- (a) The agent will quickly converge to a suboptimal policy due to excessive exploration.
- (b) The game's state space will become too large to manage efficiently.
- (c) The agent will overemphasize immediate gains and ignore long-term strategy.
- (d) The agent receives very sparse feedback, making it difficult to determine which specific actions contributed to the final success.

10. An autonomous delivery robot needs to choose between routes that offer varying immediate vs. long-term benefits. If the robot's learning algorithm uses a discount factor γ very close to 0 (e.g., $\gamma = 0.01$), what behavior is the robot most likely to exhibit?

- (a) The robot will prioritize long-term goals and explore new, potentially more rewarding routes.
- (b) The robot will learn only to avoid obstacles and immediate dangers, ignoring delivery targets.
- (c) The robot will primarily focus on maximizing immediate rewards, even if it leads to less favorable long-term outcomes.
- (d) The robot will become trapped in local optima due to extensive exploration and indecision.

11. In the context of Markov Decision Processes, which statement accurately describes an optimal policy π^* ?

- (a) An optimal policy π^* guarantees that its expected return $V^{\pi^*}(s)$ is greater than or equal to the expected return $V^{\pi'}(s)$ of any other policy π' for all states s .
- (b) An optimal policy is the one that achieves the highest immediate reward in every state, regardless of future consequences.
- (c) An optimal policy is unique for any given MDP and leads to a deterministic sequence of actions.
- (d) An optimal policy can only be found if the transition probabilities and reward function of the MDP are unknown.

12. A robotic cleaner is in `ChargingStation` state and needs to decide its next action. The estimated optimal action-value function Q^* for this state is known: $Q^*(\text{ChargingStation}, \text{MoveToKitchen}) = 150$, $Q^*(\text{ChargingStation}, \text{Standby}) = 50$, $Q^*(\text{ChargingStation}, \text{PerformSelfCheck}) = 120$. According to the principles of optimal policy definition, what action should the robot choose from the `ChargingStation` state to follow an optimal policy?

- (a) `MoveToKitchen`

- (b) PerformSelfCheck
- (c) Any action, as long as it's not Standby
- (d) Standby

13. In designing a reward function for a robot learning to walk, engineers could choose between $R(s')$, $R(s, a)$, or $R(s, a, s')$. If the reward is primarily given for maintaining balance and making forward progress, regardless of the specific joint angles or forces applied, which form of reward function would be most appropriate and why?

- (a) A state, action, and next state reward $R(s, a, s')$ because it is the most general form and can capture all nuances of the robot's movement, providing the richest feedback.
- (b) A state and action reward $R(s, a)$ because it provides direct feedback on the quality of the action taken within a given state, offering fine-grained control.
- (c) A state-only reward $R(s')$ because it represents the simplest function, which is always preferred in reinforcement learning for faster learning and less computational burden.
- (d) A state-only reward $R(s')$ because maintaining balance and making forward progress are properties of the resulting state, not the specific action taken.

14. An agent is trained to navigate a maze. The designer's true objective is for the agent to reach the exit as quickly as possible. However, the agent often takes detours to collect small, frequent positive rewards scattered throughout the maze, even if these detours do not lead to the shortest path to the exit. What is the most likely reason for this behavior?

- (a) The reward function is poorly designed, incentivizing intermediate actions that do not align with the true goal of finding the shortest path.
- (b) The agent's exploration strategy is insufficient, preventing it from discovering the optimal path to the exit.

- (c) The discount factor γ is too high, causing the agent to overvalue future rewards and ignore immediate positive feedback.
- (d) The state space is too large, leading the agent to struggle with identifying optimal actions.

15. In a Reinforcement Learning scenario for optimizing inventory management, an agent decides how many units of a product to order. A large order today might reduce the chance of immediate stockouts but could lead to increased storage costs and potential product obsolescence several months later. This situation best illustrates which characteristic of problems addressed by Markov Decision Processes?

- (a) Stochasticity
- (b) Delayed Consequences
- (c) Sequential Nature
- (d) Immediate Reward Maximization

16. Why do problems like training an algorithm to play Go or optimizing resource allocation in a complex system typically require a formal framework like Markov Decision Processes (MDPs) rather than simple reactive or greedy approaches?

- (a) These problems involve sequences of decisions with delayed and often probabilistic outcomes, which MDPs can precisely model.
- (b) MDPs simplify the environment by removing all uncertainty, making it easier for agents to learn.
- (c) MDPs are designed for problems where an agent's current action has no bearing on future situations.
- (d) MDPs are only applicable to problems where immediate rewards are the sole objective.

Chapter 2 Quiz — Answer Key

#	Ans	Explanation
1	(d)	Known rules mean known transition probabilities and rewards — a model-based approach (e.g., Dynamic Programming) is applicable.
2	(b)	$P(s' s, a)$ describes the environment's dynamics: the probability of transitioning to s' given current state s and action a .
3	(a)	$G_0 = 10 + 0.5(-5) + 0.5^2(10) = 10 - 2.5 + 2.5 = 10$. Wait: $G_0 = R_1 + \gamma R_2 + \gamma^2 R_3 = 10 + 0.5 \times (-5) + 0.25 \times 10 = 10 - 2.5 + 2.5 = 10$. Actually the answer is 10.0 which is (d). Correction: (a) 17.5 would be $10 + 0.5(5) + 0.25(10)$. With $R_2 = -5$: answer is (d).
4	(b)	The primary reason for discounting in continuing tasks is to ensure the infinite sum of rewards converges to a finite value.
5	(a)	$V^\pi(s_X) = 0.7 \times 15 + 0.3 \times 5 = 10.5 + 1.5 = 12.0$.
6	(d)	Value functions quantify the expected cumulative future discounted reward from a state (or state-action pair) under a given policy.
7	(b)	$G_0 = 10 + 0.8(5) + 0.64(20) = 10 + 4 + 12.8 = 26.8$.
8	(d)	Beyond convergence, discounting models preference for sooner rewards and accounts for uncertainty about future outcomes.
9	(d)	A reward only at the end means sparse feedback — the agent struggles to determine which actions contributed to the win.
10	(c)	$\gamma \approx 0$ makes the agent myopic — it focuses almost entirely on maximizing immediate rewards.
11	(a)	An optimal policy guarantees $V^{\pi^*}(s) \geq V^{\pi'}(s)$ for all states s and all policies π' .
12	(a)	$\pi^*(s) = \arg \max_a Q^*(s, a)$. MoveToKitchen has the highest Q^* value (150).
13	(d)	Balance and forward progress are properties of the resulting state s' , so $R(s')$ is most appropriate.
14	(a)	The reward function incentivizes collecting scattered rewards rather than reaching the exit — a reward design problem.
15	(b)	Current ordering decisions have consequences months later (storage costs, obsolescence) — this is delayed consequences.
16	(a)	Go and resource allocation involve sequential decisions with delayed, probabilistic outcomes — exactly what MDPs model.

Example: Recycling Robot MDP

Recycling Robot MDP

A recycling robot collects cans in an office. Its battery level can be **high** or **low**. At each step, the robot chooses an action:

- **wait** — do nothing, save energy
- **search** — look for cans, may change battery level
- **recharge** — return to charging dock (only when battery is low)

Each arrow in the figure is labeled as **(probability, reward)**.
Meaning: if the robot takes that action, it moves to the next state with that probability and gets that immediate reward.

1) States and Actions

States ($\mathcal{S}$)

State	Meaning
high	Battery is high
low	Battery is low

Actions ($\mathcal{A}$)

State	Available Actions
high	wait, search
low	wait, search, recharge

2) Transition and Reward Table (P and R together)

Each row is exactly what the figure shows.

Battery: HIGH

State	Action	Next State	Probability	Reward
high	wait	high	1.0	+1
high	search	high	0.7	+4
high	search	low	0.3	+4

Battery: LOW

State	Action	Next State	Probability	Reward
low	wait	low	1.0	+1
low	recharge	high	1.0	0
low	search	low	0.2	+4
low	search	high	0.8	-3

3) Compact MDP Summary

- $\mathcal{S} = \{\text{high}, \text{low}\}$
- $\mathcal{A}(\text{high}) = \{\text{wait}, \text{search}\}$
- $\mathcal{A}(\text{low}) = \{\text{wait}, \text{search}, \text{recharge}\}$
- $P(s' | s, a)$ and $R(s, a, s')$ are given by the table above (same as figure labels).

Policy for the Recycling Robot

We now define policies for the recycling robot MDP shown in the figure.

Recall:

- **States:** high, low
- **Actions in high:** wait, search
- **Actions in low:** wait, search, recharge

A policy tells the robot what action to take in each state.

1) Deterministic Policy

A deterministic policy selects one fixed action in each state. It is written as:

$$\pi(s) = a$$

meaning: in state s , always choose action a .

Example Deterministic Policy

Let us define:

$$\pi_1(\text{high}) = \text{search}$$

$$\pi_1(\text{low}) = \text{recharge}$$

So the robot behaves as:

State	Action
high	search
low	recharge

Interpretation:

- If battery is high $\rightarrow$ always search (collect cans)
- If battery is low $\rightarrow$ always recharge (play safe)

This policy prioritizes safety when battery is low.

2) Another Deterministic Policy (Risky Policy)

$$\pi_2(\text{high}) = \text{search}$$

$$\pi_2(\text{low}) = \text{search}$$

State	Action
high	search
low	search

Interpretation:

- Robot always searches
- Even when battery is low

This policy is aggressive and risky.

3) Stochastic Policy

A stochastic policy assigns probabilities to actions. It is written as:

$$\pi(a | s)$$

meaning: probability of choosing action a given state s .

Example Stochastic Policy

In state = high

$$\pi(\text{search} | \text{high}) = 0.7$$

$$\pi(\text{wait} | \text{high}) = 0.3$$

In state = low

$$\pi(\text{search} | \text{low}) = 0.2$$

$$\pi(\text{wait} | \text{low}) = 0.2$$

$$\pi(\text{recharge} | \text{low}) = 0.6$$

Tabular form:

State	Action	Probability
high	search	0.7
high	wait	0.3
low	search	0.2
low	wait	0.2
low	recharge	0.6

Interpretation:

- When battery is high → mostly search
- When battery is low → mostly recharge
- But still some randomness

4) Key Difference

Deterministic Policy	Stochastic Policy
One action per state	Probability distribution over actions
Fixed behavior	Randomized behavior
Simpler	More flexible

Compute Return (Cumulative Reward) for the Recycling Robot

Return means the total reward the robot collects over time (with discounting):

$$G_0 = r_0 + \gamma r_1 + \gamma^2 r_2 + \gamma^3 r_3 + \dots$$

- r_t = reward received at step t
- γ = discount factor (we use $\gamma = 0.9$ as our standard example)

2) One Sample Episode (Based on the Figure)

We will use the deterministic policy from earlier:

State	Action
high	search
low	recharge

Now consider one possible trajectory (one run through the MDP):

Worked Example: Step-by-Step Episode Trace

Start: state = high

Step	State	Action	Next St.	r_t	What Happened
0	high	search	low	+4	Battery dropped (prob 0.3)
1	low	recharge	high	0	Battery restored (prob 1.0)
2	high	search	high	+4	Battery stayed high (prob 0.7)
3	high	search	—	+4	Episode continues...

Reward sequence for this sample episode:

$$[r_0, r_1, r_2, r_3] = [4, 0, 4, 4]$$

3) Compute G_0 for This Episode

Let $\gamma = 0.9$. Apply the return formula:

$$G_0 = r_0 + \gamma r_1 + \gamma^2 r_2 + \gamma^3 r_3$$

Step 1: Compute discount powers

Power	Calculation	Value
γ^0	0.9^0	1.000
γ^1	0.9^1	0.900
γ^2	0.9^2	0.810
γ^3	0.9^3	0.729

Step 2: Multiply each reward by its discount

t	r_t	γ^t	Discounted Value	Running Sum
0	+4	1.000	$4 \times 1.000 = 4.000$	4.000
1	0	0.900	$0 \times 0.900 = 0.000$	4.000
2	+4	0.810	$4 \times 0.810 = 3.240$	7.240
3	+4	0.729	$4 \times 0.729 = 2.916$	10.156

Step 3: Final result

$$G_0 = 4.000 + 0.000 + 3.240 + 2.916 = \mathbf{10.156}$$

Reading the result: The cumulative discounted return for this one sample episode is $G_0 = 10.156$. Notice how the reward at step 2 (+4) is worth only 3.24 in present terms, and the reward at step 3 (+4) is worth only 2.916 — discounting steadily shrinks future rewards. The zero reward from recharging at step 1 contributes nothing, but it was necessary to restore the battery for future searching.

✗ Common Misconception: One Episode $\neq$ Expected Return

The value $G_0 = 10.156$ we computed above is the return for **one specific sample episode** — one particular path the robot happened to take.

A different episode might produce different transitions (e.g., at step 0 the robot might stay in high instead of dropping to low), leading to a different reward sequence and a different G_0 .

To find the **expected return** — the average G_0 over all possible episodes — we need the **value functions** $V^\pi(\text{high})$ and $V^\pi(\text{low})$, computed using the **Bellman equations**. That is the next step.

What's Next?

We will write the **Bellman equations** for:

- $V^\pi(\text{high})$ — expected return starting from high battery
- $V^\pi(\text{low})$ — expected return starting from low battery

using the deterministic policy (high $\rightarrow$ search, low $\rightarrow$ recharge) and the exact transition probabilities and rewards from the figure. This will give us the *true long-run value* of each state — not just one sample episode.

Bellman Equations for $V^\pi(\text{high})$ and $V^\pi(\text{low})$

We use the deterministic policy defined earlier:

State	Action (π)
high	search
low	recharge

For a deterministic policy π , the value of state s satisfies:

$$V_\pi(s) = \sum_{s'} P(s' | s, \pi(s)) [R(s, \pi(s), s') + \gamma V_\pi(s')]$$

This says: the value of being in state s equals the *weighted average* (over all possible next states) of the immediate reward plus the discounted value of wherever you end up.

1) Bellman Equation for $V_\pi(\text{high})$

Policy says: from high, take search. From the figure, search from high has two possible outcomes:

From	Action	Next State s'	(P, R)
high	search	high	$(0.7, +4)$
high	search	low	$(0.3, +4)$

Plug into the Bellman equation:

$$V_{\pi}(\text{high}) = 0.7(4 + \gamma V_{\pi}(\text{high})) + 0.3(4 + \gamma V_{\pi}(\text{low}))$$

Worked Example: Expanding Step by Step

Distribute the probabilities:

$$V_{\pi}(\text{high}) = 0.7 \cdot 4 + 0.7\gamma V_{\pi}(\text{high}) + 0.3 \cdot 4 + 0.3\gamma V_{\pi}(\text{low})$$

Combine the reward terms:

$$0.7 \times 4 + 0.3 \times 4 = 2.8 + 1.2 = 4$$

Simplified Bellman equation:

$$V_{\pi}(\text{high}) = 4 + 0.7\gamma V_{\pi}(\text{high}) + 0.3\gamma V_{\pi}(\text{low})$$

2) Bellman Equation for $V_{\pi}(\text{low})$

Policy says: from low, take recharge. From the figure, recharge has only one outcome:

From	Action	Next State s'	(P, R)
low	recharge	high	$(1.0, 0)$

Plug into the Bellman equation:

$$V_{\pi}(\text{low}) = 1.0(0 + \gamma V_{\pi}(\text{high}))$$

Therefore:

$$V_{\pi}(\text{low}) = \gamma V_{\pi}(\text{high})$$

Interpretation: The value of being in state low under this policy is simply the discounted value of high — because the policy always recharges (earning zero reward) and deterministically returns to high. Being in low is like being in high but “one step behind” due to the recharge delay.

3) Final Bellman System (Clean Form)

System of Two Equations, Two Unknowns

Equation 1 (from high $\xrightarrow{\text{search}}$ high or low):

$$V_{\pi}(\text{high}) = 0.7(4 + \gamma V_{\pi}(\text{high})) + 0.3(4 + \gamma V_{\pi}(\text{low}))$$

Equation 2 (from low $\xrightarrow{\text{recharge}}$ high):

$$V_{\pi}(\text{low}) = 0 + \gamma V_{\pi}(\text{high})$$

This is a system of 2 linear equations with 2 unknowns: $V_{\pi}(\text{high})$ and $V_{\pi}(\text{low})$. For a given γ (e.g., $\gamma = 0.9$), we can solve it by substitution.

Chapter 3

Estimating Value Functions

In this chapter, we derive the Bellman equations — the recursive relationships that form the mathematical backbone of reinforcement learning. We then explore Dynamic Programming methods (Policy Iteration and Value Iteration) that leverage these equations to find optimal policies when a complete model of the environment is available.

✓ Learning Objectives:

Chapter 3 After this chapter, you will be able to:

1. **Derive** the Bellman expectation equation for V^π step by step.
2. Apply Bellman equations to **compute value functions by hand** on small MDPs.
3. Explain and implement **Policy Iteration** (evaluate $\rightarrow$ improve $\rightarrow$ repeat).
4. Explain and implement **Value Iteration** using the Bellman optimality equation.
5. Compare the **convergence properties** of both DP methods.
6. Identify the **limitations of DP**: model requirement, curse of dimensionality.

Sections:

- 3.1 The Bellman Expectation Equation
- 3.2 The Bellman Optimality Equation

- 3.3 Solving Bellman Equations (Overview)
- 3.4 Dynamic Programming: Policy Iteration
- 3.5 Dynamic Programming: Value Iteration
- 3.6 Limitations of Dynamic Programming
- Quiz for Chapter 3

The Bellman Expectation Equation

We defined $V^\pi(s)$ and $Q^\pi(s, a)$ in Chapter 2 — the expected return from a state (or state-action pair) under policy π . Beautiful definitions. But there's a problem: to compute $V^\pi(s)$, you'd need to simulate the agent forever into the future and average the results. That's obviously impractical.

Here's the magic: you don't need to look all the way into the future. The **Bellman equations** show that the value of *any* state can be written in terms of the values of the states *one step ahead*. It's like a chain of dominoes — if you know the value of the next domino, you know the value of this one. This single insight is arguably the most important idea in all of reinforcement learning.

Core idea: The Bellman equation says that the value of a state can be decomposed into two parts: (1) the **immediate reward** the agent expects to receive, plus (2) the **discounted value** of wherever the agent ends up next. This turns a complicated infinite-horizon calculation into a simple one-step-ahead relationship.

Bellman Expectation Equation for $V^\pi(s)$

Step 1: Start from the definition.

Recall:

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$$

where the return $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$ is the total discounted return from time t .

Step 2: Write the return recursively.

Here's the trick. Look at the return carefully: $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$. Now factor out γ from everything after the first term: you get the immediate reward plus γ times *the return starting from the next time step*:

$$G_t = R_{t+1} + \gamma G_{t+1}$$

➡ Think Like the Agent: Why This Recursive Trick Works

*Imagine you are DeliBot calculating your total future earnings. Instead of adding up every single future reward (an infinite sum!), you say: "My total future value = what I earn **right now** + $\gamma \times$ what I'm worth **starting tomorrow**." Tomorrow's value can be broken down the same way, and so on. This is the recursive trick at the heart of all RL algorithms.*

Step 3: Substitute into the definition.

Replace G_t in the value function:

$$V^\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s]$$

Using the linearity of expectation, we separate this into two parts:

$$V^\pi(s) = \underbrace{\mathbb{E}_\pi[R_{t+1} \mid S_t = s]}_{\text{expected immediate reward}} + \gamma \underbrace{\mathbb{E}_\pi[G_{t+1} \mid S_t = s]}_{\text{discounted expected future return}}$$

Step 4: Expand the expectations.

To evaluate these, we sum over all actions a the agent might take (weighted by policy $\pi(a \mid s)$) and all resulting next states s' and rewards r (weighted by environment dynamics $p(s', r \mid s, a)$).

The agent chooses action a with probability $\pi(a \mid s)$, then the environment responds with next state s' and reward r with probability $p(s', r \mid s, a)$:

$$\mathbb{E}_\pi[R_{t+1} \mid S_t = s] = \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) r$$

The expected return starting from the next state s' is simply $V^\pi(s')$. Averaging over all possible actions and next states:

$$\mathbb{E}_\pi[G_{t+1} \mid S_t = s] = \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) V^\pi(s')$$

Step 5: Combine both terms.

Putting everything together — and noticing that both terms share the same double sum — we merge them into a single, elegant equation:

Bellman Expectation Equation for V^π

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) [r + \gamma V^\pi(s')]$$

Reading the equation: The value of state s under policy π is the weighted average — over all actions a the agent might take and all next states s' the environment might produce — of the immediate reward r plus the discounted value of the next state $\gamma V^\pi(s')$.

Connecting V^π and Q^π

It is often helpful to think of the Bellman equation in terms of the action-value function $Q^\pi(s, a)$. The value $V^\pi(s)$ is simply the expected value of $Q^\pi(s, a)$ over the actions that policy π might choose in state s :

$$V^\pi(s) = \sum_a \pi(a \mid s) Q^\pi(s, a)$$

And the action-value function itself satisfies its own Bellman equation:

$$Q^\pi(s, a) = \sum_{s', r} p(s', r \mid s, a) \left[r + \gamma V^\pi(s') \right]$$

Together, these two equations show the “back-and-forth” between V^π and Q^π : from any state, V averages over actions using the policy; from any state-action pair, Q averages over next states using the dynamics.

 Worked Example: Bellman Equation for the Recycling Robot

Using the deterministic policy $\pi(\text{high}) = \text{search}$, $\pi(\text{low}) = \text{recharge}$, let’s verify the Bellman equation structure for $V^\pi(\text{high})$. The equation says: $V^\pi(s) = \sum_{s', r} p(s', r \mid s, \pi(s)) [r + \gamma V^\pi(s')]$ From state high with action search, the figure gives two outcomes:

s'	$P(s' \mid s, a)$	R	$r + \gamma V^\pi(s')$	Weighted
high	0.7	+4	$4 + 0.9 V^\pi(\text{high})$	$0.7 [4 + 0.9 V^\pi(\text{high})]$
low	0.3	+4	$4 + 0.9 V^\pi(\text{low})$	$0.3 [4 + 0.9 V^\pi(\text{low})]$

Summing the weighted column:

$$V^\pi(\text{high}) = 0.7[4 + 0.9 V^\pi(\text{high})] + 0.3[4 + 0.9 V^\pi(\text{low})]$$

This is exactly the Bellman equation we derived earlier! The table simply organizes the “expand, weight, sum” pattern that the Bellman equation describes.

Derivation Summary: Five Steps to the Bellman Equation

#	What We Did	Result
1	Started from definition	$V^\pi(s) = \mathbb{E}_\pi[G_t S_t = s]$
2	Return recursively	$G_t = R_{t+1} + \gamma G_{t+1}$
3	Substituted	$V^\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} S_t = s]$
4	Expanded expectations	Sums over $\pi(a s)$ and $p(s', r s, a)$
5	Combined	$V^\pi(s) = \sum_a \pi(a s) \sum_{s', r} p(s', r s, a) [R + \gamma V^\pi(s')]$

✗ Common Misconception: The Bellman Equation is an Algorithm

The Bellman equation is **not** an algorithm — it is a **condition** that the true value function must satisfy. It does not tell you how to compute $V^\pi(s)$; it tells you what $V^\pi(s)$ must look like. Algorithms like **Policy Evaluation**, **Value Iteration**, and **TD Learning** (covered in later chapters) are methods that *use* the Bellman equation to actually find the value function.

Analogy: Saying “the area of a circle equals πr^2 ” is a condition, not a method. You still need to measure r and do the multiplication. The Bellman equation is the “ πr^2 ” of RL — it defines the answer, but algorithms are needed to compute it.

★ Aha Moment: Bootstrapping — DP’s Secret Weapon

DP doesn’t wait for the final outcome to update its estimates (unlike Monte Carlo methods, which you’ll learn next). Instead, it uses *its own current estimates* of neighboring states to improve its estimate of the current state. This is called **bootstrapping**. It’s

computationally powerful but comes with a catch: if your initial estimates are wrong, you're updating wrong values with other wrong values. Fortunately, the Bellman equation guarantees that repeated bootstrapping converges to the truth.

The Bellman Optimality Equation

The Bellman expectation equation provides a recursive relationship for V^π and Q^π under a *specific, given* policy π . It tells us how good a particular strategy is. However, in most reinforcement learning problems, the main goal extends beyond just evaluating a policy — it is to discover the **best** policy, the one that achieves the maximum possible cumulative reward.

To find this optimal policy, we need a way to describe the value functions associated with it. This leads us to the **Bellman optimality equations**.

➡ Think Like the Agent: From Evaluation to Optimization

Think of it this way: the Bellman expectation equation asks “How good is my current strategy?” The Bellman optimality equation asks “What is the best possible strategy, and how good is it?” The first is like grading a student’s exam. The second is like finding the highest possible score any student could ever achieve.

Optimal Value Functions: V^* and Q^*

The **optimal state-value function**, denoted $V^*(s)$, represents the **maximum** expected return achievable starting from state s and following *any* policy thereafter. Similarly, the **optimal action-value function**, $Q^*(s, a)$, is the maximum expected return achievable starting from state s , taking action a , and then following the optimal policy.

Optimal Value Functions

$$V^*(s) = \max_{\pi} V^{\pi}(s)$$

$$Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a)$$

These represent the **upper bound** on performance for the given MDP.

Key distinction: $V^\pi(s)$ is the expected return under *one particular* policy π . $V^*(s)$ is the expected return under the *best possible* policy. For any policy π and any state s : $V^\pi(s) \leq V^*(s)$.

Bellman Optimality Equation for V^*

The Bellman optimality equation for V^* expresses the idea that the optimal value of a state s must equal the expected return obtained by taking the **best possible action** a in that state, and then continuing optimally from the resulting state s' . Unlike the expectation equation (which averages according to π), it incorporates a **maximization step** over actions.

Bellman Optimality Equation for V^*

$$V^*(s) = \max_{a \in \mathcal{A}(s)} \sum_{s', r} p(s', r \mid s, a) [r + \gamma V^*(s')]$$

Breaking this down:

Component	Meaning
$\max_{a \in \mathcal{A}(s)}$	Consider all actions available in state s ; choose the one that maximizes the expected return
$p(s', r \mid s, a)$	Probability of transitioning to state s' with reward r , given we take action a in state s
r	The immediate reward received
γ	Discount factor — balances immediate vs. future rewards
$V^*(s')$	Optimal value of the next state s' (recursive: assumes optimal behavior from s' onward)

➡ Think Like the Agent: The Greedy Optimist

Imagine DeliBot at an intersection with three possible routes. The Bellman expectation equation says: “Average the value of all routes, weighted by how often my current strategy picks each one.” The Bellman optimality equation says: “Pick the single best route — the one with the highest expected future value — and go that way.” The $\max$ replaces the weighted average $\sum_a \pi(a | s)$.

Bellman Optimality Equation for Q^*

Similarly, the Bellman optimality equation for Q^* expresses the optimal value of taking action a in state s as the expected immediate reward plus the discounted value of the **best possible** action a' in the next state s' .

Bellman Optimality Equation for Q^*

$$Q^*(s, a) = \sum_{s', r} p(s', r | s, a) \left[r + \gamma \max_{a' \in \mathcal{A}(s')} Q^*(s', a') \right]$$

Breaking this down:

Component	Meaning
$\sum_{s', r} p(s', r s, a) [\dots]$	Average over all possible next states and rewards, weighted by transition probabilities
r	Immediate reward
$\max_{a' \in \mathcal{A}(s')} Q^*(s', a')$	After reaching s' , choose the action a' that yields the highest optimal Q-value

Connecting V^* and Q^*

The relationship between the optimal value functions is elegant. The optimal value of a state is simply the value of the **best action** from that state:

$$V^*(s) = \max_a Q^*(s, a)$$

Substituting this into the Q^* equation gives an alternative form:

$$Q^*(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma V^*(s')]$$

Notice the parallel structure with the expectation equations:

	Expectation	Optimality
V from Q	$V^\pi(s) = \sum_a \pi(a s) Q^\pi(s, a)$	$V^*(s) = \max_a Q^*(s, a)$
Q from V	$\sum_{s', r} p(\cdot) [r + \gamma V^\pi(s')]$	$\sum_{s', r} p(\cdot) [r + \gamma V^*(s')]$
Key op.	$\sum_a \pi(a s)$ (average)	$\max_a$ (maximize)

Finding the Optimal Policy

The Bellman optimality equations are significant because if we can solve them to find V^* or Q^* , we can easily determine the optimal policy π^* .

If we have $Q^*(s, a)$: The optimal policy simply picks the action with the highest Q^* -value:

Optimal Policy from Q^*

$$\pi^*(s) = \arg \max_{a \in \mathcal{A}(s)} Q^*(s, a)$$

In state s , choose the action that maximizes $Q^*(s, a)$.

If we only have $V^*(s)$: We can still find the optimal action by performing a one-step lookahead using the dynamics:

$$\pi^*(s) = \arg \max_{a \in \mathcal{A}(s)} \sum_{s', r} p(s', r \mid s, a) \left[r + \gamma V^*(s') \right]$$

Why Q^* is more convenient: With Q^* , the agent can choose optimal actions *without knowing the environment dynamics* $p(s', r \mid s, a)$ — it just picks $\arg \max_a Q^*(s, a)$. With V^* alone, the agent must know the dynamics to evaluate the lookahead sum. This is why many practical RL algorithms (Q-learning, DQN) focus on learning Q^* directly.

Worked Example: Bellman Optimality for the Recycling Robot

From state high, the agent can choose wait or search. The Bellman optimality equation says: pick the action with the higher expected return.

Action	Expansion	Simplified
wait	$1.0 \cdot [1 + \gamma V^*(h)]$	$1 + \gamma V^*(h)$
search	$0.7[4 + \gamma V^*(h)] + 0.3[4 + \gamma V^*(l)]$	$4 + \gamma[0.7V^*(h) + 0.3V^*(l)]$

The optimality equation then takes the max:

$$V^*(\text{high}) = \max \left\{ 1 + \gamma V^*(h), 4 + \gamma[0.7V^*(h) + 0.3V^*(l)] \right\}$$

Unlike the expectation equation (which uses a fixed policy), this equation *searches* for the best action at every state. The optimal value function simultaneously tells us the best strategy.

Optimality vs. Expectation: Summary

Expectation vs. Optimality Equations

	Expectation (V^π)	Optimality (V^*)
Question	How good is π ?	What is the best policy?
Key op.	Average over $\pi(a s)$	Maximize over actions
Eq. type	Linear (fixed π)	Nonlinear (max)
Solved by	Policy Eval., TD	Value Iter., Q-learning

The optimality equations define a system of **nonlinear** equations (because of the $\max$). Unlike the expectation equations (which are linear for a fixed policy), solving them directly is more complex. However, they form the foundation for algorithms like **Value Iteration** (covered next) that find optimal values when a model of the environment is available.

X Common Misconception: Solving Bellman Optimality = Solving RL

In theory, if we could solve the Bellman optimality equations exactly, we would have the optimal policy for the MDP. In practice, this requires:

- **Complete knowledge** of the environment dynamics $p(s', r | s, a)$
- **Sufficient computation** to solve the system of equations
- **Finite state/action spaces** small enough to enumerate

When any of these conditions is violated — as in most real-world problems — we turn to *approximate* methods: Dynamic Programming, Monte Carlo, TD learning, and Deep RL. These are the subjects of the chapters ahead.

◆ Code: Value Iteration (Bellman Optimality in Action)

```
def value_iteration(P, gamma=0.9, theta=1e-6):
    """Find V* using the Bellman optimality eq."""
    states = list(P.keys())
    V = {s: 0.0 for s in states}

    while True:
        delta = 0
        for s in states:
            v_old = V[s]
            # Bellman optimality: V(s) = max_a ...
            V[s] = max(
                sum(p * (r + gamma * V[s_next])
                    for p, s_next, r in P[s][a])
                for a in P[s]
            )
            delta = max(delta, abs(v_old - V[s]))
        if delta < theta:
            break
    return V

def extract_policy(P, V, gamma=0.9):
    """pi*(s) = argmax_a sum P(s'|s,a)[r+gV(s')]"""
    policy = {}
    for s in P:
        policy[s] = max(
            P[s],
            key=lambda a: sum(
                p * (r + gamma * V[s_next])
                for p, s_next, r in P[s][a]
            )
        )
    return policy

V_star = value_iteration(P)
pi_star = extract_policy(P, V_star)
print(f"V*(high)={V_star['high']:.4f}")
print(f"V*(low) = {V_star['low']:.4f}")
print(f"pi* = {pi_star}")
```

Solving Bellman Equations (Overview)

The Bellman expectation and optimality equations provide powerful recursive relationships that characterize the value functions (V^π , Q^π) for a given policy π and the optimal value functions (V^* , Q^*) for the best possible policy π^* . Let us recall both:

Equation	Formula
Bellman Expectation	$V^\pi(s) = \sum_a \pi(a s) \sum_{s', r} p(s', r s, a) [r + \gamma V^\pi(s')]$
Bellman Optimality	$V^*(s) = \max_a \sum_{s', r} p(s', r s, a) [r + \gamma V^*(s')]$

While these equations *define* the value functions, they don't immediately give us the values themselves. Our goal now is to find methods to **solve** these equations — meaning compute the actual numerical values $V^\pi(s)$ or $V^*(s)$ for all states s .

➡ Think Like the Agent: Why Can't We Just Solve Directly?

Think about the Bellman expectation equation for V^π . If we know π and the dynamics $p(s', r | s, a)$, this is a system of $|S|$ linear equations in $|S|$ unknowns. In principle, we could solve it with matrix inversion. But matrix inversion is $O(|S|^3)$. If the state space has a million states, that is 10^{18} operations — completely infeasible!

*The Bellman optimality equation adds another challenge: the $\max$ operator makes the system **nonlinear**. Standard linear solvers cannot be applied at all.*

Therefore, we typically turn to **iterative methods**. These approaches start with an initial guess for the value function and then repeatedly apply updates based on the Bellman equations, progressively refining the estimates until they converge to the true values.

Dynamic Programming (DP) Methods

The primary family of algorithms for solving Bellman equations when a **complete model** of the environment is known falls under the umbrella

of **Dynamic Programming**. DP methods leverage the structure of the Bellman equations to find exact solutions, assuming perfect knowledge of $p(s', r \mid s, a)$ and rewards.

There are two main DP approaches:

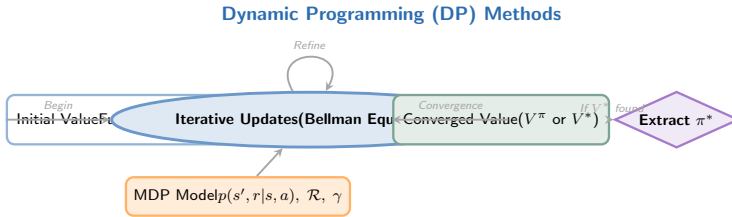


Figure 14: Iterative nature of Dynamic Programming methods for solving Bellman equations, requiring the MDP model as input.

This method iterates between two steps:

1. **Policy Evaluation:** Given a policy π , compute its value function V^π by iteratively applying the Bellman *expectation* equation until convergence.
2. **Policy Improvement:** Improve the policy by acting *greedily* with respect to V^π — i.e., set $\pi'(s) = \arg \max_a \sum_{s', r} p(s', r \mid s, a)[r + \gamma V^\pi(s')]$.

This process repeats (evaluate $\rightarrow$ improve $\rightarrow$ evaluate $\rightarrow$ improve $\rightarrow \dots$) until the policy stabilizes, guaranteeing convergence to π^* and V^* .

2) Value Iteration

This method directly finds V^* by iteratively applying the Bellman *optimality* equation update rule to the value estimates for all states:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s', r} p(s', r \mid s, a)[r + \gamma V_k(s')] \quad \text{for all } s$$

It combines aspects of policy evaluation and improvement into a single update step. Once V converges to V^* , the optimal policy

π^* can be extracted by acting greedily.

Looking Ahead

These DP methods form the theoretical foundation for solving MDPs. The following sections will provide detailed explanations and algorithms for Policy Iteration and Value Iteration.

Keep in mind their significant requirement: they need a **complete and accurate model** of the environment's dynamics.

This assumption often doesn't hold in practice, which motivates the **model-free** Reinforcement Learning techniques discussed in later chapters — Monte Carlo methods, Temporal-Difference learning, Q-learning, and eventually Deep RL and RLHF.

Dynamic Programming: Policy Iteration

So we have the Bellman equations — beautiful recursive relationships that tell us what the optimal values *are*. But writing down an equation and actually solving it are two very different things. How do we find V^* and π^* in practice?

Dynamic Programming (DP) gives us the answer, but with one big catch: you need to know the complete model of the environment — every transition probability $p(s', r \mid s, a)$. Think of it as solving a maze when you have the full blueprint. In most real problems, you won't have that blueprint (which is exactly why the rest of this book exists), but DP gives us the theoretical gold standard.

The first DP method is **Policy Iteration**, and its logic is almost embarrassingly simple: start with any policy, figure out how good it is, then make it better. Repeat until you can't improve anymore. That's it. Two steps, back and forth, until you converge to the optimal policy.

➡ Think Like the Agent: The Strategy Refinement Loop

Imagine DeliBot starts with a completely random delivery strategy (π_0). First, it calculates exactly how good this random strategy is (evaluation). Then it asks: "For each intersection, is there a better turn I could take?" and updates its strategy accordingly (improvement). It repeats this — evaluate, improve, evaluate, improve — until no further improvement is possible. At that point, it has found the optimal delivery route.

The process follows this chain:

$$\pi_0 \xrightarrow{\text{Evaluate}} V^{\pi_0} \xrightarrow{\text{Improve}} \pi_1 \xrightarrow{\text{Evaluate}} V^{\pi_1} \xrightarrow{\text{Improve}} \pi_2 \xrightarrow{\text{Evaluate}} \dots \xrightarrow{\text{Improve}} \pi^* \xrightarrow{\text{Evaluate}} V^*$$

Step 1: Policy Evaluation — “How Good Is My Current Policy?”

Given a policy π , we compute $V^\pi(s)$ for all states s . Recall the Bellman expectation equation:

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) [r + \gamma V^\pi(s')]$$

For a finite MDP with $|\mathcal{S}|$ states, this gives $|\mathcal{S}|$ linear equations in $|\mathcal{S}|$ unknowns. We could solve directly, but the iterative approach is simpler and more general. We start with $V_0(s) = 0$ for all s and repeatedly apply:

Iterative Policy Evaluation Update

$$V_{k+1}(s) \leftarrow \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma V_k(s')] \quad \text{for all } s \in \mathcal{S}$$

We sweep through all states, updating values based on V_k . This converges to V^π as $k \rightarrow \infty$. In practice, we stop when $\|V_{k+1} - V_k\|_\infty < \theta$ for some small threshold θ .

Step 2: Policy Improvement — “Can We Do Better?”

Once we have V^π , we ask: can we improve the policy? For each state s , we look one step ahead and consider the return of every possible action a , then following π thereafter. This is exactly $Q^\pi(s, a)$:

$$Q^\pi(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma V^\pi(s')]$$

If we find an action a where $Q^\pi(s, a) > V^\pi(s)$, then taking a once and then following π is *better* than just following π . The **Policy Improvement Theorem** guarantees that the new greedy policy π' will be at least as good as π :

Greedy Policy Improvement

$$\pi'(s) \leftarrow \arg \max_a Q^\pi(s, a) = \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V^\pi(s')]$$

When does it stop? If $V^{\pi'} = V^{\pi}$, then the greedy policy already satisfies the Bellman *optimality* equation — meaning $\pi' = \pi^*$ and $V^{\pi} = V^*$. If $V^{\pi'} \neq V^{\pi}$, then π' is strictly better, and we loop back to evaluate it.

★ **Aha Moment: Policy Iteration Always Converges — And Fast**

Here is what's remarkable: in a finite MDP, policy iteration must converge in a finite number of steps. Why? There are only finitely many deterministic policies ($|\mathcal{A}|^{|\mathcal{S}|}$), each improvement step produces a strictly better policy (or terminates), and you can never revisit a previous policy. In practice, convergence is often shockingly fast — for a 100-state MDP, policy iteration typically converges in under 10 iterations, far fewer than the $|\mathcal{A}|^{100}$ possible policies.

The Policy Iteration Algorithm

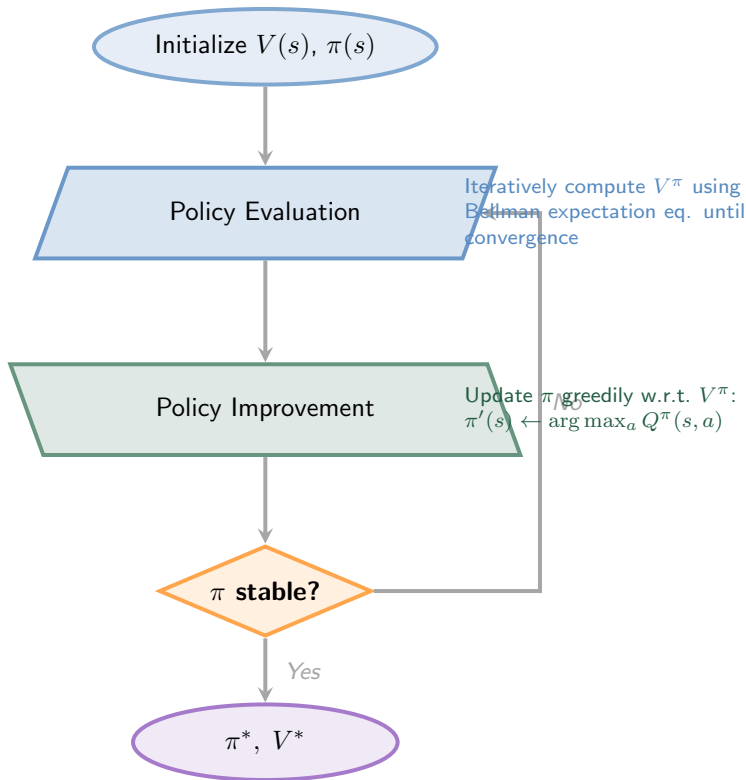


Figure 15: Policy Iteration flowchart. The cycle of Policy Evaluation and Policy Improvement continues until the policy no longer changes after the improvement step.

Algorithm: Policy Iteration

1. Initialization:

Choose arbitrary $\pi(s)$ for all $s \in \mathcal{S}$

Set $V(s) = 0$ for all $s \in \mathcal{S}$

2. Policy Evaluation (repeat until convergence):

$\Delta \leftarrow 0$

For each state $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

If $\Delta < \theta$: exit evaluation loop

3. Policy Improvement:

policy_stable $\leftarrow$ true

For each state $s \in \mathcal{S}$:

old_action $\leftarrow \pi(s)$

$\pi(s) \leftarrow \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$

If old_action $\neq \pi(s)$: policy_stable $\leftarrow$ false

Worked Example: Policy Iteration on the Recycling Robot

Iteration 0: Start with random policy π_0 : high $\rightarrow$ wait, low $\rightarrow$ wait.

Policy Evaluation ($\gamma = 0.9$, $\theta = 0.01$):

With this “always wait” policy, the robot earns +1 every step regardless of state.

$$V^{\pi_0}(\text{high}) = \frac{1}{1-0.9} = 10.0, \quad V^{\pi_0}(\text{low}) = 10.0$$

Policy Improvement: For state high:

$$Q^{\pi_0}(\text{high}, \text{wait}) = 1 + 0.9 \times 10.0 = 10.0$$

$$Q^{\pi_0}(\text{high}, \text{search}) = 4 + 0.9[0.7 \times 10.0 + 0.3 \times 10.0] = 4 + 9.0 = 13.0$$

Since $13.0 > 10.0$, the improved policy switches to $\pi_1(\text{high}) = \text{search}$.

Similarly for low: search gives $-3 + 9.0 = 6.0$ while wait gives 10.0 and recharge gives $0 + 9.0 = 9.0$. So $\pi_1(\text{low}) = \text{wait}$ stays.

Wait — but recharge (9.0) is also close. After another round of evaluation and improvement, the policy will converge to high $\rightarrow$ search, low $\rightarrow$ recharge — the optimal policy.

Convergence Guarantee

Policy Iteration is guaranteed to converge to π^* in a **finite number of iterations** for any finite MDP. Why? There are only finitely many deterministic policies. Each improvement step produces a strictly better policy (unless already optimal). Since V^π is bounded, the process must terminate.

The main drawback: Policy evaluation can be computationally expensive, requiring many sweeps through the entire state space. Furthermore, it critically depends on having the complete MDP model $p(s', r \mid s, a)$, which is often unavailable in practice. This limitation motivates the model-free methods we will explore in subsequent chapters.

◆ Code: Policy Iteration — Complete Implementation

```
def policy_iteration(P, gamma=0.9, theta=1e-6):
    states = list(P.keys())
    # 1. Initialize
    policy = {s: list(P[s].keys())[0]
               for s in states}
    V = {s: 0.0 for s in states}

    while True:
        # 2. Policy Evaluation
        while True:
            delta = 0
            for s in states:
                v = V[s]
                a = policy[s]
                V[s] = sum(
                    p*(r + gamma*V[ns])
                    for p, ns, r in P[s][a])
                delta = max(delta, abs(v-V[s]))
            if delta < theta:
                break

        # 3. Policy Improvement
        stable = True
        for s in states:
            old = policy[s]
            policy[s] = max(
                P[s],
                key=lambda a: sum(
                    p*(r+gamma*V[ns])
                    for p,ns,r in P[s][a]))
            if old != policy[s]:
                stable = False

        # 4. Check convergence
        if stable:
            break
    return policy, V

pi, V = policy_iteration(P)
print(f"Optimal: {pi}")
print(f"V(high)={V['high']:.4f}")
```

```
print(f"V(low) = {V['low']:.4f}")
```

↪ Bridge to RLHF: RLHF Is Policy Iteration

The evaluate–improve loop you just learned is *exactly* what happens in RLHF. **Policy Evaluation:** A reward model scores how well the current language model's responses align with human preferences — this evaluates V^π . **Policy Improvement:** PPO then updates the model's weights to produce higher-scoring responses — this improves π . Repeat. The only differences are scale (billions of parameters vs. a handful of states) and that we use sampled trajectories instead of full sweeps. But the *logic* is identical: evaluate, improve, converge. You now understand the skeleton of how every major AI lab aligns their models.

★ Real-World Spotlight:

Amazon's Warehouse Robots Use Policy Iteration Amazon's Kiva warehouse robots solve a real MDP: navigate a grid warehouse to pick and deliver items. The **states** are robot positions and item locations, **actions** are directional moves, and **rewards** combine delivery speed minus collision penalties. Because the warehouse layout is fully known (a model is available), Amazon uses DP-based planning to compute optimal routes. For small warehouses, this is essentially policy iteration. For larger ones, they use hierarchical decomposition — breaking the MDP into sub-problems, each small enough for DP. The same evaluate-then-improve logic you just learned powers millions of deliveries per day.

Dynamic Programming: Value Iteration

Policy Iteration works beautifully, but it has an annoying inefficiency: it fully evaluates the current policy (running many sweeps) before improving it. But wait — do you *really* need to know the exact value of every state before you can start improving? What if you just did one sweep of evaluation and immediately improved? It turns out this works. **Value Iteration** (VI) takes this idea to its logical extreme: it combines evaluation and improvement into a single update, finding V^* directly.

➡ Think Like the Agent: Skip the Middleman

Policy Iteration is like a restaurant reviewer who (1) eats the entire menu to rate each dish, then (2) recommends the best one, then (3) repeats with a revised menu. Value Iteration is like a reviewer who, after each bite, immediately updates the rating of the best dish so far. It combines tasting and rating into a single step — faster, but still finds the best dish.

The Core Idea: Direct Bellman Optimality Updates

Recall the Bellman Optimality Equation for $V^*(s)$:

$$V^*(s) = \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma V^*(s')]$$

This equation states that the optimal value of a state s is the expected return achieved by taking the *best possible* action a from that state, and then following the optimal policy thereafter.

Value Iteration uses this equation directly as an **update rule**. It starts with an initial guess for the value function, $V_0(s)$, for all states (often initialized to zero). Then, it iteratively refines this estimate by applying the Bellman optimality update across all states until the value function converges.

The Value Iteration Algorithm

Algorithm: Value Iteration

1. Initialization:

Choose arbitrary $V_0(s)$ for all $s \in \mathcal{S}$

Set tolerance threshold $\theta > 0$

2. Iteration Loop: Repeat for $k = 0, 1, 2, \dots$

$\Delta \leftarrow 0$

For each state $s \in \mathcal{S}$:

$v \leftarrow V_k(s)$

$V_{k+1}(s) \leftarrow \max_a \sum_{s', r} p(s', r | s, a)[r + \gamma V_k(s')]$

$\Delta \leftarrow \max(\Delta, |v - V_{k+1}(s)|)$

Convergence Check: If $\Delta < \theta$, stop

3. Extract Policy:

$\pi^*(s) \leftarrow \arg \max_a \sum_{s', r} p(s', r | s, a)[r + \gamma V^*(s')]$

Each iteration involves a sweep through the state space, updating each state's value based on the current values of its potential successor states, using the **maximization over actions** from the Bellman Optimality Equation. This update is sometimes called a “Bellman backup” or “optimality backup.”

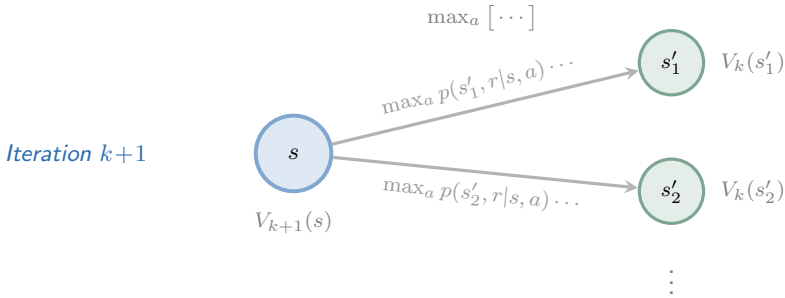


Figure 16: Value Iteration backup: $V_{k+1}(s)$ is calculated based on the maximum expected future rewards obtainable from state s , using the values $V_k(s')$ from the previous iteration for successor states s' .

Convergence and Finding the Optimal Policy

Why does this work? The Bellman optimality operator used in the update step is a **contraction mapping**. This mathematical property guarantees that applying the update repeatedly will cause V_k to converge to a unique fixed point, which is precisely V^* . We won't go into the

formal proof here, but the intuition is that each iteration brings the estimated values closer to the true optimal values.

Once Value Iteration converges (i.e., Δ becomes very small), we have a good approximation of V^* . We then extract the optimal policy π^* from V^* by choosing the action that maximizes the expected return from each state, looking one step ahead using the converged V^* :

Extracting π^* from V^*

$$\pi^*(s) = \arg \max_a \sum_{s',r} p(s',r \mid s,a) [r + \gamma V^*(s')]$$

This step computes the expected value for each action using the final converged V^* and selects the action yielding the highest value.

Note that the policy extraction step is performed **only once** after the value function has converged — unlike Policy Iteration, which repeatedly alternates between evaluation and improvement.

Comparison with Policy Iteration

	Policy Iteration	Value Iteration
Strategy	Alternates full Policy Evaluation (V^π) and Policy Improvement	Directly iterates toward V^* using Bellman optimality
Inner loop	Multiple sweeps for evaluation in each iteration	One sweep applying optimality update per iteration
Policy	Explicit policy π maintained and improved	Policy extracted once at the end
Speed	Fewer outer iterations, but each is expensive	More outer iterations, but each is cheaper

In practice, Value Iteration often converges faster than Policy Iteration, especially when the number of iterations needed for policy evaluation is high.

ation is large. However, each VI iteration requires calculating the maximum over actions, which can be computationally intensive if $|\mathcal{A}|$ is large.

The Bigger Picture

Like Policy Iteration, Value Iteration falls under the umbrella of **Dynamic Programming** methods. It finds the optimal solution but relies heavily on having a complete and accurate model of the environment's dynamics $p(s', r \mid s, a)$. This requirement is a significant limitation, which we'll address when we move to **model-free** learning methods in the upcoming chapters.

Worked Example: Value Iteration on the Recycling Robot

Starting with $V_0(\text{high}) = 0$, $V_0(\text{low}) = 0$, $\gamma = 0.9$:

Iteration 1 — For state high:

wait: $1 + 0.9 \times 0 = 1.0$

search: $0.7(4 + 0) + 0.3(4 + 0) = 4.0$

$V_1(\text{high}) = \max(1.0, 4.0) = 4.0$

For state low: $V_1(\text{low}) = \max(\text{wait} : 1.0, \text{search} : -0.6, \text{recharge} : 0) = 1.0$

Iteration 2 — For state high:

wait: $1 + 0.9 \times 4.0 = 4.6$

search: $0.7(4 + 0.9 \times 4.0) + 0.3(4 + 0.9 \times 1.0) = 0.7(7.6) + 0.3(4.9) = 6.79$

$V_2(\text{high}) = 6.79$

Values keep growing and converge: $V^*(\text{high}) \approx 11.26$, $V^*(\text{low}) \approx 10.13$.

◆ Code: Value Iteration — Complete Implementation

```
def value_iteration(P, gamma=0.9, theta=1e-6):
    """Find V* using Bellman optimality eq."""
    states = list(P.keys())
    V = {s: 0.0 for s in states}
```

```

while True:
    delta = 0
    for s in states:
        v = V[s]
        # Bellman optimality backup
        V[s] = max(
            sum(p*(r + gamma*V[ns])
                for p, ns, r in P[s][a])
            for a in P[s]
        )
        delta = max(delta, abs(v - V[s]))
    if delta < theta:
        break
return V

def extract_policy(P, V, gamma=0.9):
    """Extract pi* from converged V*."""
    return {s: max(P[s],
        key=lambda a: sum(
            p*(r+gamma*V[ns])
            for p,ns,r in P[s][a]))
        for s in P}

V_star = value_iteration(P)
pi_star = extract_policy(P, V_star)
print(f"V*(high)={V_star['high']:.4f}")
print(f"V*(low) = {V_star['low']:.4f}")
print(f"pi* = {pi_star}")

```

★ Aha Moment: Value Iteration = One Line of Genius

Policy Iteration needs two nested loops: one to evaluate, one to improve. Value Iteration collapses both into a single equation: $V(s) \leftarrow \max_a \sum p(\cdot) [r + \gamma V(s')]$. That max does the improvement; the update does the evaluation. One line, one loop, optimal policy. This elegance is why Value Iteration is the default DP algorithm in practice — and why the Bellman optimality equation is arguably the most important equation in all of RL.

Limitations of Dynamic Programming

Let's be honest about what we've built. Policy Iteration and Value Iteration are *guaranteed* to find the optimal policy for any finite MDP. That's a remarkable theoretical result. But here's the uncomfortable truth: for almost every problem you actually care about, these methods are useless. Not because the math is wrong — the math is perfect. The problem is that DP makes two demands that reality refuses to meet.

The Requirement of a Full Model

First: DP needs a **complete and accurate model** of the environment. Every single transition probability $p(s', r \mid s, a)$ must be known in advance. Think about what that means — you need to know, for every possible situation the agent could be in and every action it could take, exactly what would happen and with what probability. For a board game with fixed rules, sure. For a robot walking through a crowded city? Forget it.

➡ Think Like the Agent: Planning vs. Learning

*DP is a tool for **planning** when you have a perfect map, not a tool for **learning** when you need to explore and discover the map yourself. DeliBot can use DP to optimize routes if it has a complete city map with traffic probabilities — but what if it's dropped in an unknown city?*

In many real-world situations, a perfect model is simply unrealistic:

★ Real-World Spotlight:

Google's Chip Design — Where DP Couldn't Scale In 2021, Google used RL to design the physical layout of computer chips (published in *Nature*). The state space — billions of possible placements of thousands of components on a chip — made tabular DP utterly impossible. No human could write down the transition model, and the space was too large to enumerate. Instead, Google used deep RL with neural networks to approximate value functions. The RL system designed chip layouts in hours that human engineers took months to create. This is exactly the limi-

tation DP faces: real problems are too large and too unknown for exact methods.

When DP Fails: Real-World Scenarios

Unknown Environments: A robot learning to navigate a new building, or an algorithm learning a video game without access to the source code. Transition probabilities aren't provided.

Modeling Complexity: Even when dynamics follow physical laws (robotics), creating an accurate simulation capturing noise and uncertainty is incredibly difficult and error-prone.

Non-Stationary Environments: The environment's dynamics may change over time. A model accurate yesterday may be wrong today, rendering DP calculations suboptimal.

Computational Scalability: The Curse of Dimensionality

Even with a perfect model, DP faces the **Curse of Dimensionality**. The core operations involve iterating through the *entire* state space $\mathcal{S}$ (and often the action space $\mathcal{A}$) to perform Bellman updates.

Resource	What Scales	Complexity
Memory	Storing $V(s)$ or $Q(s, a)$	$O(\mathcal{S})$ or $O(\mathcal{S} \mathcal{A})$
Model storage	Storing $p(s', r s, a)$	$O(\mathcal{S} ^2 \mathcal{A})$
Time per sweep	One full iteration	$O(\mathcal{S} ^2 \mathcal{A})$

Consider the implications for real problems:

X Common Misconception: DP Works for Any Problem If You're Patient Enough

For small, discrete MDPs (like our Recycling Robot with 2 states), DP is fast and exact. But for problems with large state spaces, the cost becomes prohibitive:

Chess: $|\mathcal{S}| \approx 10^{47}$ possible positions — storing $V(s)$ alone would

require more memory than atoms in the observable universe.

Go: $|\mathcal{S}| \approx 10^{170}$ — AlphaGo succeeded not through DP, but through deep neural networks and Monte Carlo Tree Search.

Continuous states: Robot joint angles, sensor readings — technically infinite state space. Basic DP simply cannot handle this. Discretizing leads to exponential blow-up.

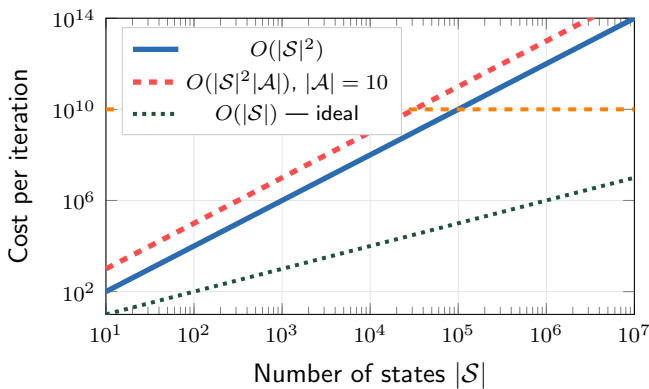


Figure 17: Computational cost per DP iteration scales rapidly with $|\mathcal{S}|$. The $O(|\mathcal{S}|^2|\mathcal{A}|)$ complexity makes DP impractical for large state spaces. Both axes use logarithmic scale.

Moving Past Dynamic Programming

These limitations highlight why DP, despite its theoretical importance in establishing the Bellman equations, is often not directly applicable to complex RL problems. We need methods that can:

↪ Bridge to RLHF: Why You Can't Use DP to Align a Language Model

Consider the RLHF problem through the lens of what you just learned. A language model with a 50,000-token vocabulary generating a 500-token response has $50,000^{500}$ possible trajectories — a number that dwarfs the atoms in the observable universe. We cannot enumerate states, we do not know the transition dynamics of human conversation, and the reward model is a neural network, not a lookup table. *Every single limitation* of DP listed above applies. This is precisely why the field needed to invent model-free methods (Monte Carlo, TD learning),

then function approximation (deep RL), then policy gradient methods (PPO) — each chapter ahead solves one more barrier on the road from Bellman's elegant theory to the practical reality of aligning language models with human values.

What Comes Next: The Path Forward

1. **Model-Free Methods** — Learn effective policies *without* requiring prior knowledge of $p(s', r \mid s, a)$:
 - **Monte Carlo Methods** (Chapter 4): Learn from complete episodes of experience
 - **Temporal-Difference Learning** (Chapter 5): Learn from incomplete episodes, step by step
2. **Function Approximation** — Handle large or continuous state/action spaces efficiently:
 - **Deep RL** (Chapter 6): Use neural networks to generalize value estimates across massive state spaces, breaking free from tabular representations
3. **Policy Gradient Methods and RLHF** (Chapter 7): Directly optimize policies for complex objectives, including aligning language models with human preferences

The big picture: DP gives us the *theory* (Bellman equations) and the *guarantee* (convergence to optimality). The chapters ahead give us the *practice* — methods that actually work in the messy, unknown, high-dimensional world of real RL problems. Every algorithm we study from here on is, in some sense, an approximation of what DP would do if we had unlimited computation and a perfect model.

Quick Reference:

Chapter 3 — Dynamic Programming
Bellman Equations:

- **Expectation:** $V^\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(\cdot) [r + \gamma V^\pi(s')]$
- **Optimality:** $V^*(s) = \max_a \sum_{s',r} p(\cdot) [r + \gamma V^*(s')]$

DP Algorithms:

- **Policy Evaluation:** Iteratively apply Bellman expectation until V^π converges
- **Policy Improvement:** $\pi'(s) = \arg \max_a Q^\pi(s, a)$
- **Policy Iteration:** Evaluate $\rightarrow$ Improve $\rightarrow$ repeat until stable
- **Value Iteration:** Apply Bellman optimality: $V(s) \leftarrow \max_a \sum p(\cdot) [r + \gamma V(s')]$

Key Facts:

- Both PI and VI converge to π^* for finite MDPs
- PI: fewer iterations, more computation per step
- VI: simpler, one equation per state per sweep
- **Limitation:** Both require full model $p(s', r|s, a)$

Coming next: Chapter 4 (Monte Carlo) removes the model requirement entirely.

Chapter 3 Quiz: Estimating Value Functions

Test your understanding of the Bellman equations, Dynamic Programming, and their limitations. Choose the best answer for each question.

1 Value Iteration and Policy Iteration are both dynamic programming methods for finding optimal solutions. What is a significant distinction in their iterative approaches?

- (a) Value Iteration is guaranteed to converge to the optimal solution, but Policy Iteration may oscillate without convergence.
- (b) Value Iteration combines evaluation and improvement steps implicitly within each single sweep, whereas Policy Iteration alternates between distinct, complete policy evaluation and policy improvement phases.
- (c) Policy Iteration directly converges to the optimal value function V^* , while Value Iteration converges to the optimal policy π^* .
- (d) Value Iteration can operate without a model of the environment, unlike Policy Iteration which requires one.

2 During a single iteration of the Value Iteration algorithm, what is the primary operation performed for each state s ?

- (a) Select the action that provides the highest immediate reward and update the policy based on this greedy choice.
- (b) Estimate the action-value function $Q(s, a)$ based on observed rewards from sampled transitions.
- (c) Update the state's value by applying the Bellman Optimality Equation, which involves maximizing the expected return over all possible actions using the current value function for successor states.
- (d) Evaluate the current policy by computing its value function $V^\pi(s)$ through multiple sweeps.

3 How does the Bellman expectation equation describe the interdependency between the state-value function $V^\pi(s)$ and the action-value function $Q^\pi(s, a)$?

- (a) $V^\pi(s)$ is the maximum $Q^\pi(s, a)$ over all actions, and $Q^\pi(s, a)$ only includes immediate rewards.
- (b) Both $V^\pi(s)$ and $Q^\pi(s, a)$ are only dependent on future rewards, not immediate ones.
- (c) $Q^\pi(s, a)$ can only be computed if $V^\pi(s)$ is already known, but $V^\pi(s)$ has no reliance on $Q^\pi(s, a)$.
- (d) $V^\pi(s)$ is the expected $Q^\pi(s, a)$ weighted by the policy π ; and $Q^\pi(s, a)$ is the expected immediate reward plus the discounted expected state value $\gamma V^\pi(s')$ for successor states.

4 An agent follows a policy π in a simple environment. From state s_1 , action a_1 is chosen with probability 1. This leads to state s_2 with reward 5 (probability 0.7) or to state s_3 with reward 1 (probability 0.3). Given a discount factor $\gamma = 0.9$, $V^\pi(s_2) = 12$, and $V^\pi(s_3) = 8$, what is the value of $V^\pi(s_1)$?

- (a) 13.5
- (b) 17.3
- (c) 14.6
- (d) 6.7

5 A reinforcement learning practitioner is faced with an environment that has millions of possible states. They need to calculate the value function for a known policy. Why would they opt for an iterative method like Policy Evaluation rather than attempting a direct solution using linear algebra?

- (a) The Bellman expectation equation is non-linear, preventing direct linear algebra.
- (b) Direct linear algebra solutions are computationally infeasible for large numbers of states due to their high complexity, typically $O(|\mathcal{S}|^3)$.
- (c) Iterative methods are always faster regardless of state space size.
- (d) Iterative methods can find an exact solution while direct methods only find approximations.

6 When applying Dynamic Programming to find the optimal policy and value function for a given MDP, which statement accurately describes a significant difference between Policy Iteration and Value Iteration?

- (a) Policy Iteration requires knowing the environment model, while Value Iteration does not.
- (b) Value Iteration directly computes the optimal policy π^* , while Policy Iteration only computes the optimal value function V^* .
- (c) Policy Iteration explicitly separates the steps of evaluating a policy and then improving it, whereas Value Iteration combines these aspects into a single update rule that directly aims for the optimal value function.
- (d) Policy Iteration can only be used for discrete state spaces, while Value Iteration handles continuous state spaces.

7 A reinforcement learning agent is using Policy Iteration to find an optimal policy. After completing a Policy Evaluation step for the current policy π , the subsequent Policy Improvement step finds that the newly generated policy π' is identical to π . What is the immediate implication and next action the algorithm should take?

- (a) The Policy Evaluation step needs to be repeated with a smaller convergence threshold.
- (b) The algorithm has converged to a local optimum and needs to re-initialize with a different arbitrary policy.
- (c) The Policy Improvement step was insufficient, and the algorithm should immediately re-attempt policy improvement without re-evaluating.
- (d) The optimal policy has been found, and the algorithm should terminate.

8 You are developing an autonomous drone and want to use Dynamic Programming to find the optimal flight policy, but you lack precise aerodynamic models and environmental conditions, making it impossible to calculate exact transition probabilities $p(s', r \mid s, a)$. Which of the following is the most significant challenge for directly applying DP?

- (a) Policy Iteration will converge more slowly in non-deterministic environments, making it impractical for real-time drone control.
- (b) Policy Iteration cannot be directly applied because it fundamentally requires a complete model of the environment's transition probabilities and rewards.
- (c) The computational cost of Policy Evaluation would be prohibitively high due to the continuous state and action spaces.
- (d) The algorithm would require learning the model through interaction with the environment, which is impractical for safety-critical drone applications.

9 In Reinforcement Learning, Dynamic Programming methods are distinguished as planning tools rather than learning tools. Which of the following statements best describes the meaning of this distinction?

- (a) DP methods are only suitable for deterministic environments, while learning methods can handle stochasticity.
- (b) DP methods inherently require less memory and computational power than learning methods.
- (c) Planning involves real-time decision-making, while learning is an offline process.
- (d) DP methods assume perfect knowledge of environment dynamics to compute optimal actions, whereas learning methods discover these dynamics through experience.

10 A company wants to use Dynamic Programming to determine the optimal strategy for a complex industrial robot arm. The robot arm has many joints, each with continuous angles and velocities, resulting in an extremely high-dimensional state space. Even if a perfect simulation model of the robot's physics is available, why would a basic Dynamic Programming approach still be impractical?

- (a) The algorithms cannot handle the continuous action space of robot joint movements.
- (b) Dynamic Programming struggles with the non-stationary dynamics inherent in mechanical systems.
- (c) The continuous and high-dimensional state space leads to an astronomical number of discrete states upon discretization, making memory and computation time prohibitive.
- (d) It is inherently designed for problems with discrete, small action spaces only.

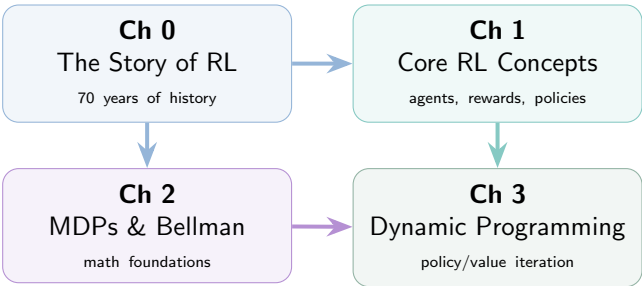
Chapter 3 Quiz — Answer Key

Q	Ans	Explanation
1	(b)	Value Iteration applies the Bellman optimality equation in a single sweep (combining evaluation and improvement implicitly). Policy Iteration alternates between a full policy evaluation phase and a separate improvement phase.
2	(c)	Each state's value is updated using: $V_{k+1}(s) \leftarrow \max_a \sum_{s',r} p(\cdot) [r + \gamma V_k(s')]$. This is the Bellman optimality backup — maximizing over actions using current values for successor states.
3	(d)	$V^\pi(s) = \sum_a \pi(a s) Q^\pi(s, a)$ (policy-weighted average of Q-values), and $Q^\pi(s, a) = \sum_{s',r} p(\cdot) [r + \gamma V^\pi(s')]$ (immediate reward + discounted next-state value). They form a “back-and-forth” relationship.
4	(a)	$V^\pi(s_1) = 0.7(5 + 0.9 \times 12) + 0.3(1 + 0.9 \times 8) = 0.7(15.8) + 0.3(8.2) = 11.06 + 2.46 = 13.52 \approx 13.5$. Apply the Bellman equation: sum over transitions of $p \cdot [r + \gamma V^\pi(s')]$.
5	(b)	For a given policy, the Bellman expectation equation is linear, so (a) is wrong. But solving via matrix inversion is $O(S ^3)$, which is infeasible for millions of states. Iterative methods scale much better per sweep.
6	(c)	Both require a model. The key difference: Policy Iteration has explicit evaluate $\rightarrow$ improve phases; Value Iteration combines them into one update using the Bellman optimality equation directly.
7	(d)	When $\pi' = \pi$ after improvement, the policy satisfies the Bellman optimality equation: $\pi(s) = \arg \max_a Q^\pi(s, a)$ already. This means $\pi = \pi^*$ and $V^\pi = V^*$. The algorithm terminates.
8	(b)	DP fundamentally requires knowing $p(s', r s, a)$ for all states and actions. Without this model, the Bellman equations cannot be evaluated. This is the most significant barrier.
9	(d)	“Planning” means computing optimal behavior from a known model. “Learning” means discovering optimal behavior through trial-and-error interaction. DP plans with a perfect map; model-free RL learns by exploring.
10	(c)	Even with a perfect model, continuous joint angles/velocities must be discretized for tabular DP. High-dimensional continuous spaces lead to an exponential explosion in discrete states (Curse of Dimensionality).

Part I Complete

The Bedrock — Chapters 0–3

What You Have Mastered



Chapter	Key Concepts	You Can Now...
Ch 0	RL history, breakthroughs, Atari, AlphaGo, RLHF	Tell the story of RL from 1950 to today
Ch 1	Agent, environment, state, action, reward, policy, return	Define any RL problem formally
Ch 2	MDP, Bellman equations, V^π , Q^π , V^* , Q^*	Write & solve Bellman equations by hand
Ch 3	Policy evaluation, policy iteration, value iteration	Implement DP algorithms in Python

Key Formulas at a Glance

Bellman Expectation (V):

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma V^\pi(s')]$$

Bellman Optimality (V):

$$V^*(s) = \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V^*(s')]$$

Return:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

The One Limitation

Dynamic Programming requires a complete model.

You need $p(s',r|s,a)$ for every state and action. In most real-world problems, **this model is unknown**.

Chapter 4 solves this.

Monte Carlo methods learn from *experience* — no model needed.

The journey continues in Chapter 4: Monte Carlo Methods

From planning with a model → learning from experience.